

Árvores

Estruturas de Dados

Prof. Dr. Franklin César Flores

Departamento de Informática

Universidade Estadual de Maringá

Árvores

- Este conjunto de slides foi utilizado durante as aulas das disciplinas:
- 6884 – Estruturas de Dados – Turma 02 (Ciência da Computação)
- 9893 – Estruturas de Dados – Turma 31 (Informática)
- Maringá, 2º semestre letivo de 2024

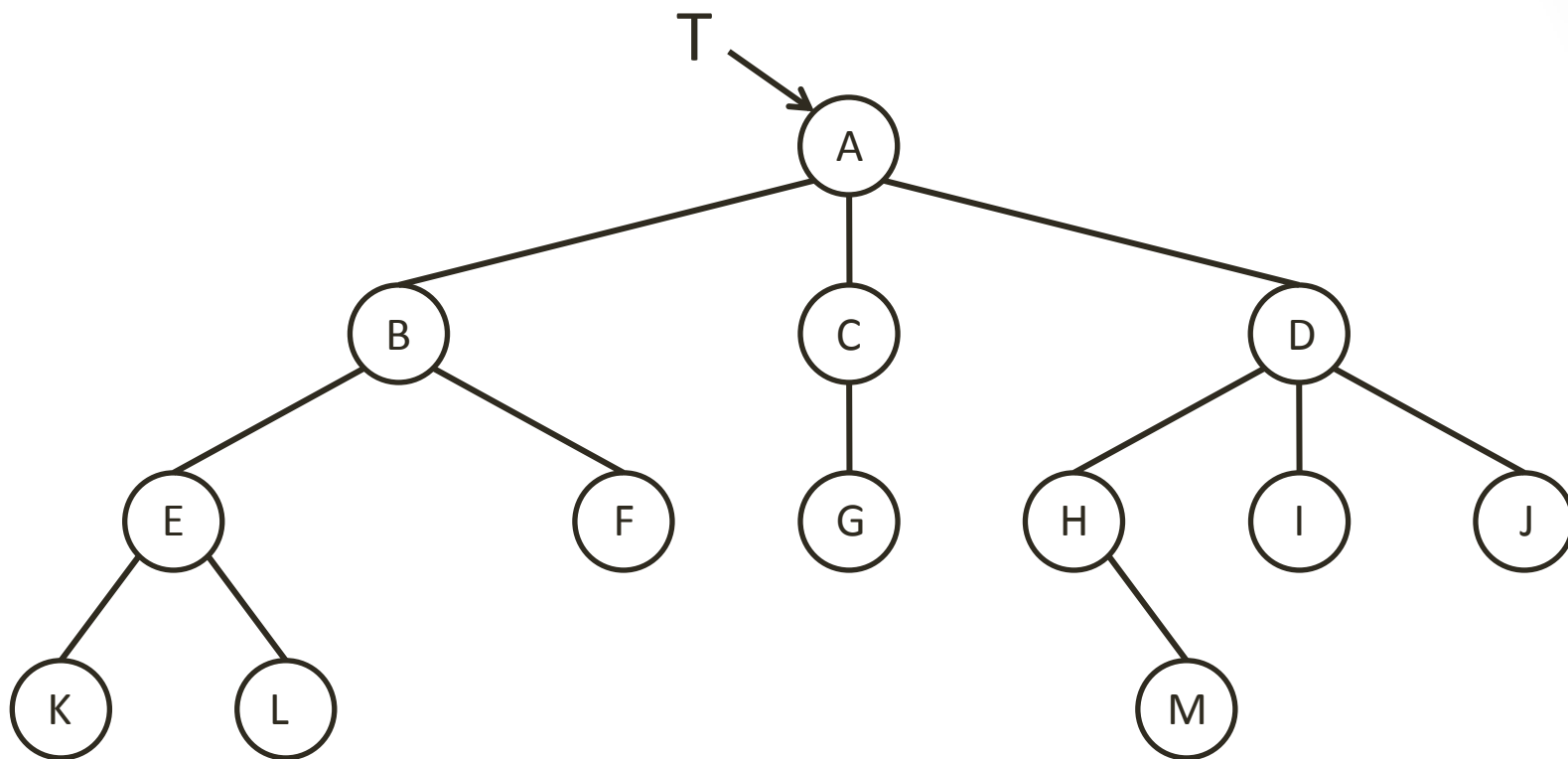
Roteiro

- Árvores
- Árvores binárias
- Árvores binárias de busca
- Árvores AVL
- Exercícios
- Bibliografia

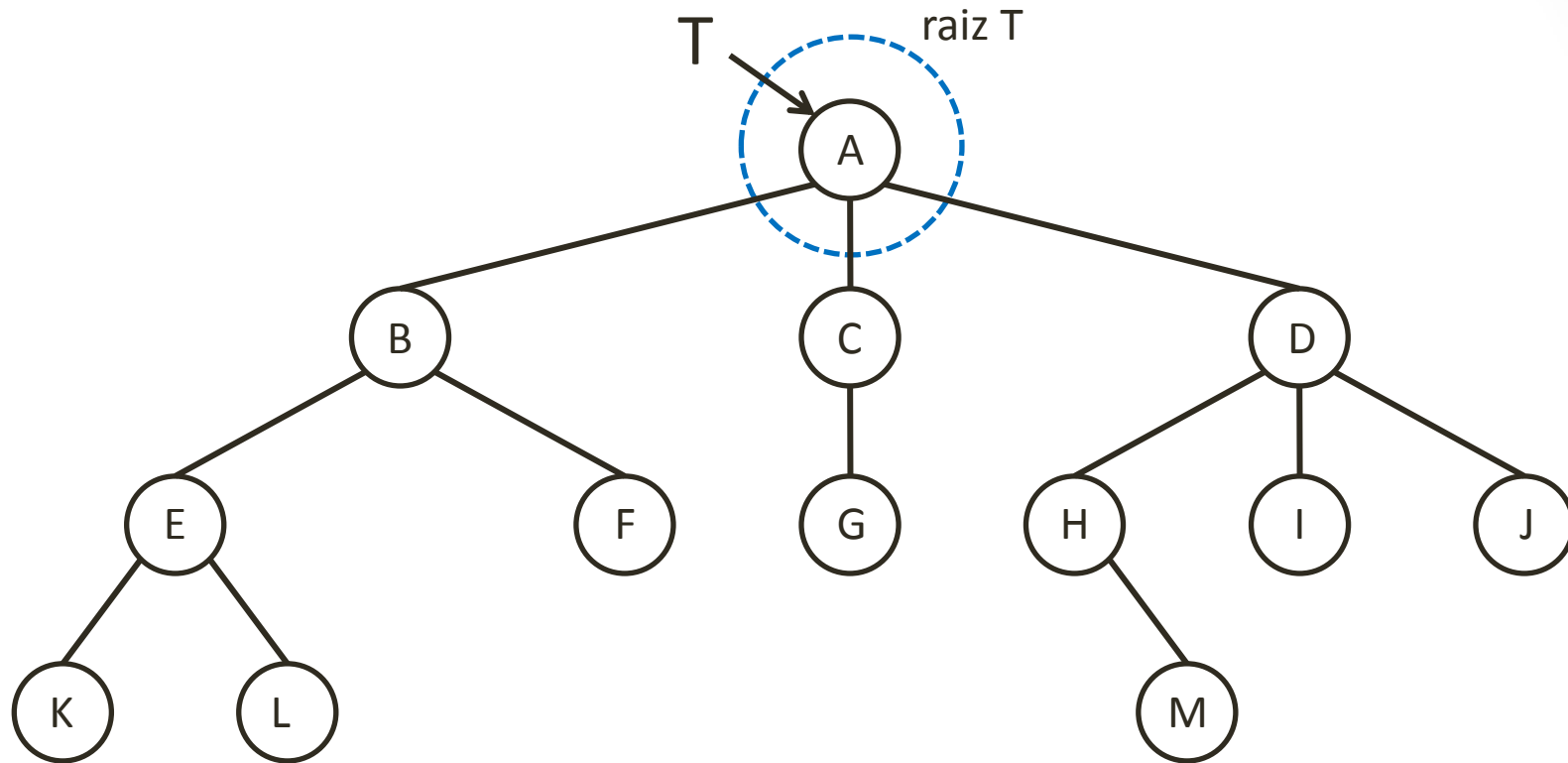
Árvores

- Um **árvore** é uma estrutura que:
- OU é vazia
- OU consiste de um conjunto finito não vazio T de nós tais que:
 - i. existe um nó especial: a raiz
 - ii. os nós restantes são particionados em m conjuntos disjuntos $T_1, T_2, T_3, \dots, T_m$, cada um dos quais é uma árvore, chamada de subárvore da raiz

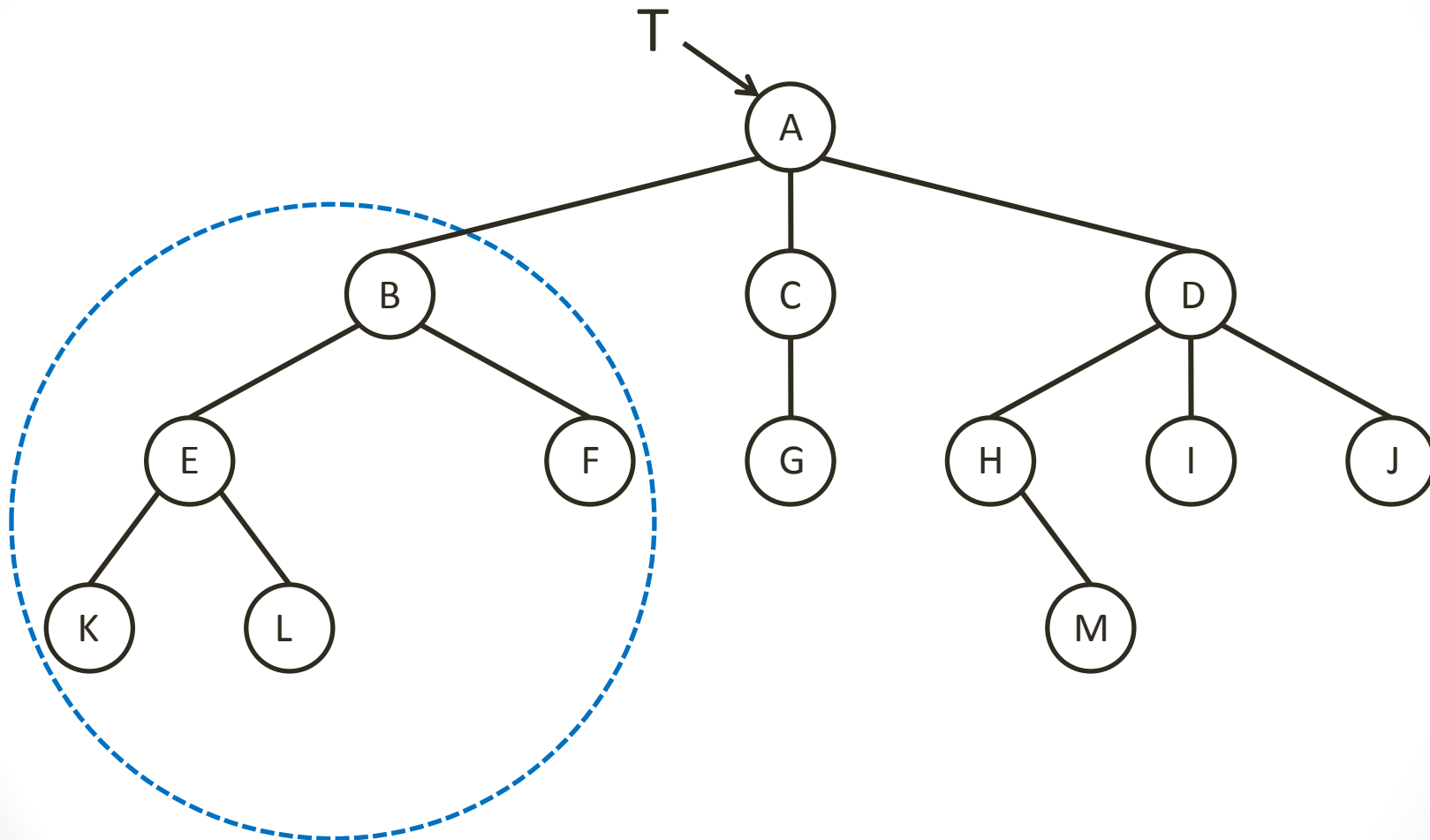
Árvores



Árvores

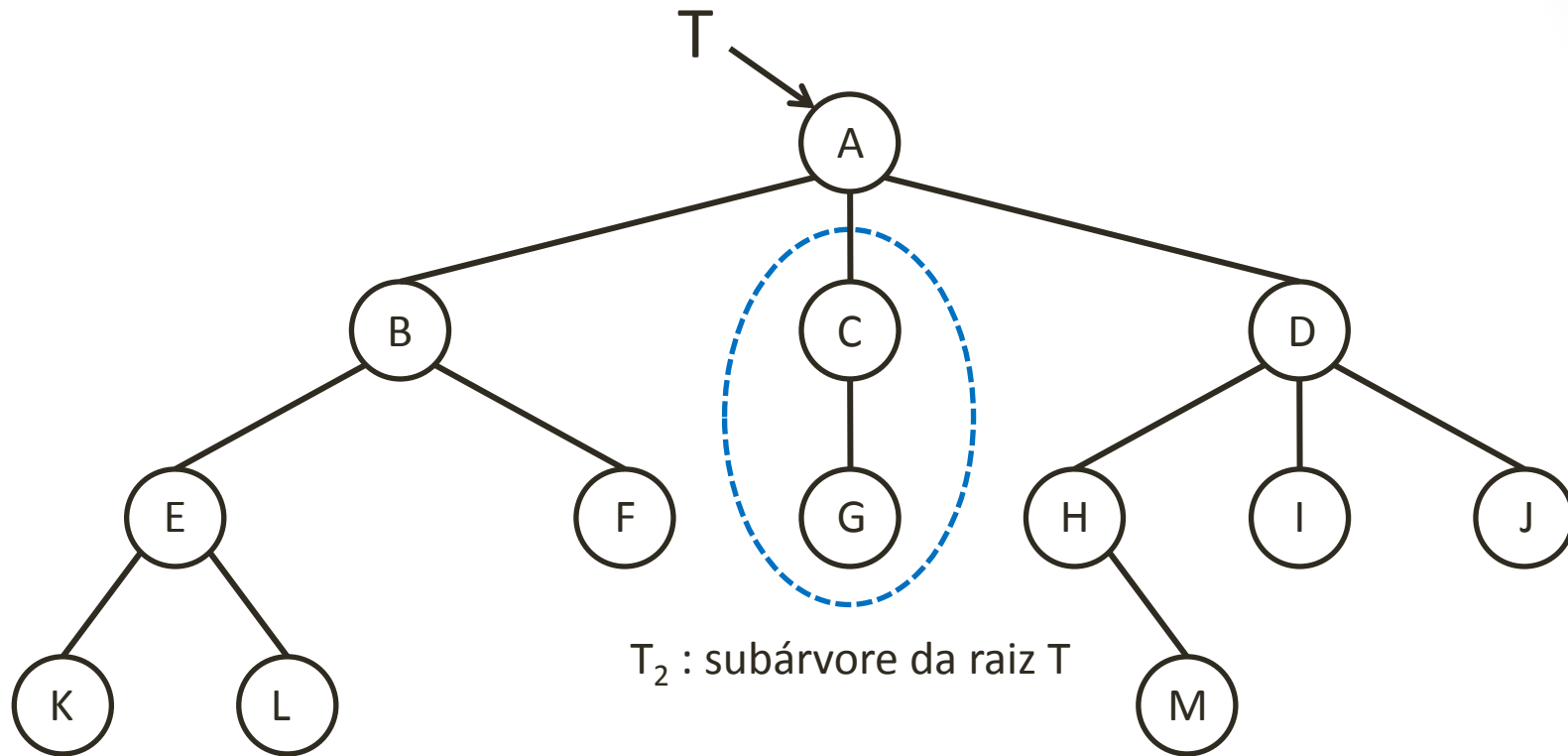


Árvores

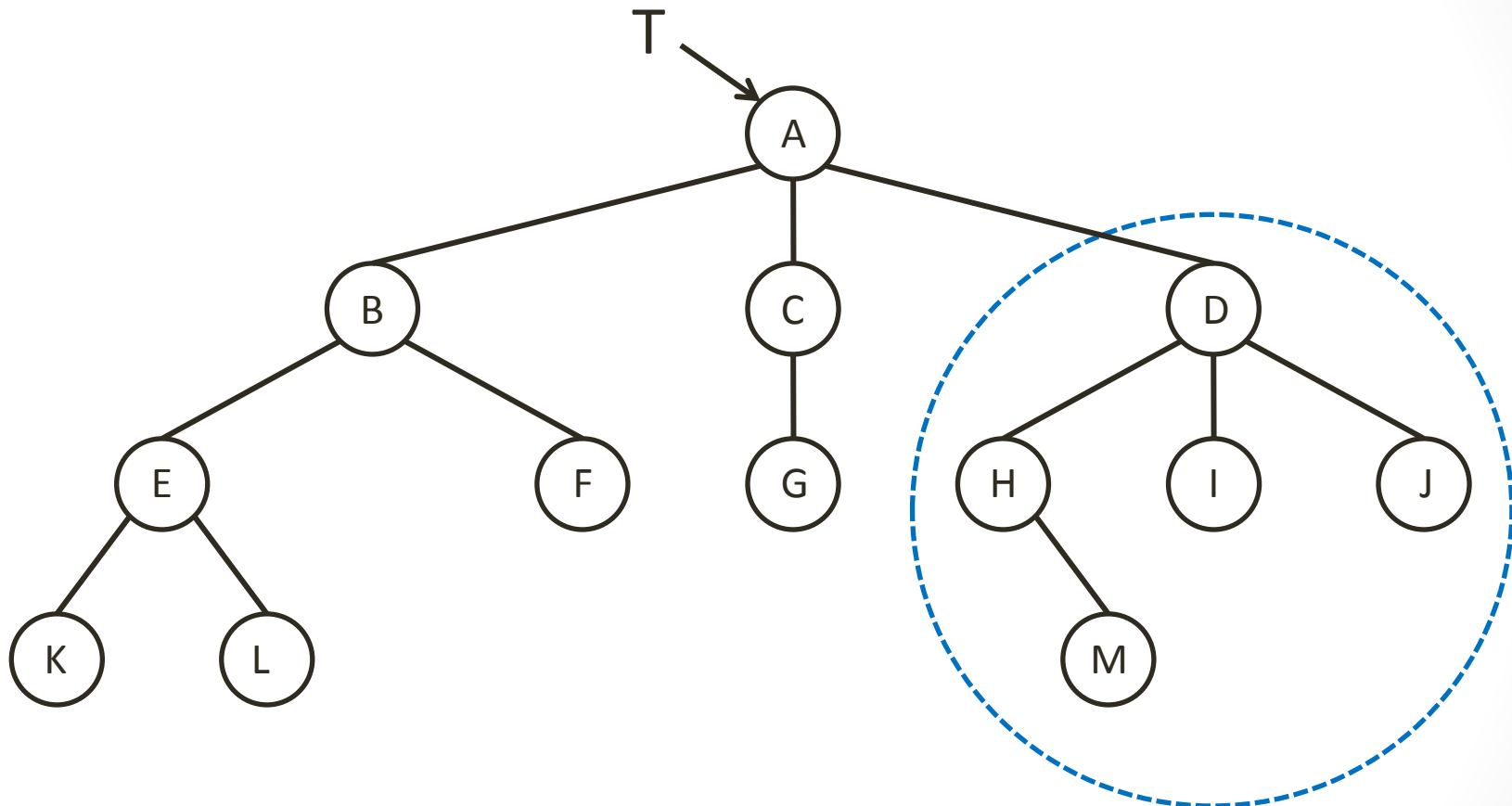


T_1 : subárvore da raiz T

Árvores



Árvores



T_3 : subárvore da raiz T

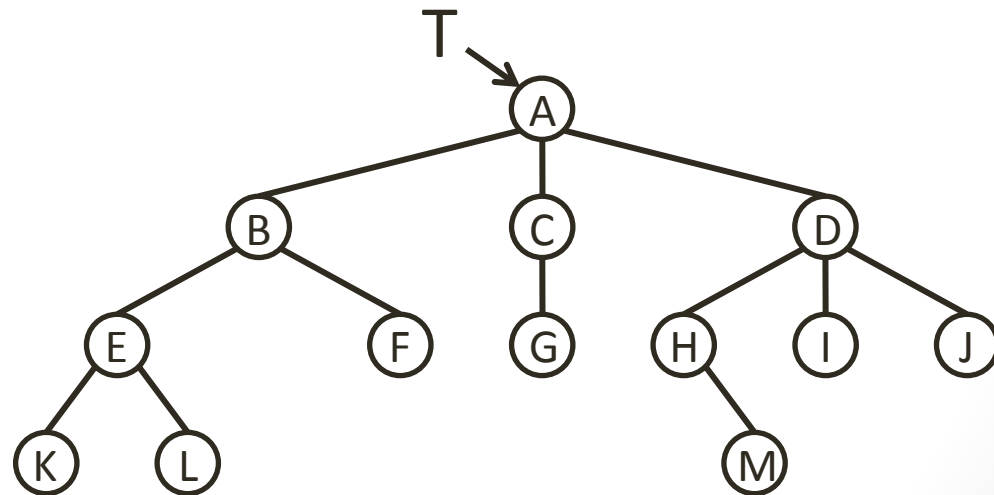
Árvores

- Algumas definições:

1. GRAU de um nó:

- É o número de subárvores deste nó

- $\text{grau}(A) =$
- $\text{grau}(H) =$
- $\text{grau}(E) =$
- $\text{grau}(I) =$



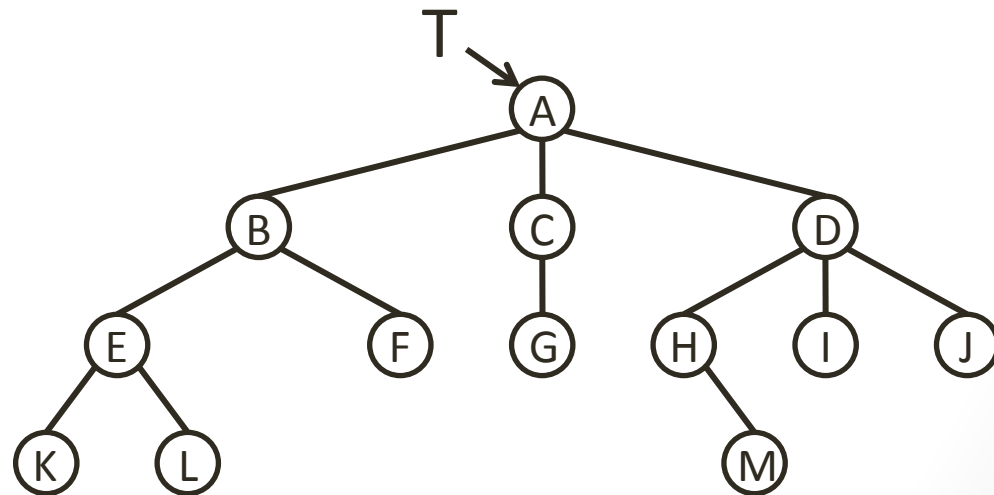
Árvores

- Algumas definições:

1. GRAU de um nó:

- É o número de subárvores deste nó

- $\text{grau}(A) = 3$
- $\text{grau}(H) = 1$
- $\text{grau}(E) = 2$
- $\text{grau}(I) = 0$

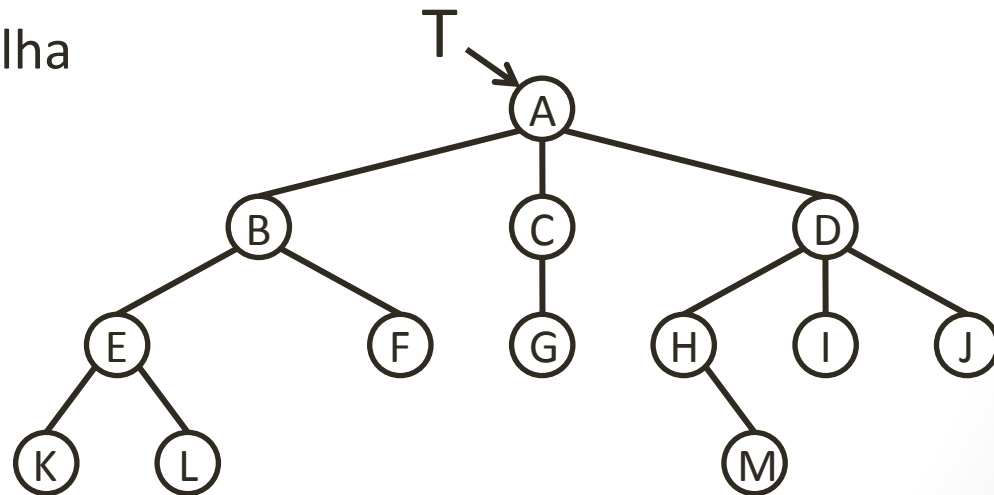


Árvores

- Algumas definições:

2. NÓ TERMINAL ou FOLHA:

- É um nó de grau zero
- Quem são os nós folha desta árvore?

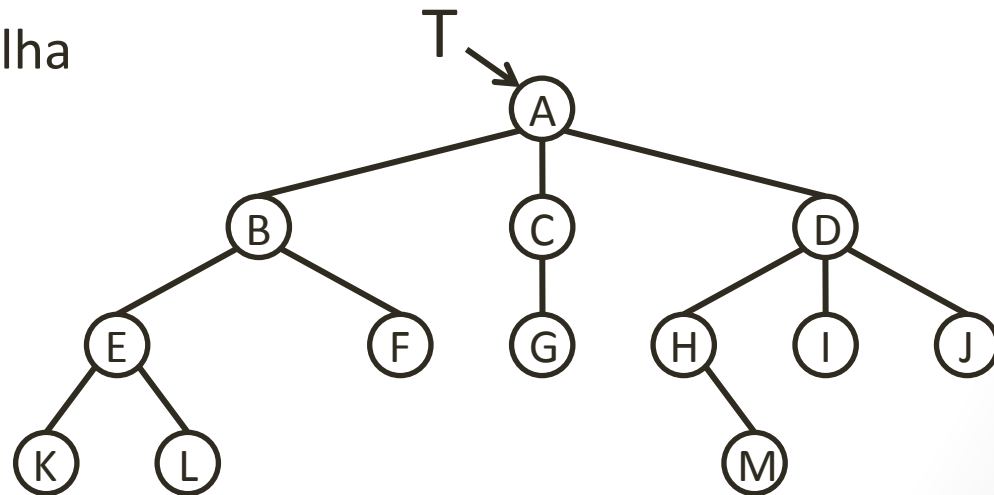


Árvores

- Algumas definições:

2. NÓ TERMINAL ou FOLHA:

- É um nó de grau zero
- Quem são os nós folha desta árvore?
- K , L , F , G , M , I , J



Árvores

- Algumas definições:

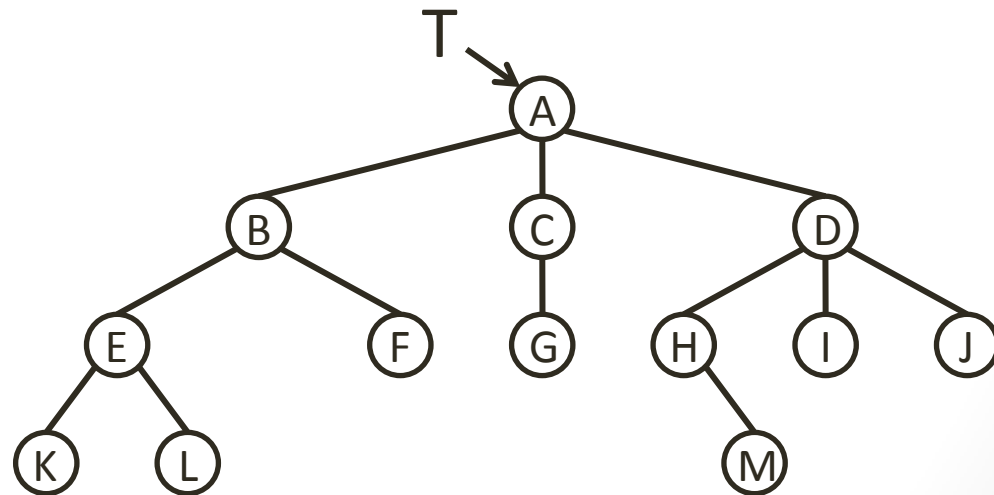
3. **FILHOS** de um nó:

- São as raízes de suas subárvores

- $\text{filhos}(D) =$

- $\text{filhos}(C) =$

- $\text{filhos}(I) =$



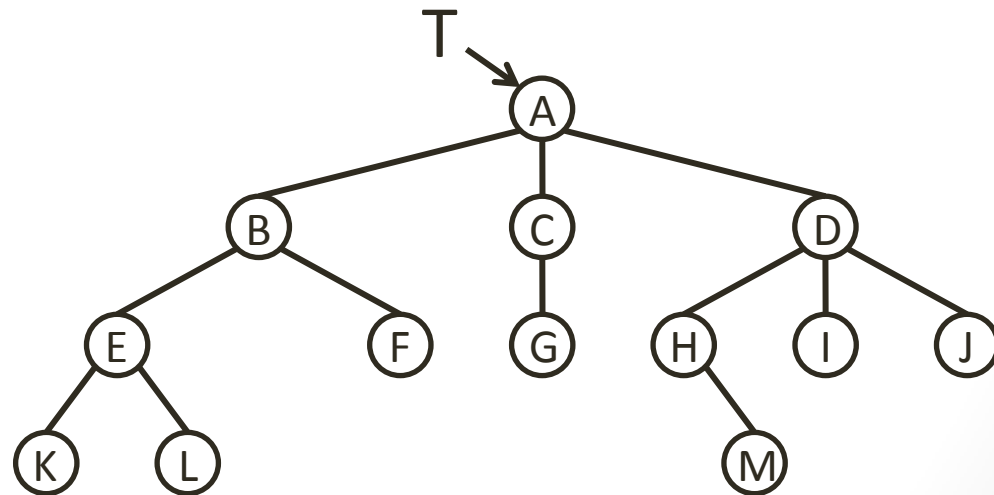
Árvores

- Algumas definições:

3. FILHOS de um nó:

- São as raízes de suas subárvores

- $\text{filhos}(D) = \{H, I, J\}$
- $\text{filhos}(C) = \{G\}$
- $\text{filhos}(I) = \emptyset$

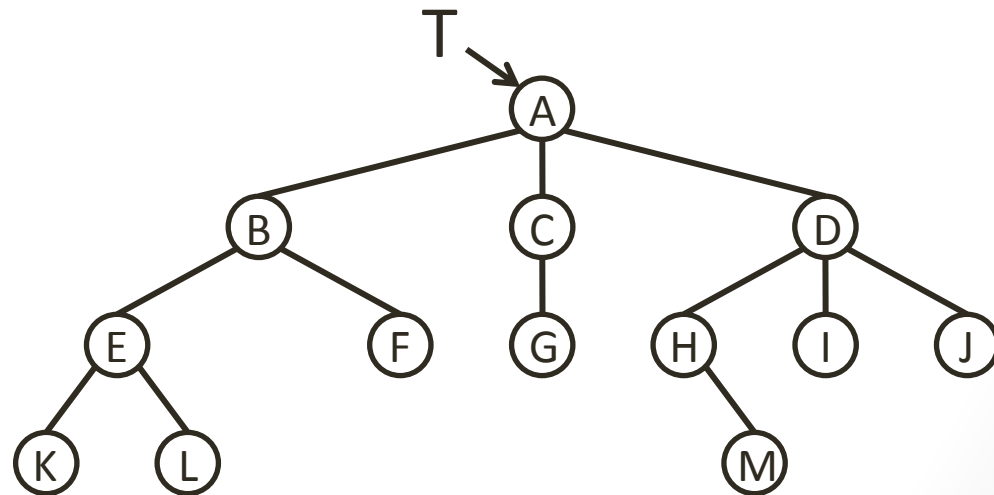


Árvores

- Algumas definições:

4. CAMINHO na árvore:

- É uma sequência $x_1, x_2, x_3, \dots, x_n$ de nós tal que, para cada i , x_i é filho de x_{i-1} ($1 < i \leq n$).



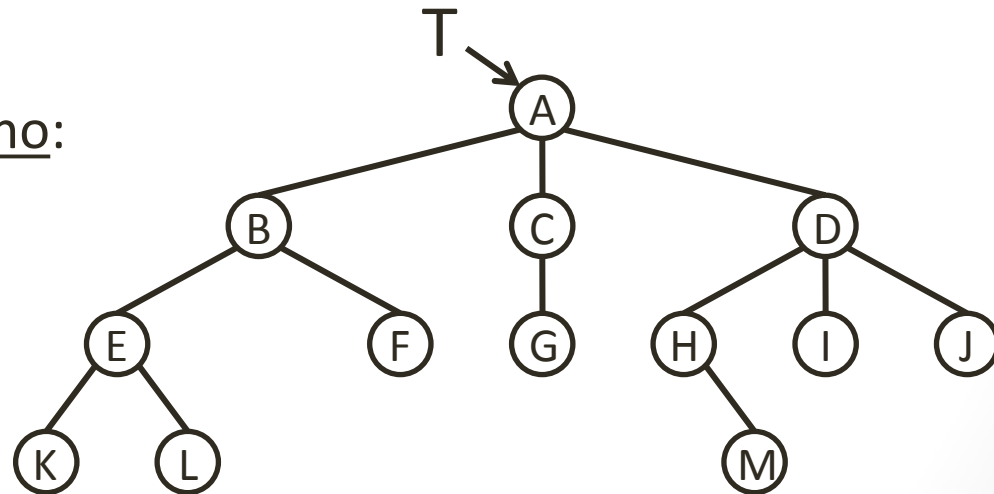
Árvores

- Algumas definições:

4. CAMINHO na árvore:

- É uma sequência $x_1, x_2, x_3, \dots, x_n$ de nós tal que, para cada i , x_i é pai de x_{i+1} ($1 \leq i < n$).

- exemplos de caminho:



Árvores

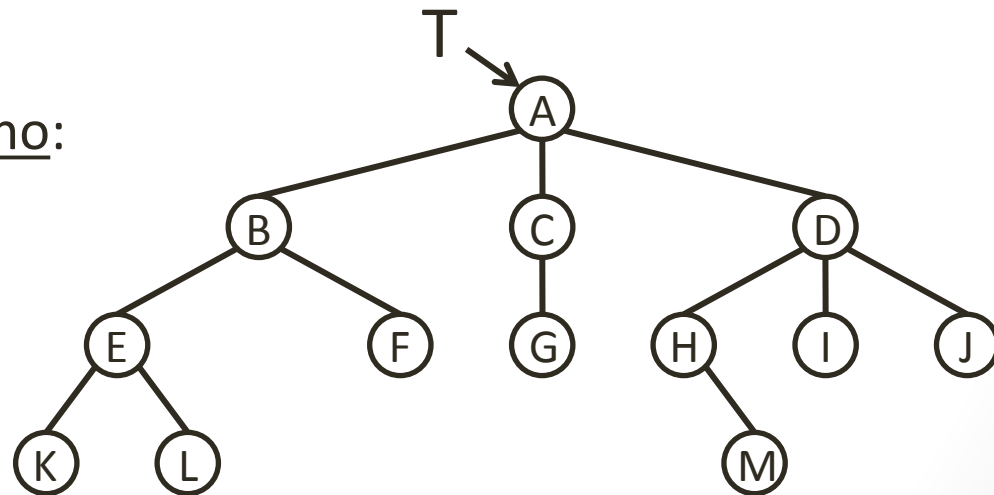
- Algumas definições:

4. CAMINHO na árvore:

- O comprimento do caminho $x_1, x_2, x_3, \dots, x_n$ é $n - 1$

- exemplos de caminho:

- A, B, E, K
- A, B, E, L
- A, C, G
- D, I
- F



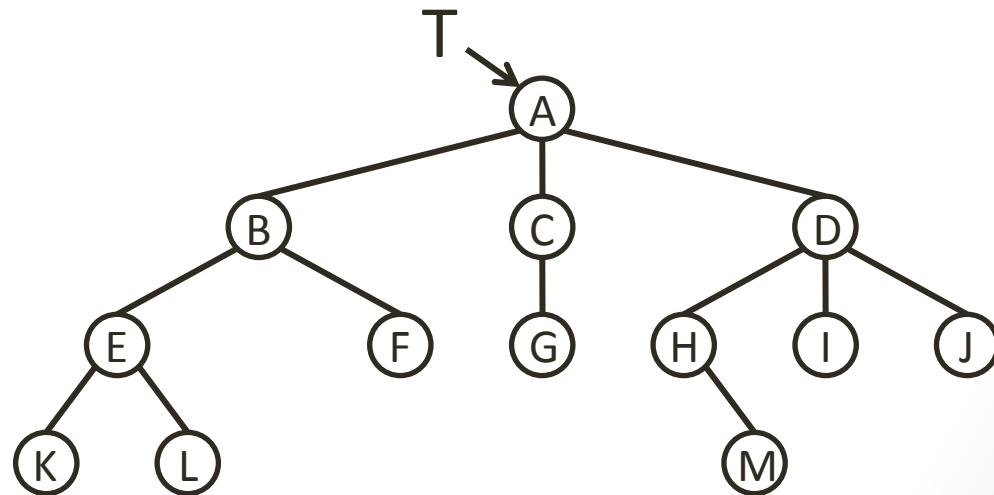
Árvores

- Algumas definições:

5. **ANCESTRAIS** (próprios) de um nó:

- São todos os nós no caminho da raiz até este nó

- $\text{anceprop}(G) =$
- $\text{anceprop}(M) =$
- $\text{anceprop}(B) =$
- $\text{anceprop}(A) =$



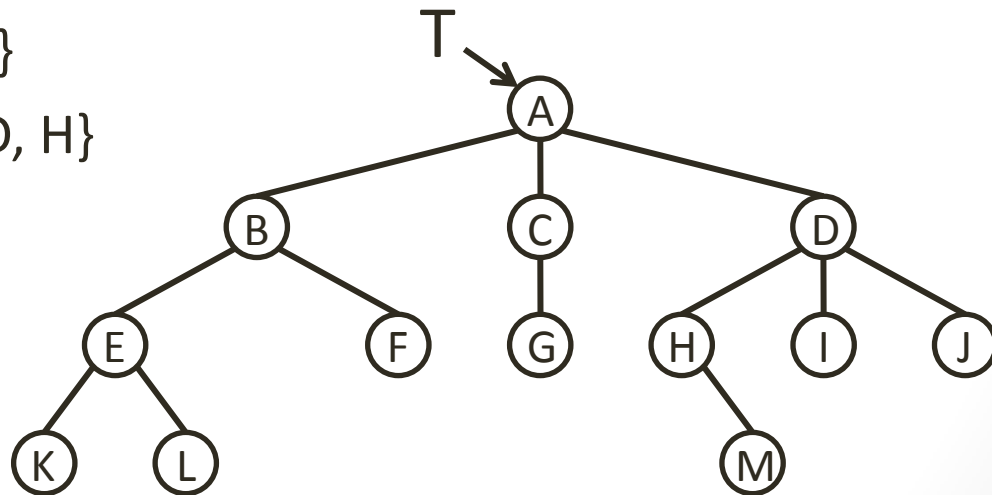
Árvores

- Algumas definições:

5. ANCESTRAIS (próprios) de um nó:

- São todos os nós no caminho da raiz até este nó

- $\text{anceprop}(G) = \{A, C\}$
- $\text{anceprop}(M) = \{A, D, H\}$
- $\text{anceprop}(B) = \{A\}$
- $\text{anceprop}(A) = \emptyset$



Árvores

- Algumas definições:

6. **DESCENDENTES** (próprios) de um nó:

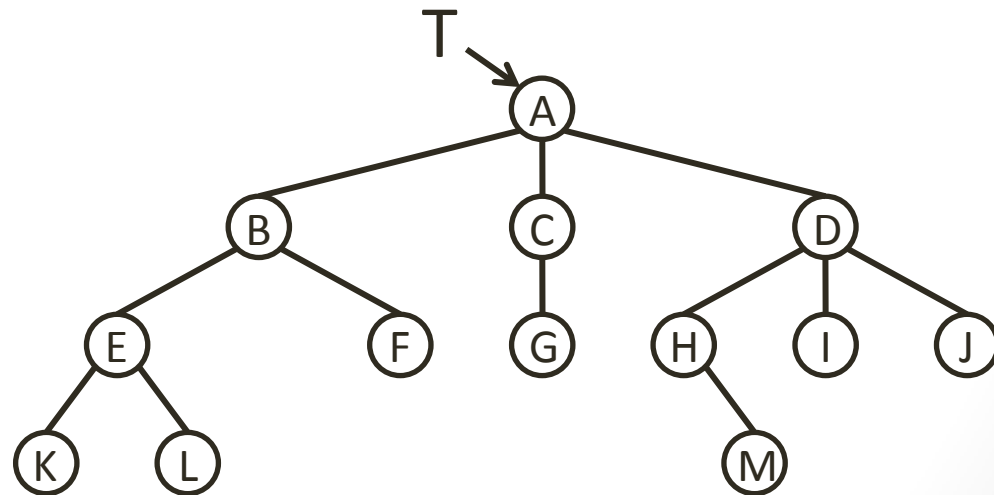
- São todos os nós de todas as subárvores deste nó

- $\text{descprop}(D) =$

- $\text{descprop}(E) =$

- $\text{descprop}(C) =$

- $\text{descprop}(L) =$



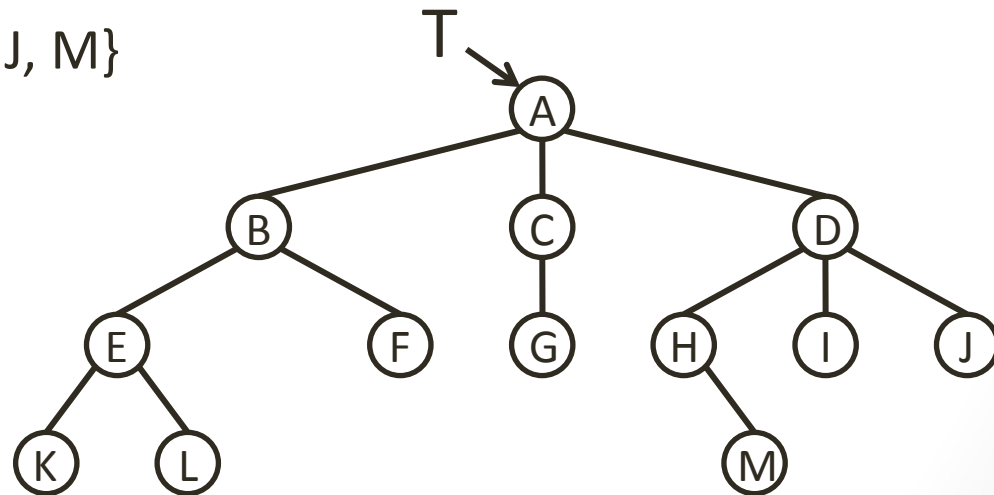
Árvores

- Algumas definições:

6. DESCENDENTES (próprios) de um nó:

- São todos os nós de todas as subárvores deste nó

- $\text{descprop}(D) = \{H, I, J, M\}$
- $\text{descprop}(E) = \{K, L\}$
- $\text{descprop}(C) = \{G\}$
- $\text{descprop}(L) = \emptyset$



Árvores

- Algumas definições:

7. NÍVEL ou PROFUNDIDADE de um nó:

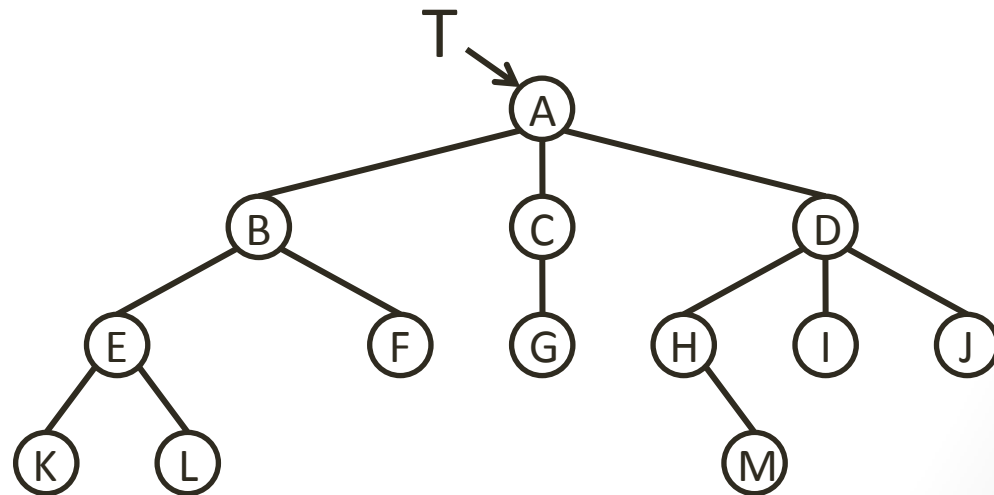
- É o comprimento do caminho que vai da raiz até este nó

- nível(M) =

- nível(C) =

- nível(E) =

- nível(A) =



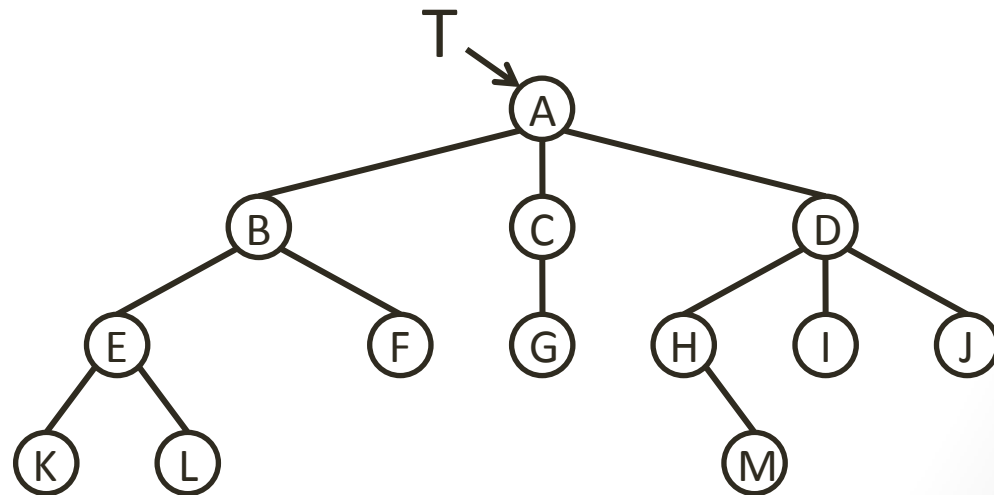
Árvores

- Algumas definições:

7. NÍVEL ou PROFUNDIDADE de um nó:

- É o comprimento do caminho que vai da raiz até este nó

- $\text{nível}(M) = 3$
- $\text{nível}(C) = 1$
- $\text{nível}(E) = 2$
- $\text{nível}(A) = 0$



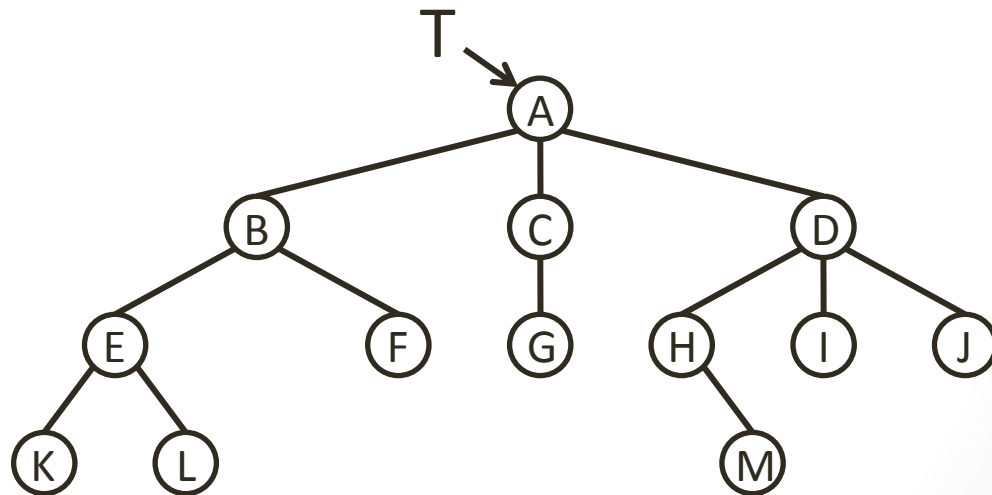
Árvores

- Algumas definições:

8. **ALTURA** de uma árvore:

- É o maior nível de um nó nesta árvore

- $\text{altura}(A) =$

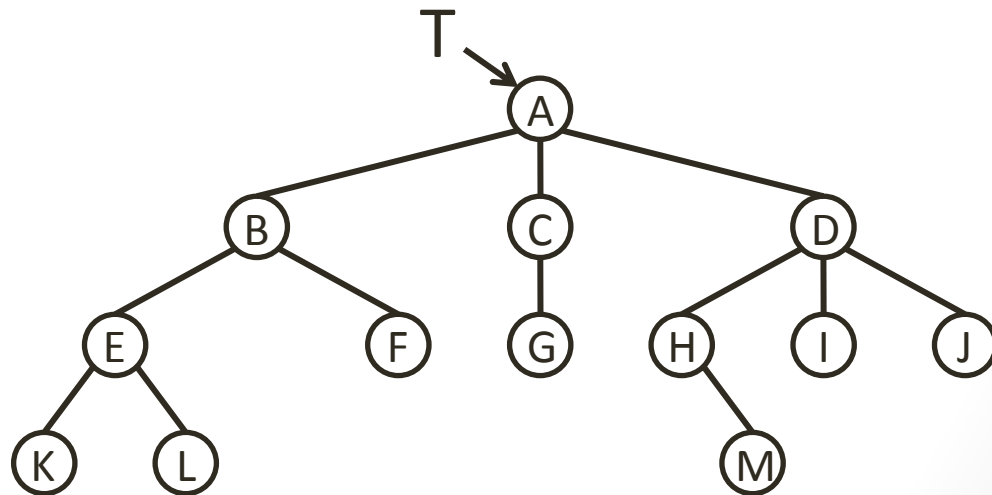


Árvores

- Algumas definições:

8. **ALTURA** de uma árvore:

- É o maior nível de um nó nesta árvore
- $\text{altura}(A) = 3$

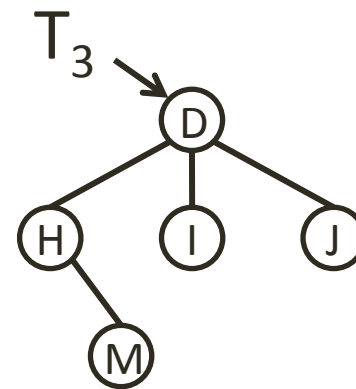
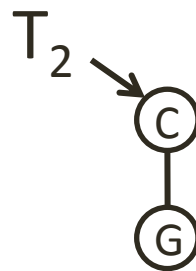
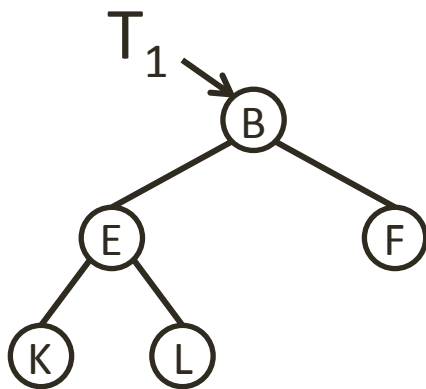


Árvores

- Algumas definições:

9. FLORESTA:

- É um conjunto de árvores disjuntas



Árvores

- Algumas aplicações de árvores:
- Busca
- Bancos de dados
- Compressão
- Processamento de imagens
- Computação gráfica
- Aprendizado de máquina
- Jogos

Árvores

- Exemplos de árvores:
- Árvore binária
 - Árvore binária de busca
 - Árvore binária de busca balanceada (AVL)
 - Árvore binária de busca balanceada (rubro-negra)
- Árvore B
 - Árvore B+
 - Árvore B*

Árvores

- Exemplos de árvores:
- Árvore de Huffman
- Árvore trie
 - Árvore trie binária
 - Árvore PATRICIA
- Quadtree e Octree
- Árvore k-d

Árvores binárias

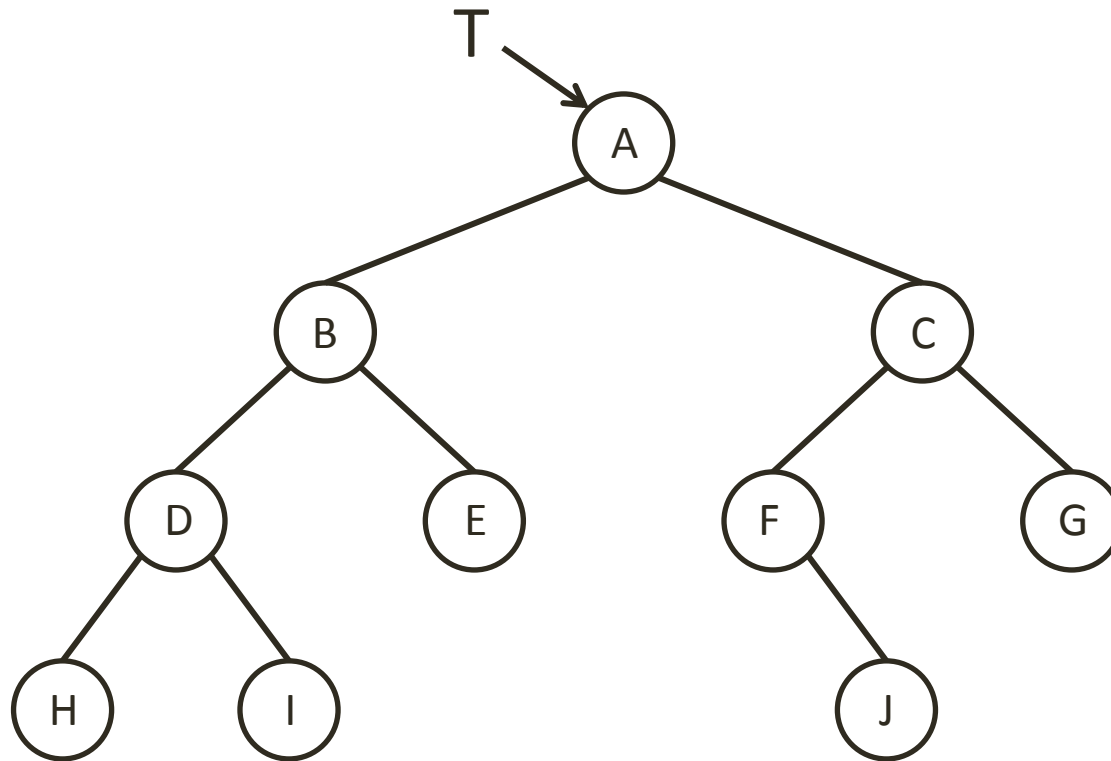
- Um **árvore binária** é um conjunto de nós que:
- OU é vazio
- OU consiste de uma raiz e de duas árvores binárias disjuntas chamadas de subárvore esquerda e subárvore direita da raiz

Árvores binárias



- OU é vazio

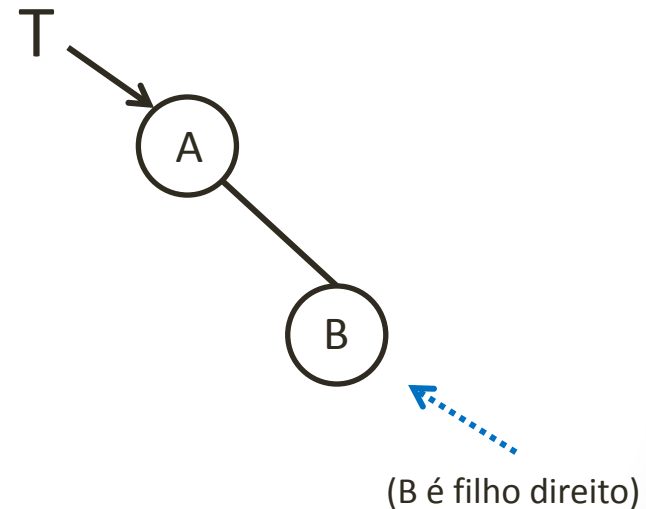
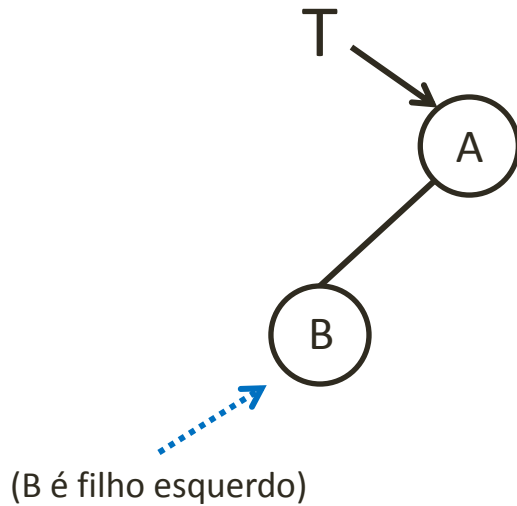
Árvores binárias



- OU consiste de uma raiz e de duas árvores binárias disjuntas chamadas de subárvore esquerda e subárvore direita da raiz

Árvores binárias

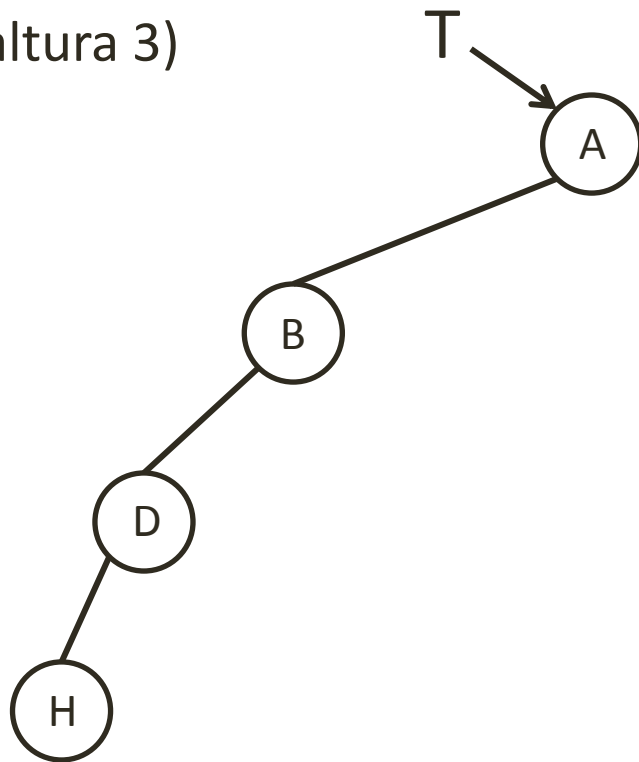
- OBS : as duas árvores binárias abaixo são diferentes



Árvores binárias

- Exemplos de árvores binárias: árvores degeneradas

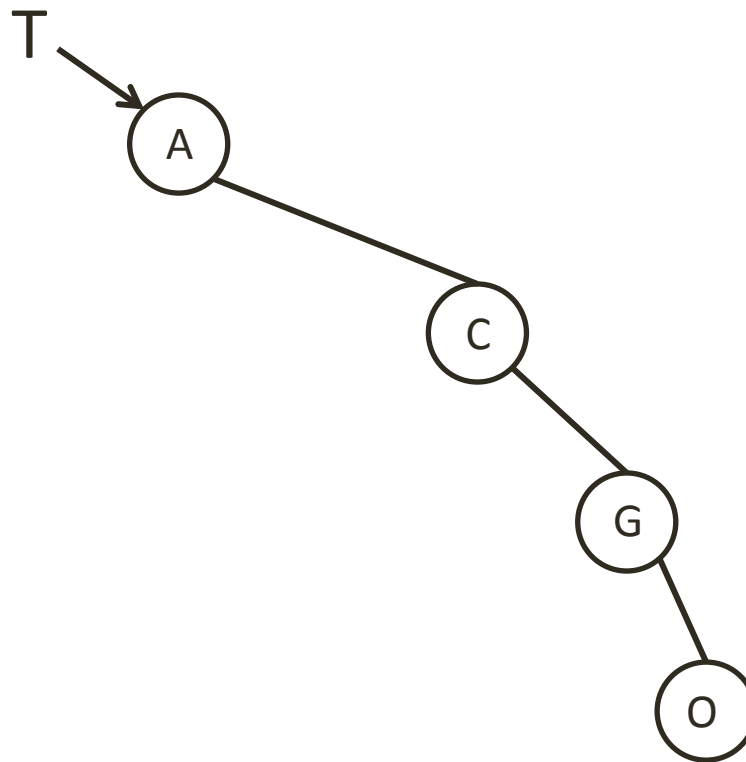
- (ex : altura 3)



Árvores binárias

- Exemplos de árvores binárias: árvores degeneradas

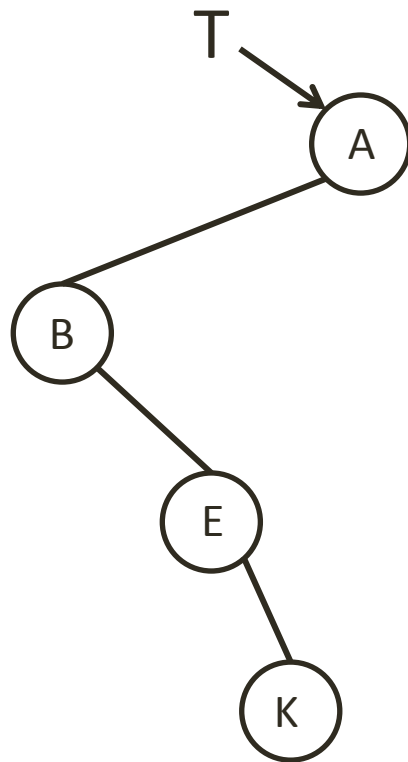
- (ex : altura 3)



Árvores binárias

- Exemplos de árvores binárias: árvores degeneradas

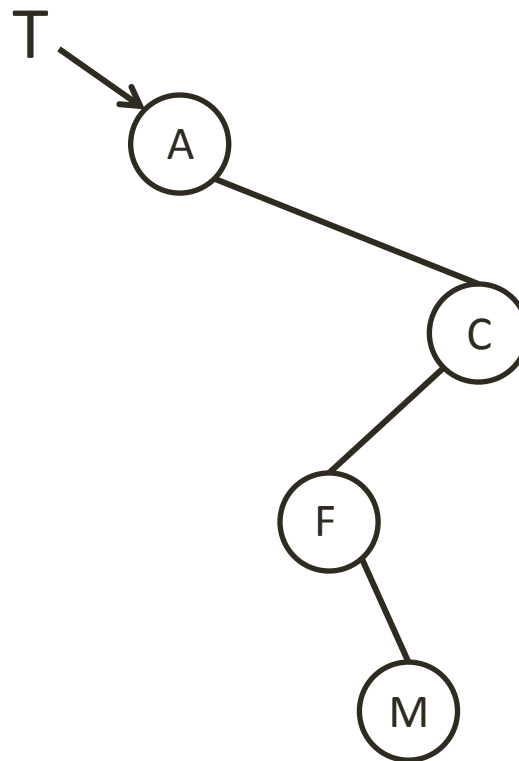
- (ex : altura 3)



Árvores binárias

- Exemplos de árvores binárias: árvores degeneradas

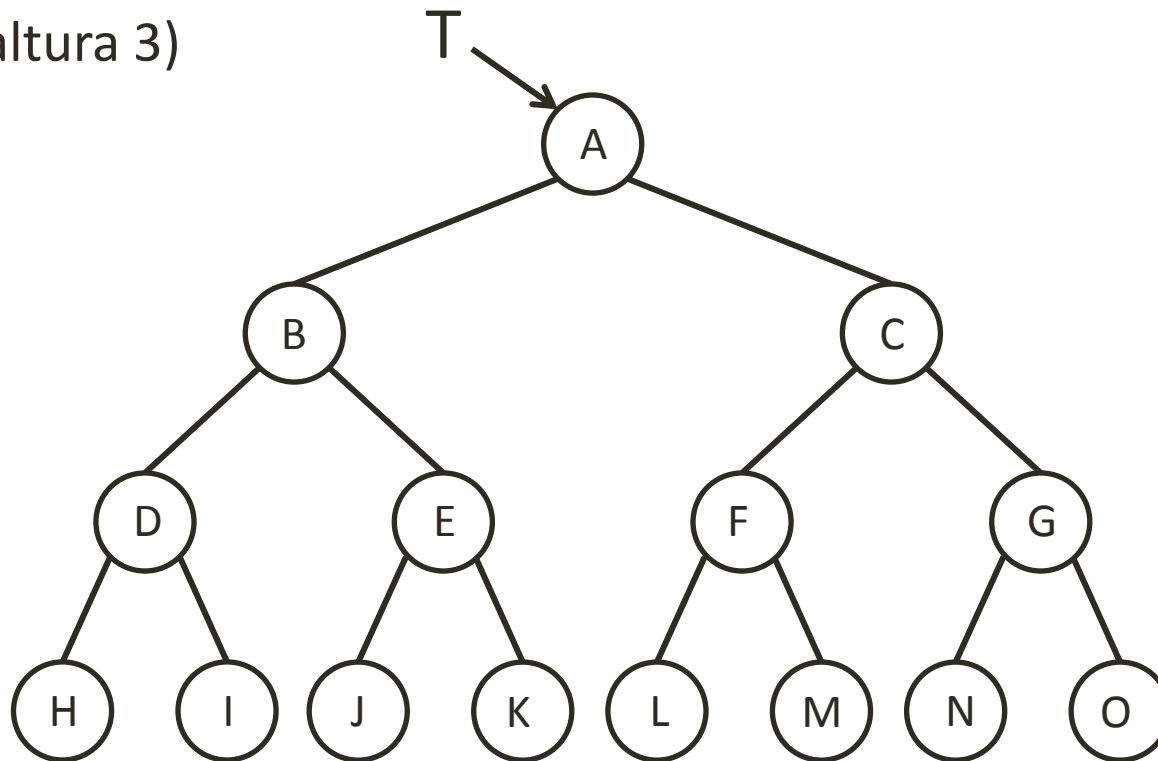
- (ex : altura 3)



Árvores binárias

- Exemplos de árvores binárias: árvores completas

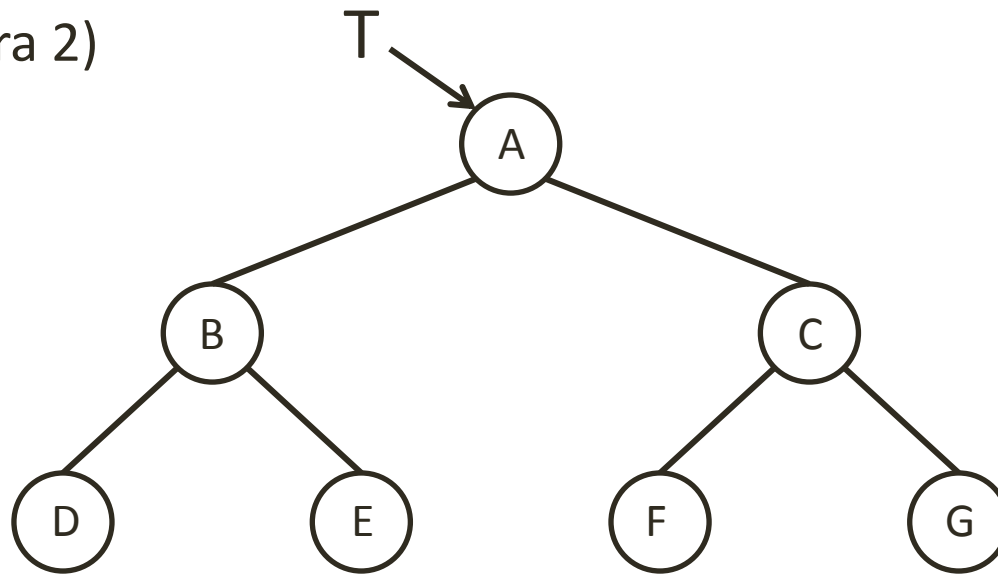
- (ex : altura 3)



Árvores binárias

- Exemplos de árvores binárias: árvores completas

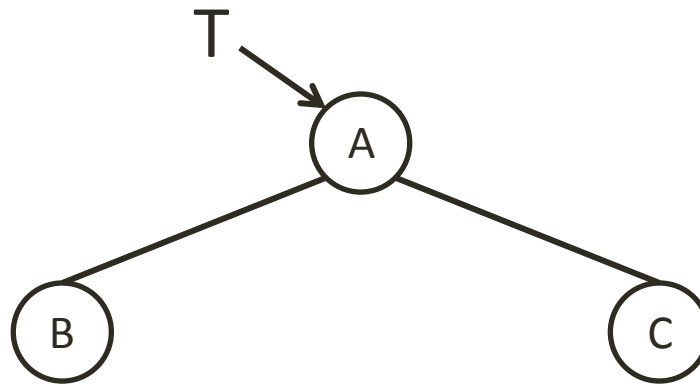
- (ex : altura 2)



Árvores binárias

- Exemplos de árvores binárias: árvores completas

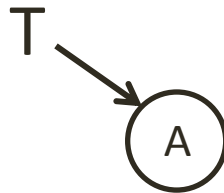
- (ex : altura 1)



Árvores binárias

- Exemplos de árvores binárias: árvores completas

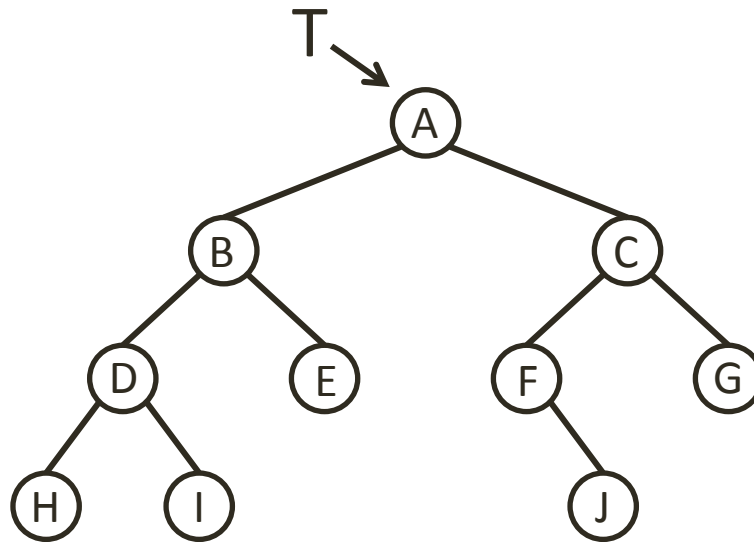
- (ex : altura 0)



Árvores binárias

- OBS : seja T uma árvore binária:

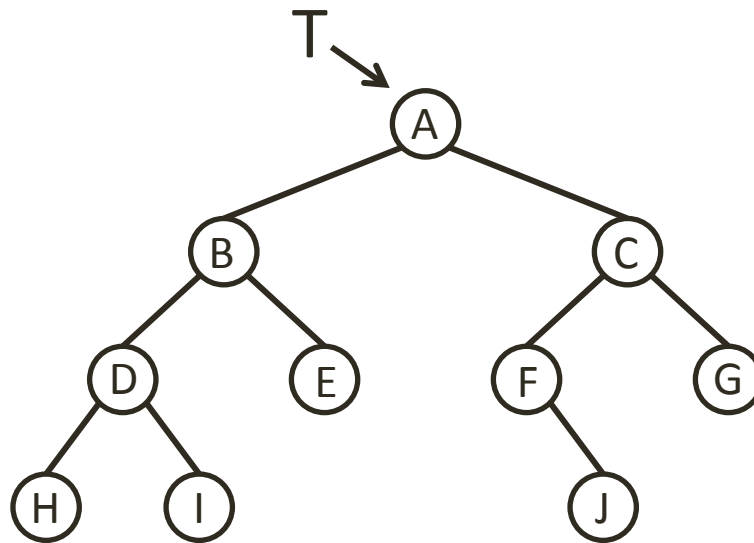
a) o número máximo de nós no nível i de T é 2^i , $\forall i \geq 0$



Árvores binárias

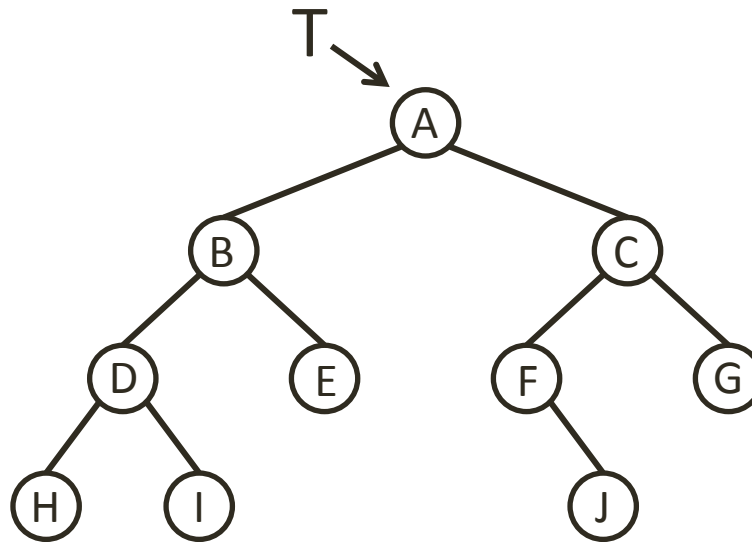
- OBS : seja T uma árvore binária:

a) o número máximo de nós no nível i de T é 2^i , $\forall i \geq 0$



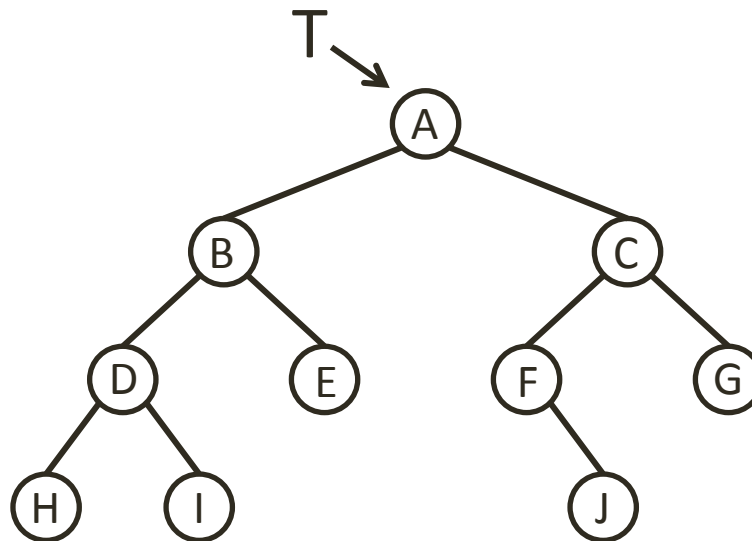
Árvores binárias

- OBS : seja T uma árvore binária:
- b) se k é a altura da árvore, o número máximo de nós em T é $2^{k+1} - 1$, $\forall k \geq 0$



Árvores binárias

- OBS : seja T uma árvore binária:
- b) se k é a altura da árvore, o número máximo de nós em T é $2^{k+1} - 1$, $\forall k \geq 0$

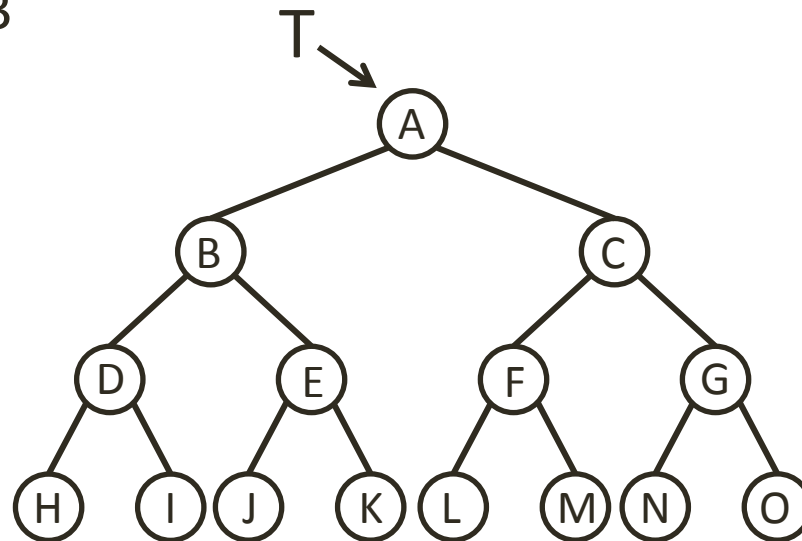


Árvores binárias

- OBS : seja T uma árvore binária:

c) T é completa de altura k se tiver nós e altura k

- exemplo: $k = 3$

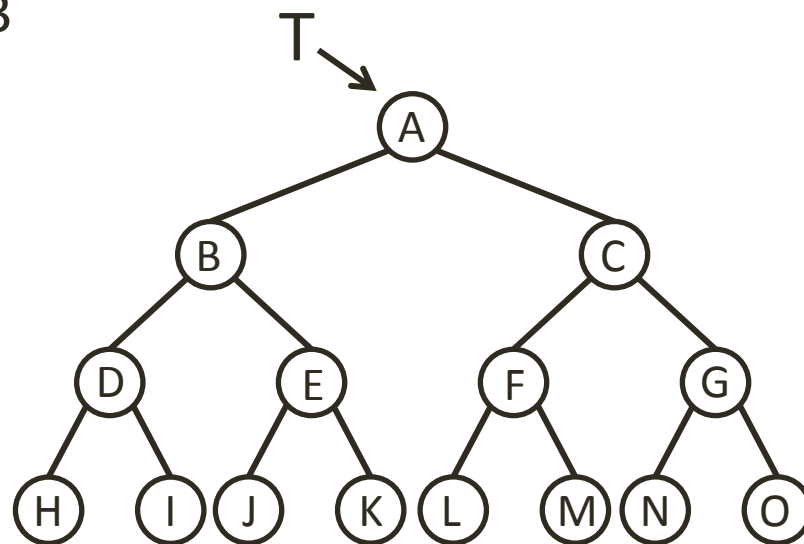


Árvores binárias

- OBS : seja T uma árvore binária:

c) T é completa de altura k se tiver $2^{k+1} - 1$ nós e altura k

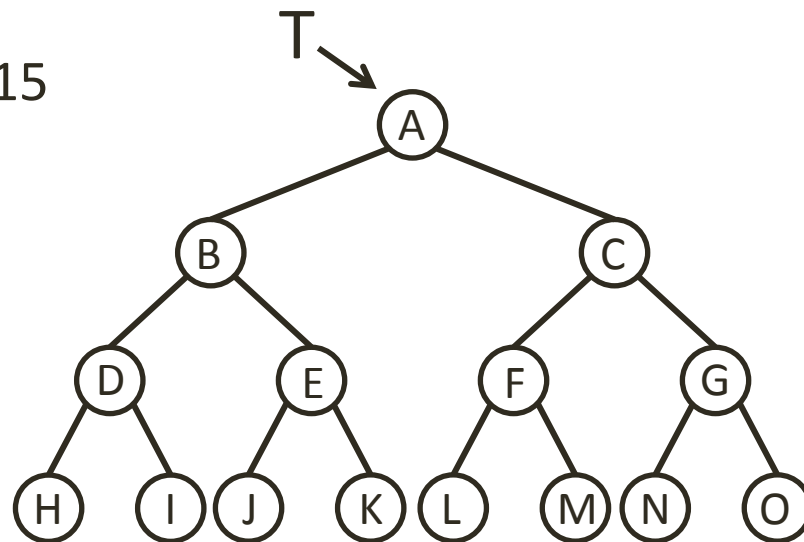
- exemplo: k = 3



Árvores binárias

- OBS : seja T uma árvore binária:
- d) A altura de uma árvore binária completa T com n nós é aproximadamente $\log_2 n$

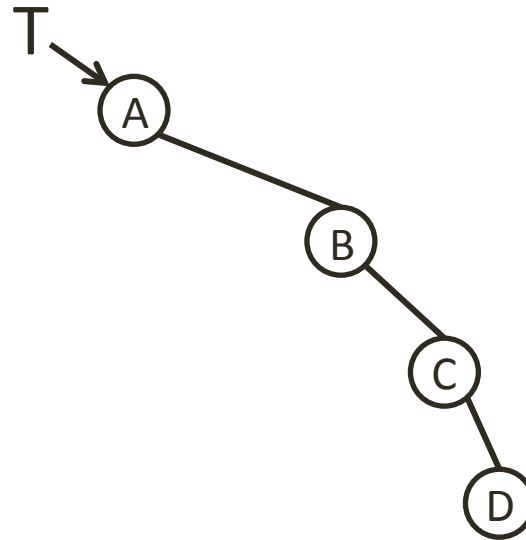
- exemplo: $n = 15$



Árvores binárias

- OBS : seja T uma árvore binária:
- e) A altura de uma árvore binária degenerada T com n nós é $n - 1$

- exemplo: $n = 4$

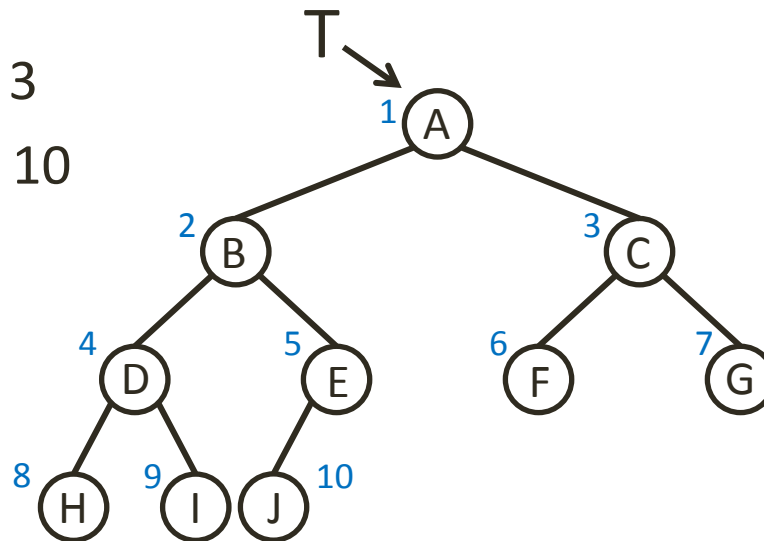


Árvores binárias

- OBS : seja T uma árvore binária:

f) T é quase completa de altura k e n nós quando tem o seguinte formato

- exemplo: $k = 3$
 $n = 10$



Árvores binárias

- Como representar árvores binárias (utilizando alocação dinâmica)

tipo TpNó = registro
início



info : tipo;

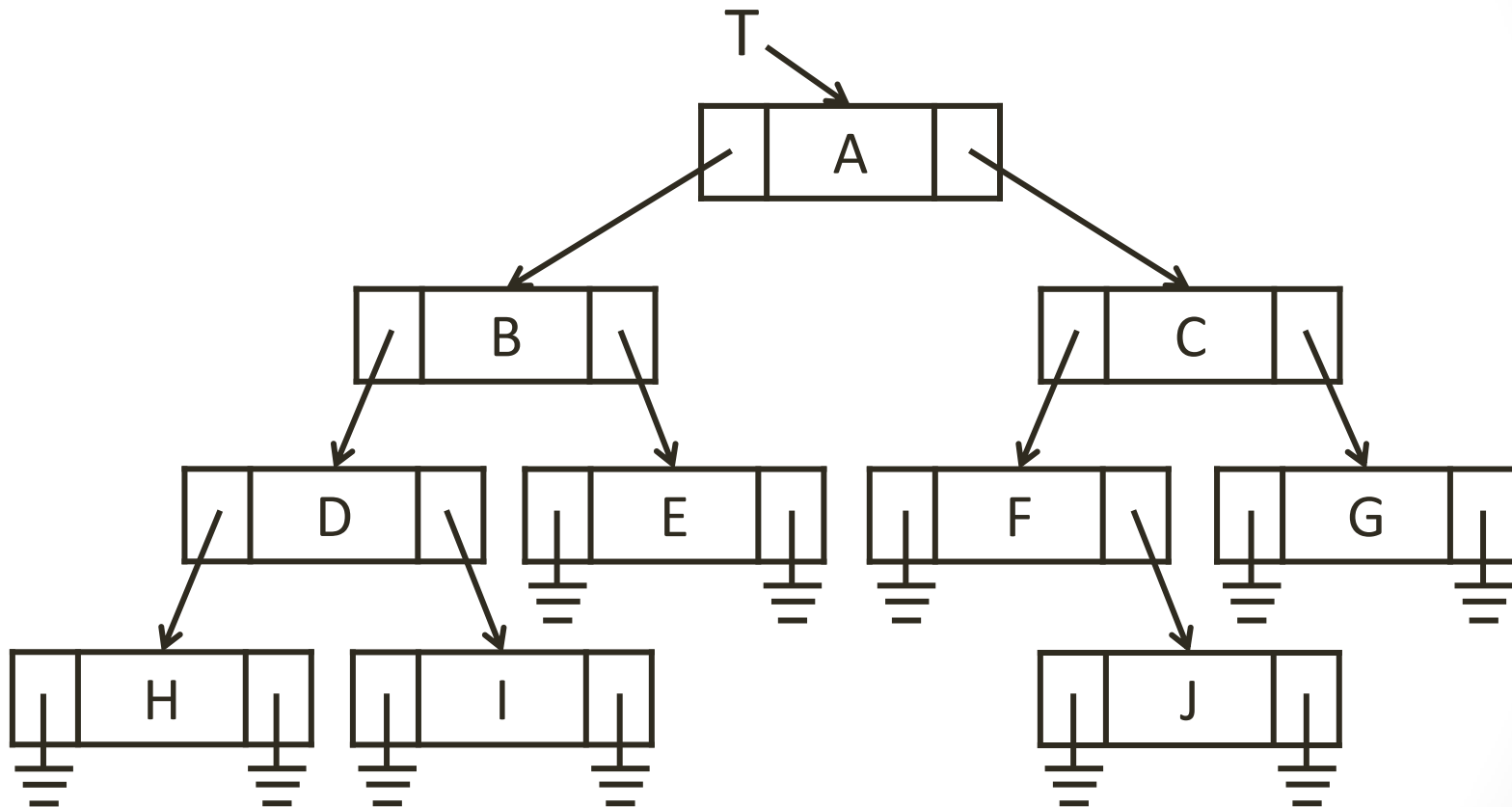
esq, dir : ^TpNó;

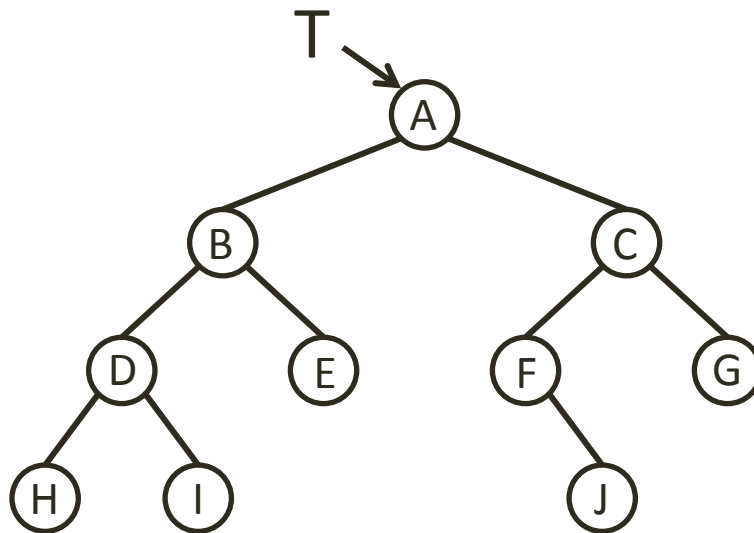
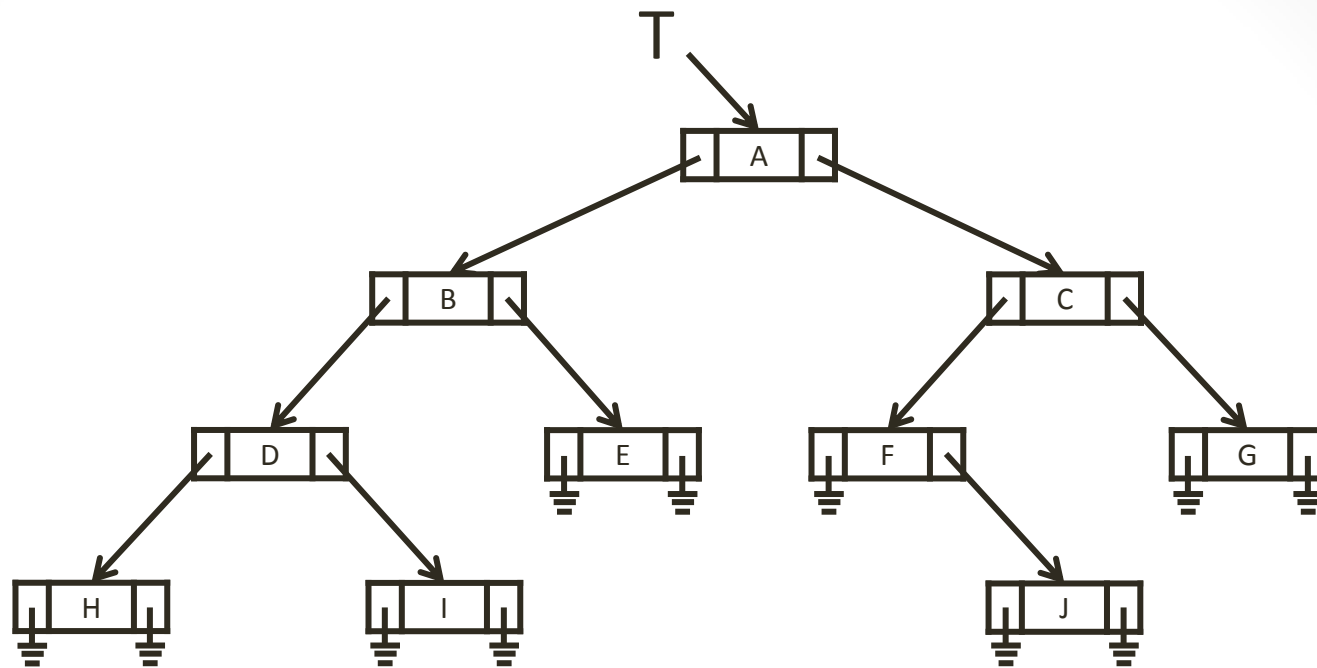
fim;

onde:

- info : campo de informação (pode ter um ou mais campos)
- esq : ponteiro para o filho esquerdo deste nó
- dir : ponteiro para o filho direito deste nó

Árvores binárias

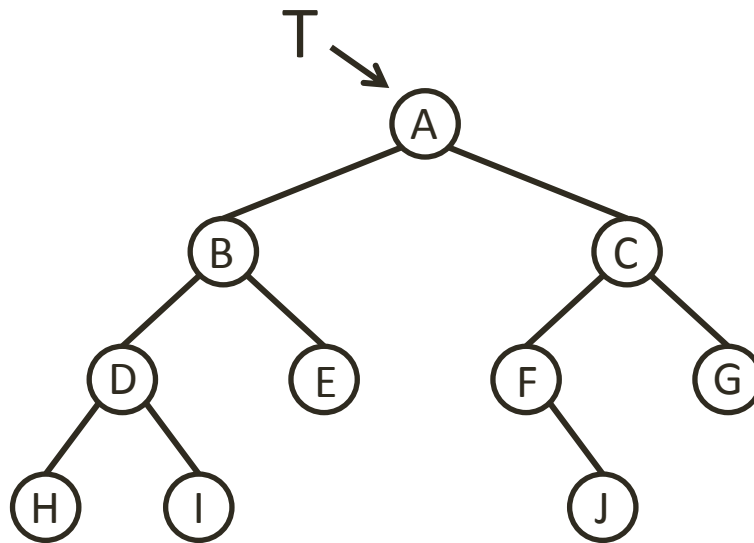


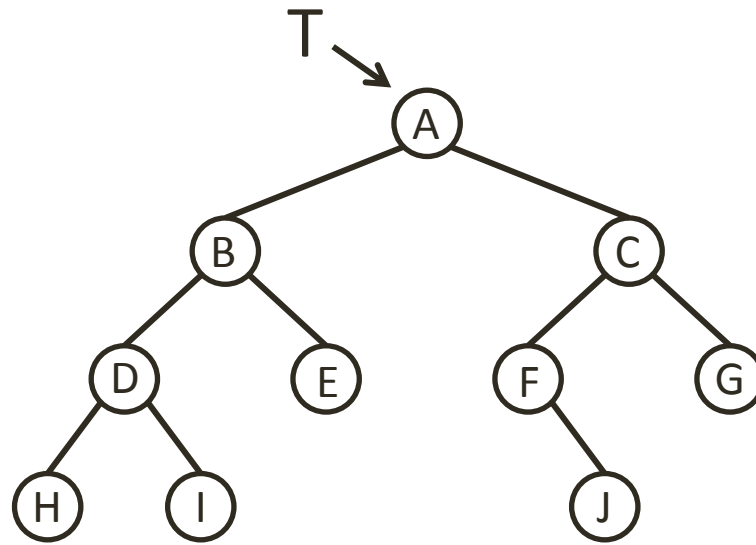


Árvores binárias

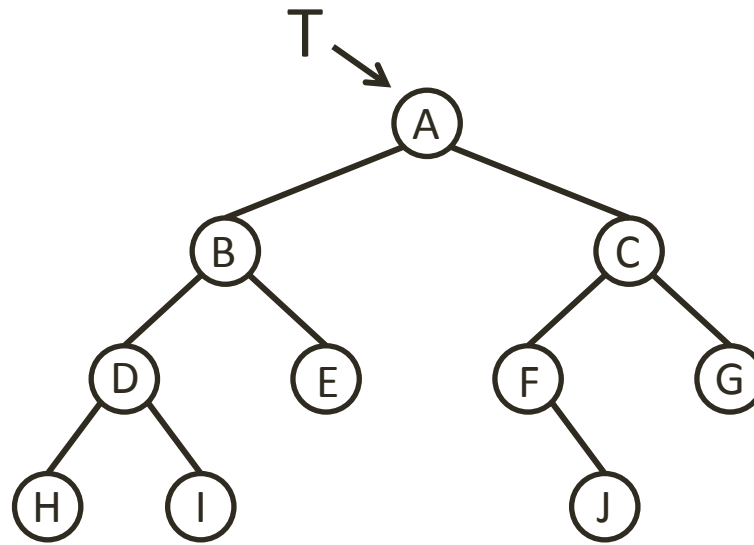
- Alguns algoritmos (recursivos) para árvores binárias:

a) Contar o número de nós em uma árvore binária





- ContaNós(T);



ContaNós(T);

se T = nil

então

ContaNós $\leftarrow$ 0;

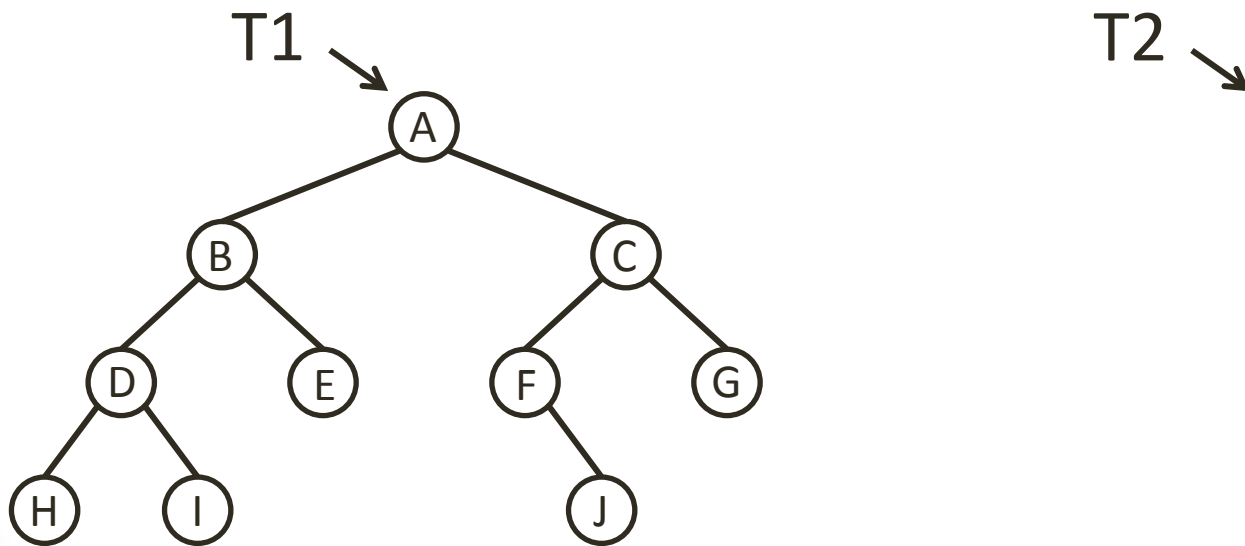
senão

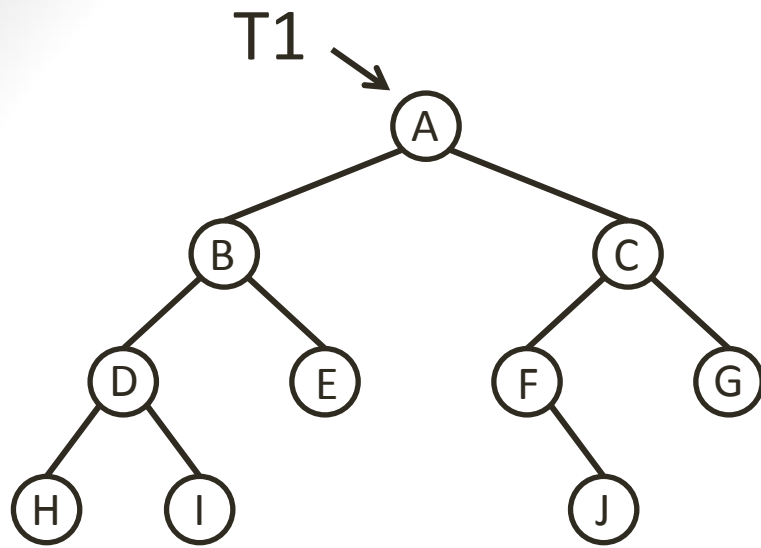
ContaNós $\leftarrow$ 1 + ContaNós(T^.esq) + ContaNós(T^.dir);

Árvores binárias

- Alguns algoritmos (recursivos) para árvores binárias:

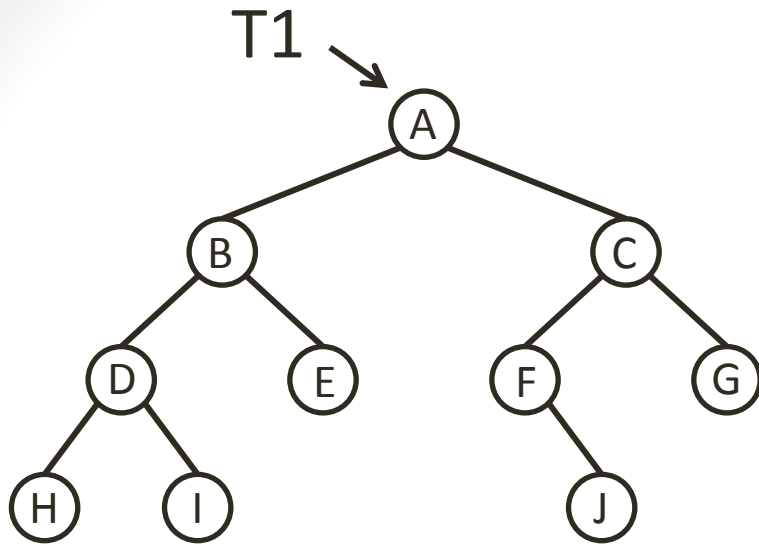
b) Fazer uma cópia de uma árvore binária





T2

- Cópia(T1, T2);



T2

Cópia(T1, T2);

se T1 = nil

então

T2 ← nil;

senão

novo(T2);

T2^.info ← T1^.info;

Cópia(T1^.esq, T2^.esq);

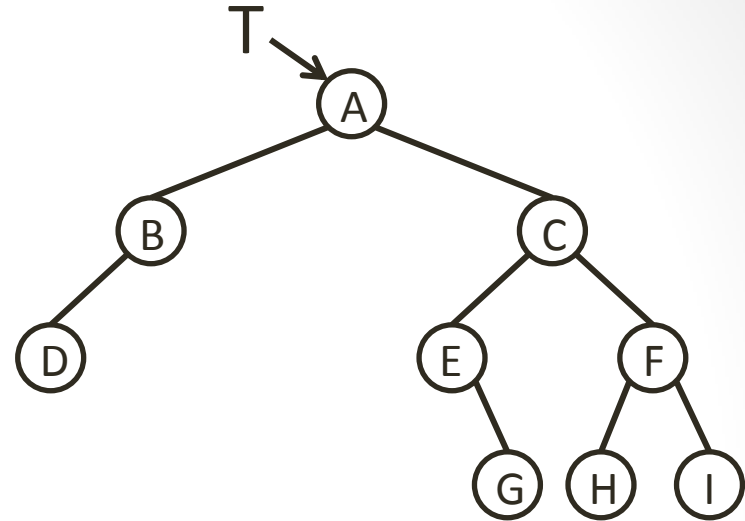
Cópia(T1^.dir, T2^.dir);

Árvores binárias

- Percursos em árvores binárias
- Em alguns algoritmos, é necessário que todos os nós da árvore sejam visitados em uma determinada ordem
- Percursos em profundidade:
 - In-ordem
 - Pré-ordem
 - Pós-ordem
- Percurso em largura

Árvores binárias

- Percursos em árvores binárias



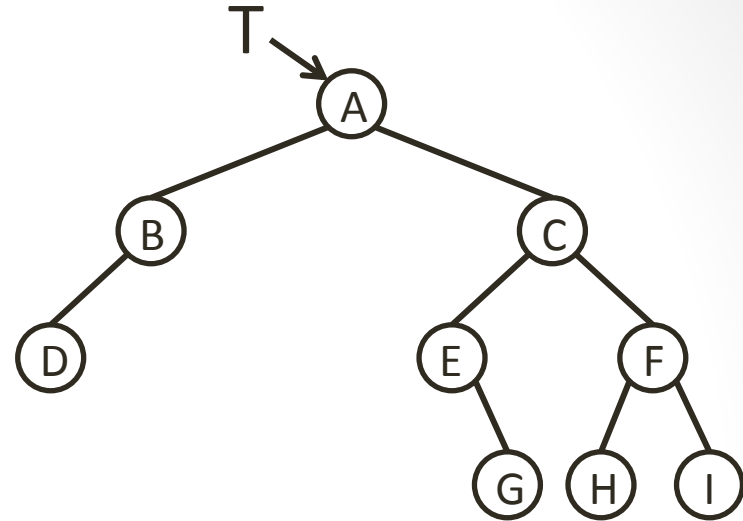
- IN-ORDEM :

- percorre a subárvore esquerda da raiz em in-ordem
- visita o nó (processa o nó)
- percorre a subárvore direita da raiz em in-ordem

percurso em T (in-ordem) :

Árvores binárias

- Percursos em árvores binárias



- IN-ORDEM :

- percorre a subárvore esquerda da raiz em in-ordem
- visita o nó (processa o nó)
- percorre a subárvore direita da raiz em in-ordem

percurso em T (in-ordem) : **D, B, A, E, G, C, H, F, I**

Árvores binárias

- Percursos em árvores binárias : in-ordem (recursivo)

```
InOrdem(T);
```

```
  se T <> nil
```

```
  então
```

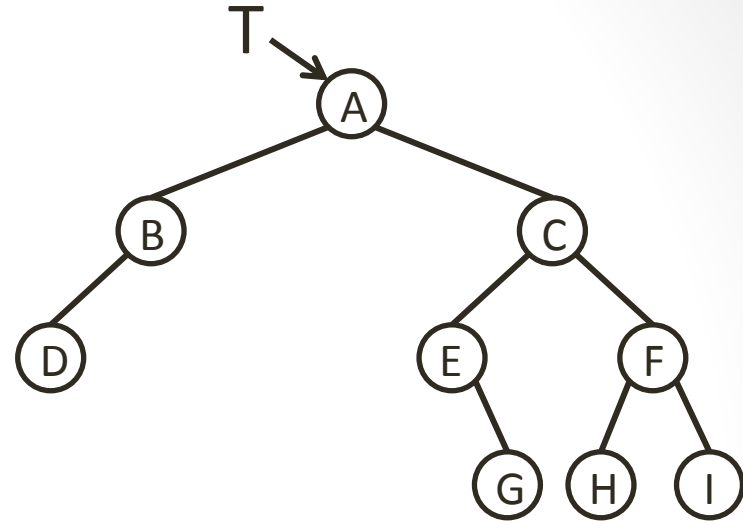
```
    InOrdem(T^.esq);
```

```
    VisitaNó(T);
```

```
    InOrdem(T^.dir);
```

Árvores binárias

- Percursos em árvores binárias



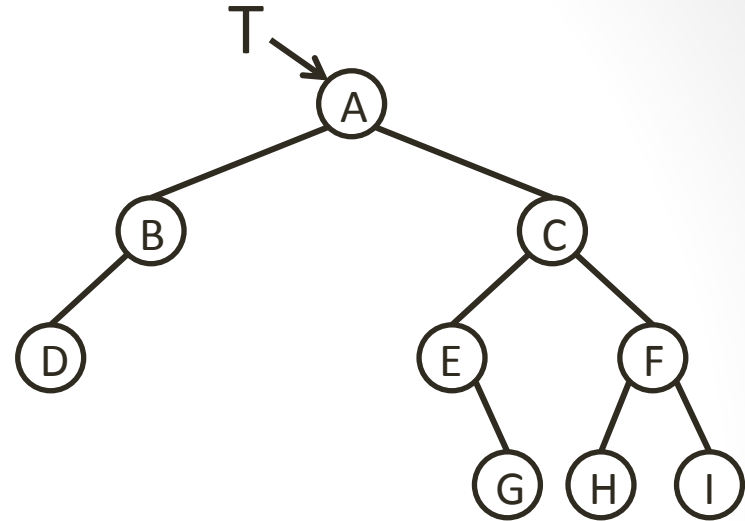
- PRÉ-ORDEM :

- visita o nó (processa o nó)
- percorre a subárvore esquerda da raiz em pré-ordem
- percorre a subárvore direita da raiz em pré-ordem

percurso em T (pré-ordem) :

Árvores binárias

- Percursos em árvores binárias



- PRÉ-ORDEM :
 - visita o nó (processa o nó)
 - percorre a subárvore esquerda da raiz em pré-ordem
 - percorre a subárvore direita da raiz em pré-ordem

percurso em T (pré-ordem) : **A, B, D, C, E, G, F, H, I**

Árvores binárias

- Percursos em árvores binárias : pré-ordem (recursivo)

PréOrdem(T);

se T <> nil

então

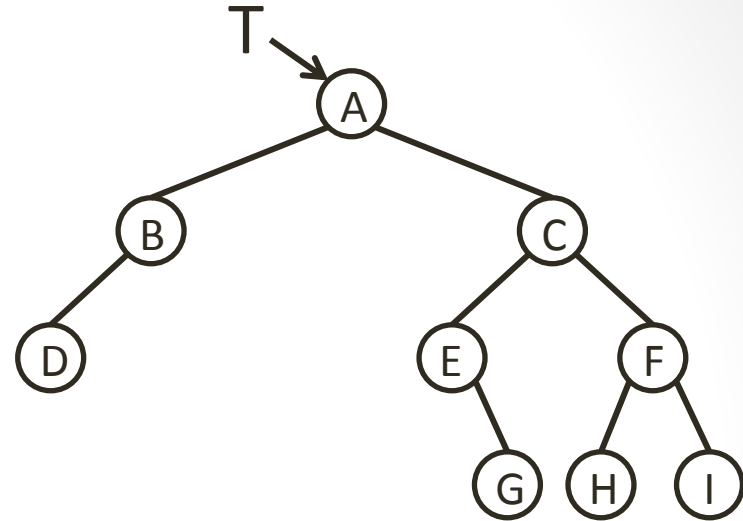
VisitaNó(T);

PréOrdem(T^.esq);

PréOrdem(T^.dir);

Árvores binárias

- Percursos em árvores binárias

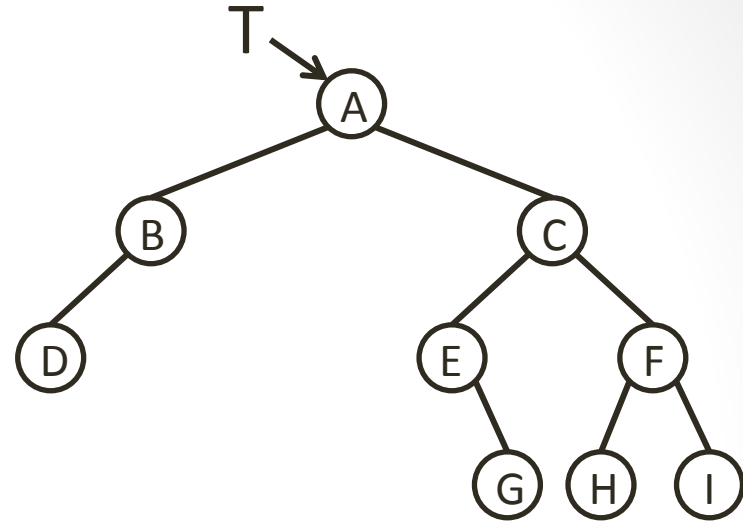


- PÓS-ORDEM :
 - percorre a subárvore esquerda da raiz em pós-ordem
 - percorre a subárvore direita da raiz em pós-ordem
 - visita o nó (processa o nó)

percurso em T (pós-ordem) :

Árvores binárias

- Percursos em árvores binárias



- PÓS-ORDEM :
 - percorre a subárvore esquerda da raiz em pós-ordem
 - percorre a subárvore direita da raiz em pós-ordem
 - visita o nó (processa o nó)

percurso em T (pós-ordem) : D, B, G, E, H, I, F, C, A

Árvores binárias

- Percursos em árvores binárias : pós-ordem (recursivo)

```
PósOrdem(T);
```

```
  se T <> nil
```

```
  então
```

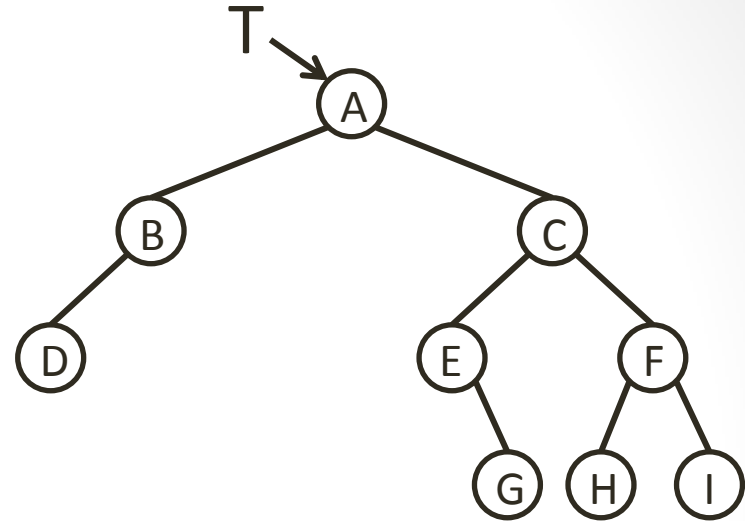
```
    PósOrdem(T^.esq);
```

```
    PósOrdem(T^.dir);
```

```
    VisitaNó(T);
```

Árvores binárias

- Percursos em árvores binárias



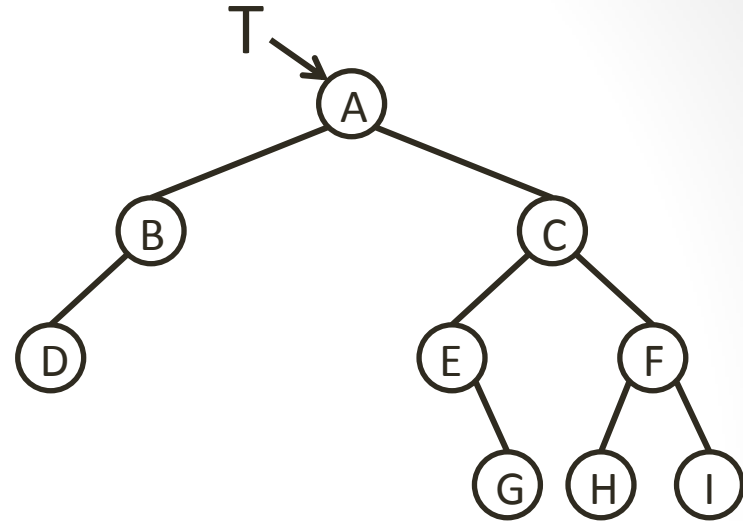
- LARGURA :

- visita os nós da árvore em ordem crescente de nível (inicia-se na raiz, que está no nível zero. Depois, visitam-se os nós do nível 1. Depois, os do nível 2, e assim por diante)
- em cada nível, os nós são visitados da esquerda para a direita

percurso em T (largura) :

Árvores binárias

- Percursos em árvores binárias



- LARGURA :

- visita os nós da árvore em ordem crescente de nível (inicia-se na raiz, que está no nível zero. Depois, visitam-se os nós do nível 1. Depois, os do nível 2, e assim por diante)
- em cada nível, os nós são visitados da esquerda para a direita

percurso em T (largura) : **A, B, C, D, E, F, G, H, I**

Árvores binárias

- Percursos em árvores binárias : largura (breadth-first search)

BFS(T);

se T <> nil

então

InicializaFila(Q);

InsereFila(Q, T);

enquanto NÃO FilaVazia(Q)

faça

RemoveFila(Q, X);

VisitaNó(X);

se X^.esq <> nil

então

InsereFila(Q, X^.esq);

se X^.dir <> nil

então

InsereFila(Q, X^.dir);

Árvores binárias

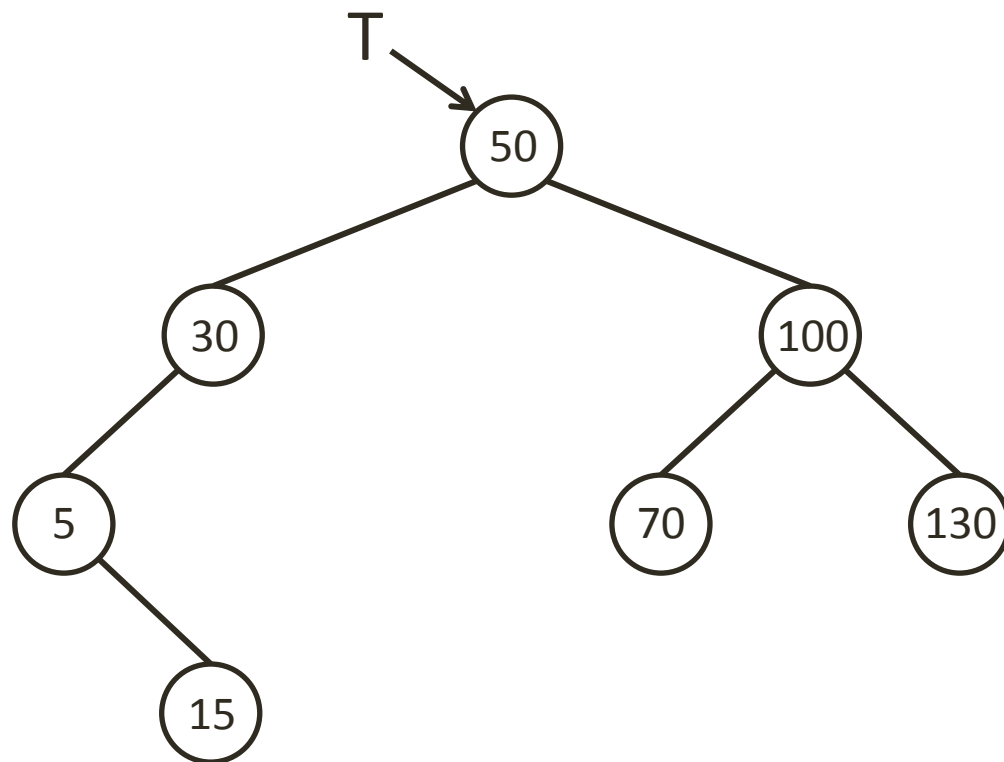
- Percursos em árvores binárias
- Considere as funções $\text{ContaNós}(T)$ e $\text{Cópia}(T1, T2)$, implementadas anteriormente
- A função $\text{ContaNós}(T)$ corresponde a qual dos percursos estudados?
- E a função $\text{Cópia}(T1, T2)$?

Árvores binárias de busca

- Considere um conjunto que possua uma relação de ordem total $\leq$ (menor ou igual) entre seus elementos
- Uma **árvore binária de busca** é uma árvore binária em que, para todo nó P desta árvore:
 - i. todo nó Q na subárvore esquerda de P satisfaz **$Q.info < P.info$**
 - ii. todo nó S na subárvore direita de P satisfaz **$P.info < S.info$**

Árvores binárias de busca

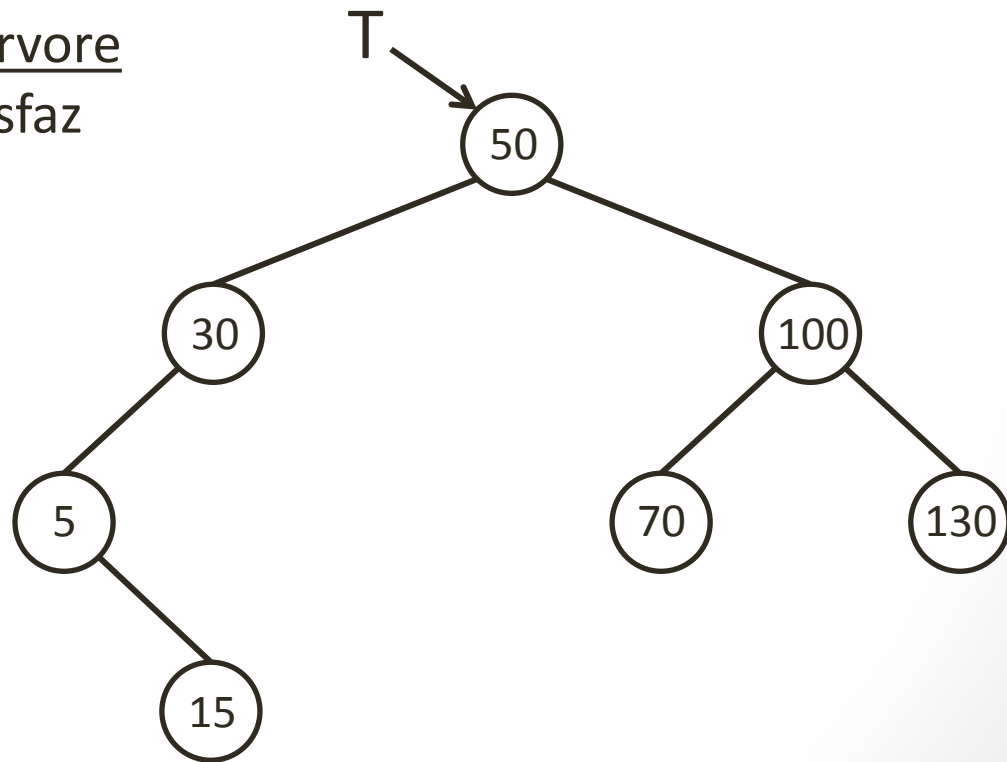
- Exemplo de árvore binária de busca



Árvores binárias de busca

- Para todo nó P de uma árvore binária de busca T :

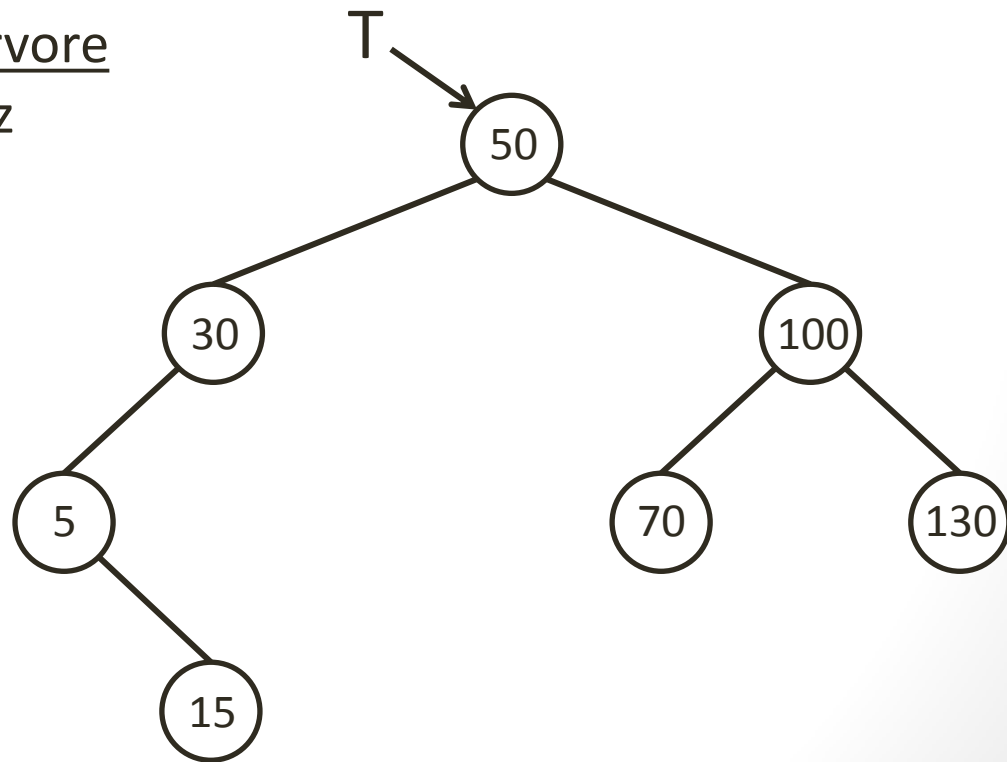
- i. todo nó Q na subárvore esquerda de P satisfaz $Q^{\text{info}} < P^{\text{info}}$



Árvores binárias de busca

- Para todo nó P de uma árvore binária de busca T :

- ii. todo nó S na subárvore direita de P satisfaz
 $P^{\wedge}.info < S^{\wedge}.info$



Árvores binárias de busca

- A árvore binária de busca é uma estrutura que admite métodos eficientes de busca, ordenação e manipulação dos elementos armazenados
- A busca em tal árvore pode ser tão eficiente quanto a busca binária aplicada em um vetor ordenado.
 - (quando a árvore estiver balanceada)
- Porém, a busca em uma árvore binária de busca pode também se tornar um caso de busca linear
 - (quando a árvore binária de busca é uma árvore degenerada)

Árvores binárias de busca

- Inserções e remoções de elementos em uma árvore binária de busca são eficientes
 - (compare com inserções e remoções em um vetor ordenado)
- Os tempos de execução das operações de manipulação de uma árvore binária de busca são proporcionais à altura da árvore
 - $O(\log_2 n)$ no caso de árvore balanceadas
 - $O(n)$ no caso de árvores degeneradas
- O percurso in-ordem em uma árvore binária de busca permite acessar seus elementos sequencialmente e em **ordem crescente**

Árvores binárias de busca

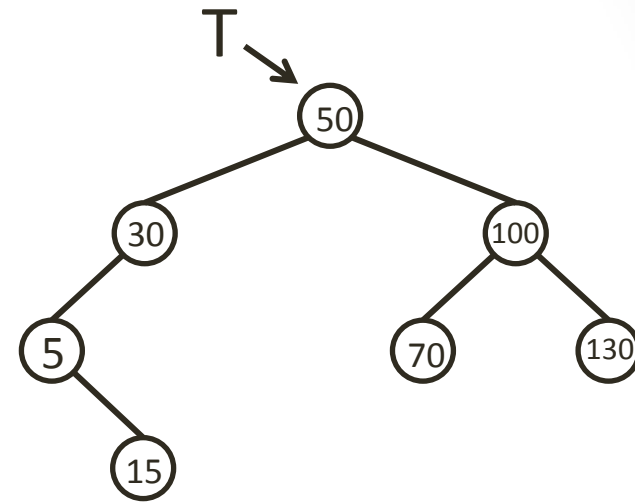
- Algumas funções de gerenciamento de árvores binárias de busca:
- InOrdem(T);
 - Os elementos da árvore T são visitados em **ordem crescente**
- MínimoABB(T);
 - Devolve um ponteiro para o nó que contém o menor elemento da árvore T, OU
 - Devolve **nil** se a árvore T for vazia
- MáximoABB(T);
 - Devolve um ponteiro para o nó que contém o maior elemento da árvore T, OU
 - Devolve **nil** se a árvore T for vazia

Árvores binárias de busca

- Algumas funções de gerenciamento de árvores binárias de busca:
- BuscaABB(T, x);
 - Busca por um elemento x na árvore T. Devolve um ponteiro para o nó que contenha x, OU
 - Devolve **nil** se x não estiver na árvore T
- InsereABB(T, x);
 - Insere o elemento x na árvore T
- RemoveABB(T, x);
 - Remove o elemento x da árvore T

Árvores binárias de busca

- MínimoABB(T); {iterativo}



Complexidade:

Árvores binárias de busca

```
MínimoABB(T);      {iterativo}

P ← T;

se P <> nil
então
    enquanto P^.esq <> nil
    faça
        P ← P^.esq;
MínimoABB ← P;
```

Complexidade:

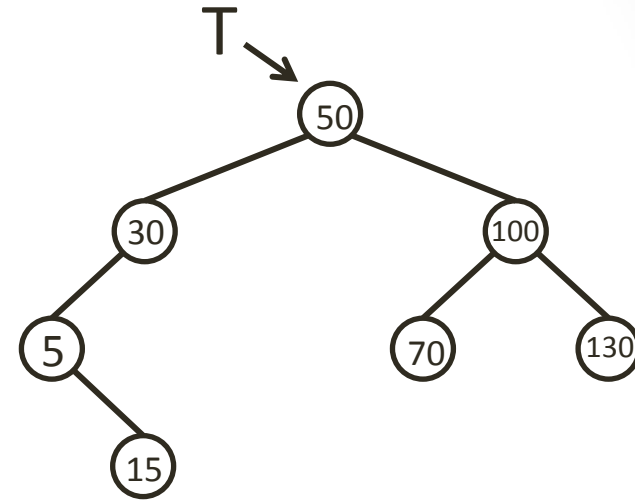
$O(n)$

$O(\log_2 n)$

(se a árvore T estiver balanceada)

Árvores binárias de busca

- MínimoABB(T); {recursivo}



Complexidade:

Árvores binárias de busca

MínimoABB(T); {recursivo}

se (T = nil) OU (T^.esq = nil)

então | MínimoABB $\leftarrow$ T;

senão | MínimoABB $\leftarrow$ MínimoABB(T^.esq);

Complexidade:

$O(n)$

$O(\log_2 n)$

(se a árvore T estiver balanceada)

Árvores binárias de busca

```
MáximoABB(T);      {iterativo}

P ← T;

se P <> nil
então
    enquanto P^.dir <> nil
    faça
        P ← P^.dir;
MáximoABB ← P;
```

Complexidade:

$O(n)$

$O(\log_2 n)$

(se a árvore T estiver balanceada)

Árvores binárias de busca

MáximoABB(T); {recursivo}

se (T = nil) OU (T^.dir = nil)

então | MáximoABB $\leftarrow$ T;

senão | MáximoABB $\leftarrow$ MáximoABB(T^.dir);

Complexidade:

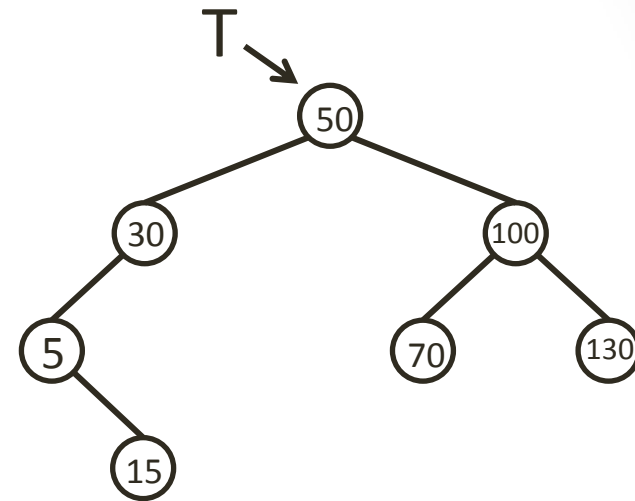
$O(n)$

$O(\log_2 n)$

(se a árvore T estiver balanceada)

Árvores binárias de busca

- BuscaABB(T, x); {iterativo}



Complexidade:

Árvores binárias de busca

BuscaABB(T, x); {iterativo}

$P \leftarrow T;$

enquanto $(P \neq \text{nil}) \ \underline{\text{E}} \ (P.^{\text{info}} \neq x)$

<u>faça</u>		<u>se</u> $x < P.^{\text{info}}$
		<u>então</u> $P \leftarrow P.^{\text{esq}};$
		<u>senão</u> $P \leftarrow P.^{\text{dir}};$

BuscaABB $\leftarrow P;$

Complexidade:

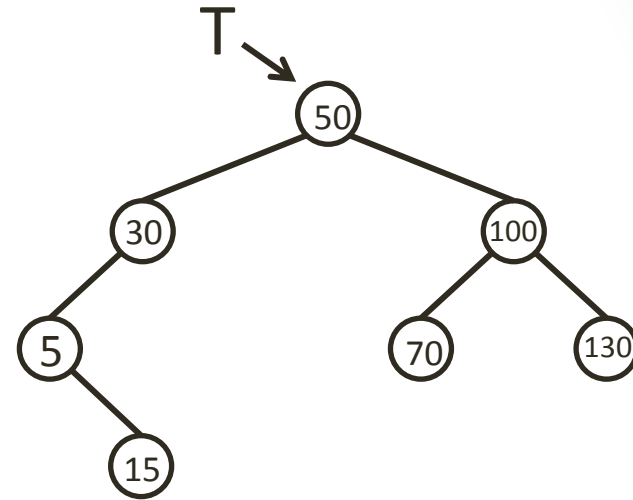
$O(n)$

$O(\log_2 n)$

(se a árvore T estiver balanceada)

Árvores binárias de busca

- BuscaABB(T, x); {recursivo}



Complexidade:

Árvores binárias de busca

BuscaABB(T, x); {recursivo}

se (T = nil) OU (T^.info = x)

então | BuscaABB $\leftarrow$ T;

senão | se x < T^.info

 | então | BuscaABB $\leftarrow$ BuscaABB(T^.esq, x);

 | senão | BuscaABB $\leftarrow$ BuscaABB(T^.dir, x);

Complexidade:

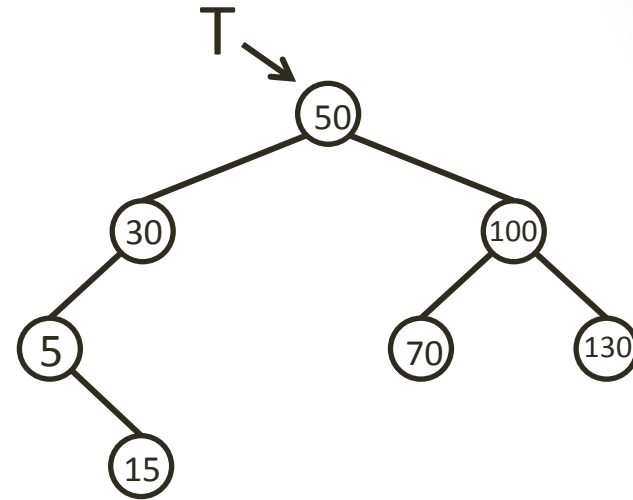
$O(n)$

$O(\log_2 n)$

(se a árvore T estiver balanceada)

Árvores binárias de busca

- `InserreABB(T, x);`



Complexidade:

Árvores binárias de busca

InserirABB(T, x);

se T = nil

então

novo(T);

T^.info ← x;

T^.esq ← nil;

T^.dir ← nil;

senão

se x < T^.info

então

InserirABB(T^.esq, x);

senão

se x > T^.info

então

InserirABB(T^.dir, x);

Complexidade:

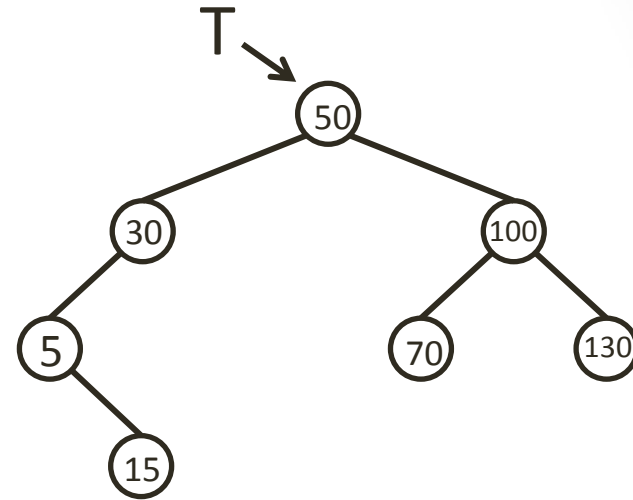
$O(n)$

$O(\log_2 n)$

(se a árvore T estiver balanceada)

Árvores binárias de busca

- RemoveABB(T, x);



Complexidade:

RemoveABB(T, x);

Complexidade:

$O(n)$

$O(\log_2 n)$

(se a árvore T estiver balanceada)

se T <> nil

então

se x < T^.info

então

RemoveABB(T^.esq, x);

senão

se x > T^.info

então

RemoveABB(T^.dir, x);

senão

se (T^.esq <> nil) E (T^.dir <> nil)

então

P ← MínimoABB(T^.dir);

T^.info ← P^.info;

RemoveABB(T^.dir, P^.info);

senão

Q ← T;

se T^.esq = nil

então

T ← T^.dir;

senão

T ← T^.esq;

dispose(Q);

Árvores binárias de busca

- Complexidade das funções apresentadas

Função	Iterativo	Recursivo
MínimoABB(T);	$O(n)$	$O(n)$
MáximoABB(T);	$O(n)$	$O(n)$
BuscaABB(T, x);	$O(n)$	$O(n)$
InsererABB(T, x);	-	$O(n)$
RemoveABB(T, x);	-	$O(n)$

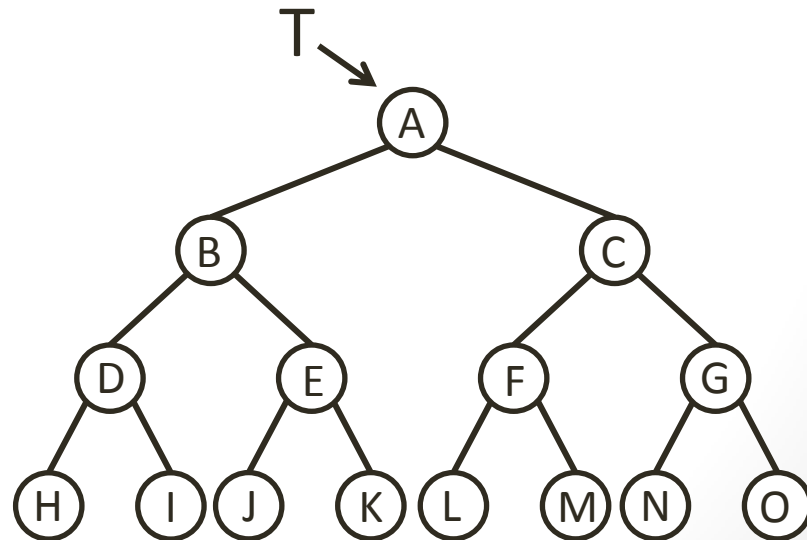
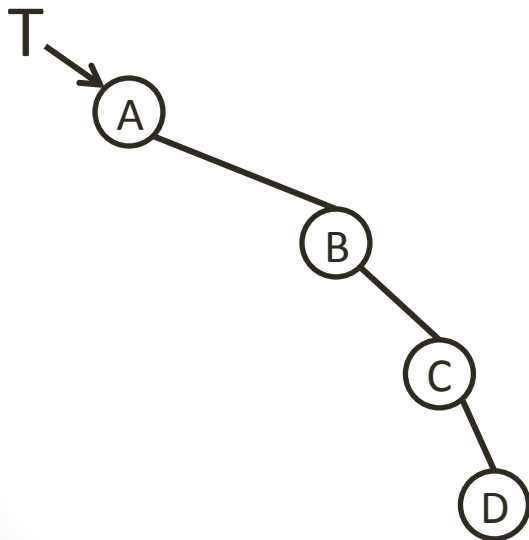
Árvores binárias de busca

- Complexidade das funções apresentadas
(quando a árvore binária de busca está balanceada)

Função	Iterativo	Recursivo
MínimoABB(T);	$O(\log_2 n)$	$O(\log_2 n)$
MáximoABB(T);	$O(\log_2 n)$	$O(\log_2 n)$
BuscaABB(T, x);	$O(\log_2 n)$	$O(\log_2 n)$
InsererABB(T, x);	-	$O(\log_2 n)$
RemoveABB(T, x);	-	$O(\log_2 n)$

Árvores AVL

- Árvores balanceadas
- A complexidade das funções de gerenciamento de árvores binárias de busca que estudamos depende do formato destas árvores



Árvores AVL

- Árvores balanceadas
- Se a árvore binária de busca for uma árvore degenerada, a complexidade das operações será $O(n)$, onde n é o número de elementos da árvore
- Se, por exemplo, inserirmos os elementos em uma árvore binária de busca em ordem crescente (ou decrescente), o resultado seria uma árvore degenerada

Árvores AVL

- Árvores balanceadas
- Se a árvore binária de busca for uma árvore degenerada, a complexidade das operações será $O(n)$, onde n é o número de elementos da árvore
- A busca em uma árvore degenerada é equivalente à busca em uma lista linear

Árvores AVL

- Árvores balanceadas
- Se a árvore binária de busca for uma árvore completa, a complexidade das operações será $O(\log_2 n)$, onde n é o número de elementos da árvore
- Desejaríamos manter esta complexidade para as operações nesta árvore. Contudo, seria muito caro manter a árvore binária como uma árvore completa após cada operação de inserção ou remoção

Árvores AVL

- Árvores balanceadas
- Existem alternativas de menor custo para aproximar o formato da árvore binária de busca do formato de uma árvore completa
- Assim, o tempo de execução das operações de gerenciamento da árvore em questão seria próximo ao tempo de execução das operações associadas a uma árvore completa

Árvores AVL

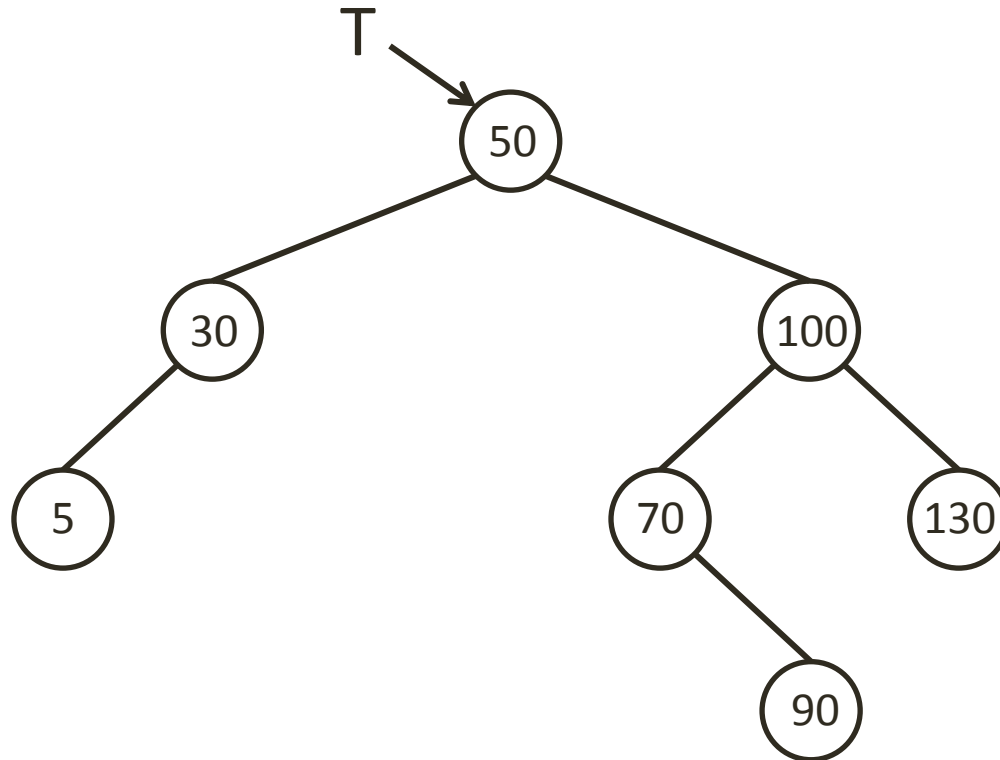
- Árvores balanceadas
- Existem alternativas de menor custo para aproximar o formato da árvore binária de busca do formato de uma árvore completa
- Uma árvore que seja tal aproximação é uma árvore balanceada.

Árvores AVL

- Árvores balanceadas
- Exemplos de árvores binárias de busca balanceadas
 - Árvores rubro-negras
 - Árvores AVL
- Nesta disciplina, estudaremos as árvores AVL

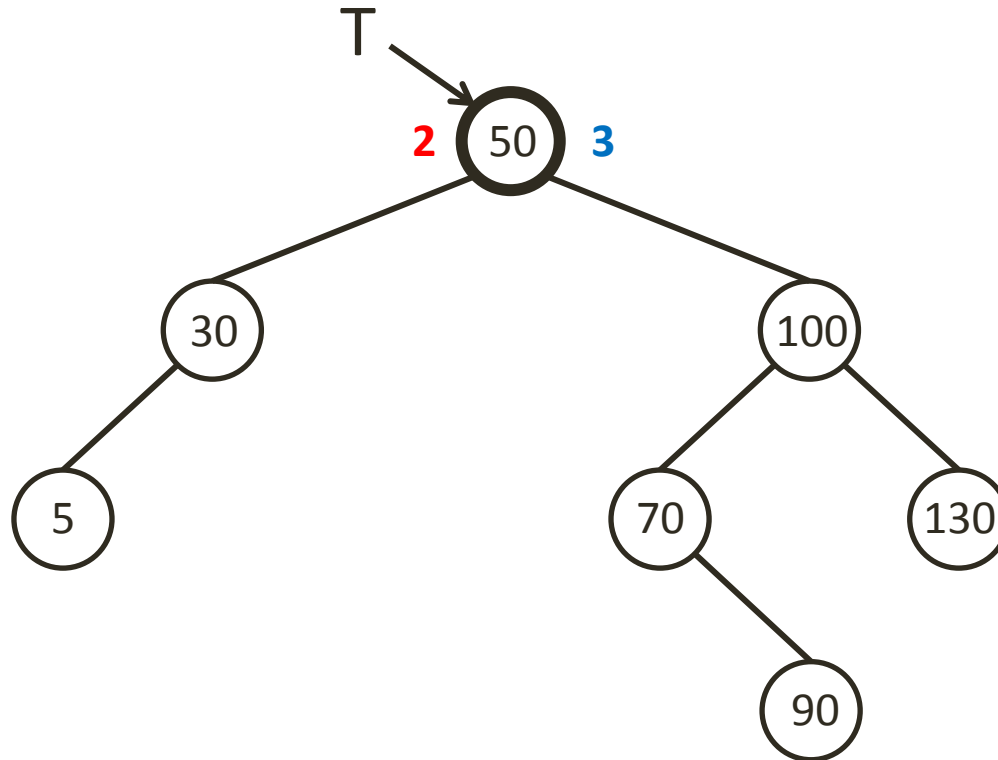
Árvores AVL

- Árvores AVL são árvores balanceadas onde, para todo nó P desta árvore, a diferença de altura entre as subárvores direita e esquerda de P é no máximo **1**.



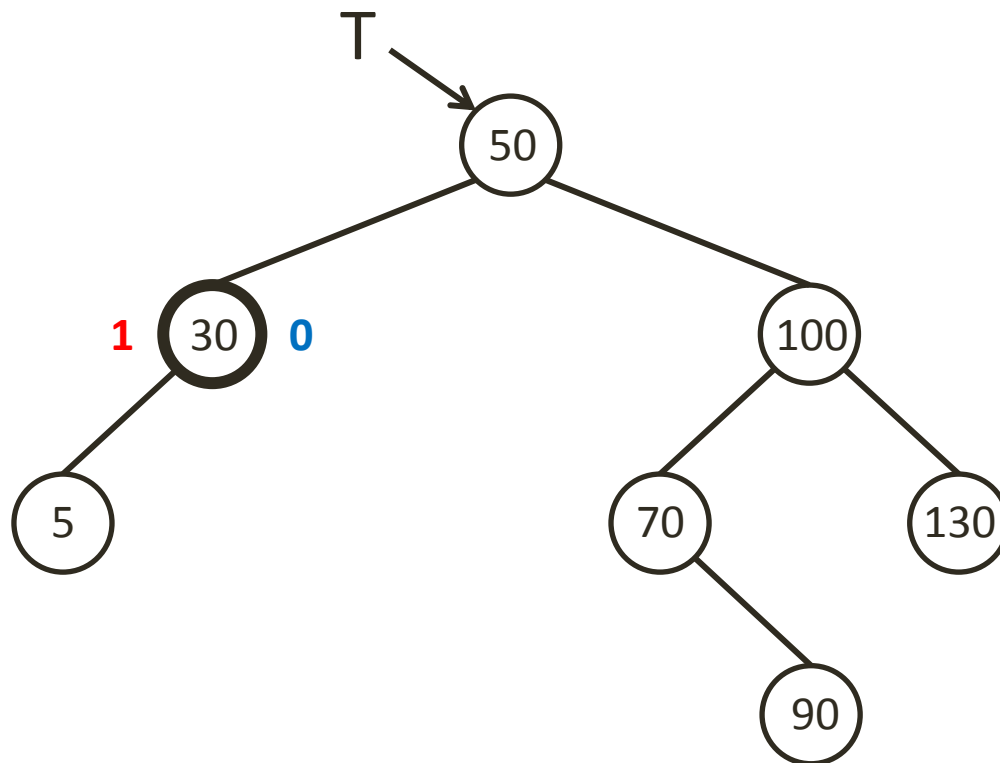
Árvores AVL

- Árvores AVL são árvores balanceadas onde, para todo nó P desta árvore, a diferença de altura entre as subárvores direita e esquerda de P é no máximo **1**.



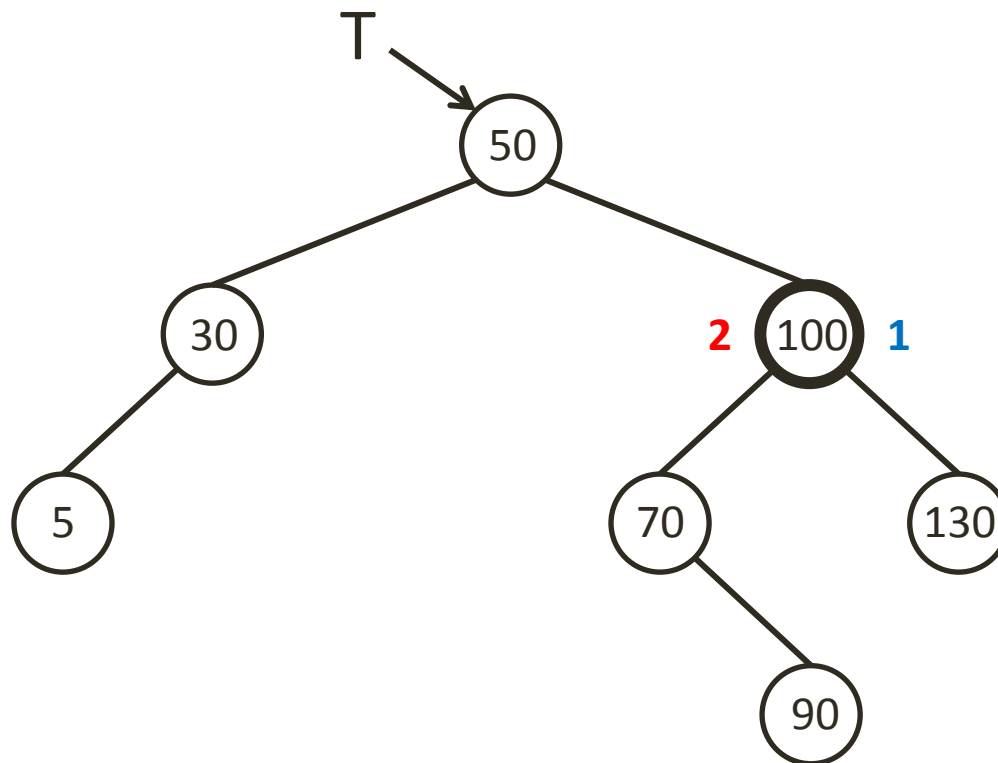
Árvores AVL

- Árvores AVL são árvores balanceadas onde, para todo nó P desta árvore, a diferença de altura entre as subárvores direita e esquerda de P é no máximo **1**.



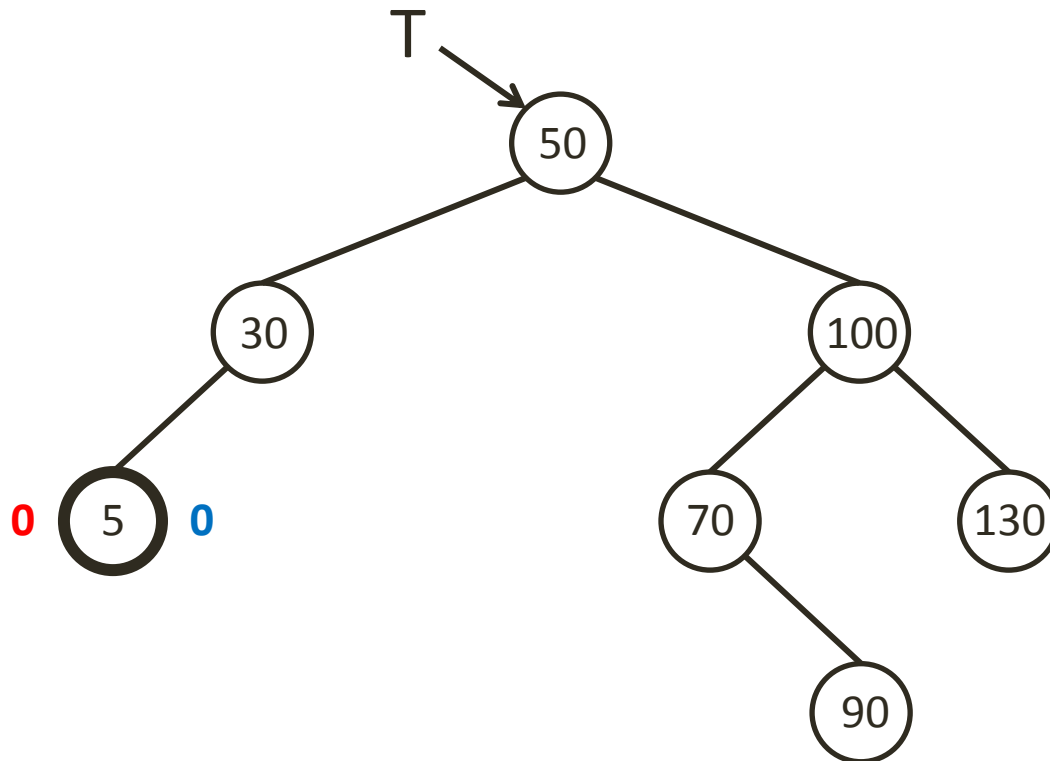
Árvores AVL

- Árvores AVL são árvores balanceadas onde, para todo nó P desta árvore, a diferença de altura entre as subárvores direita e esquerda de P é no máximo **1**.



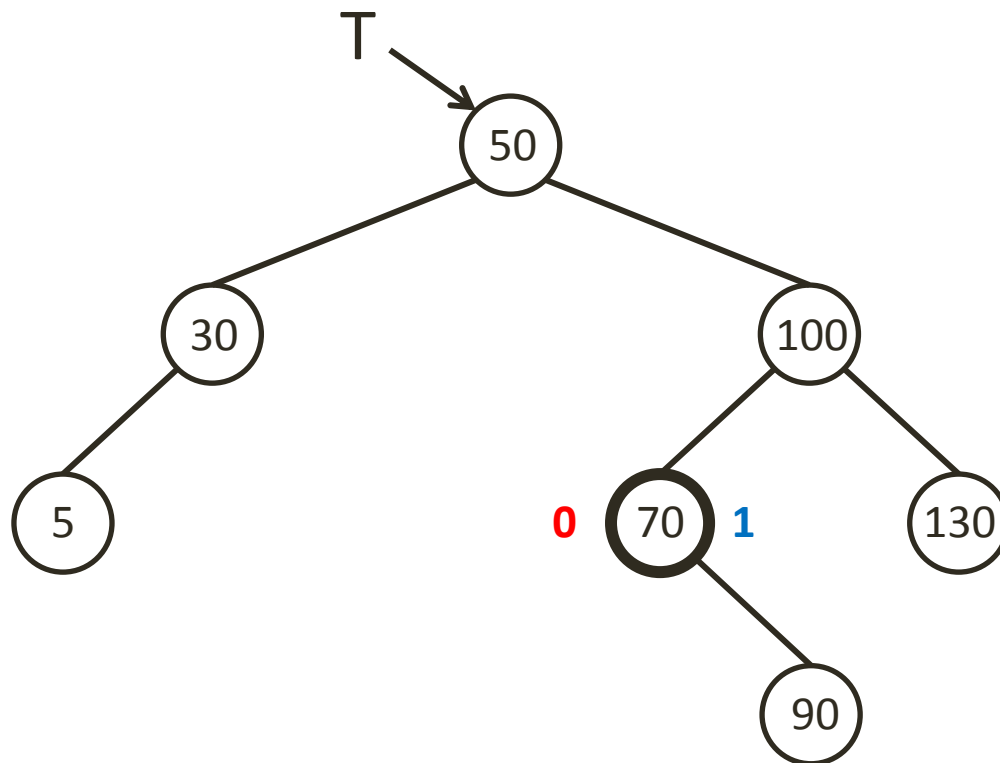
Árvores AVL

- Árvores AVL são árvores balanceadas onde, para todo nó P desta árvore, a diferença de altura entre as subárvores direita e esquerda de P é no máximo **1**.



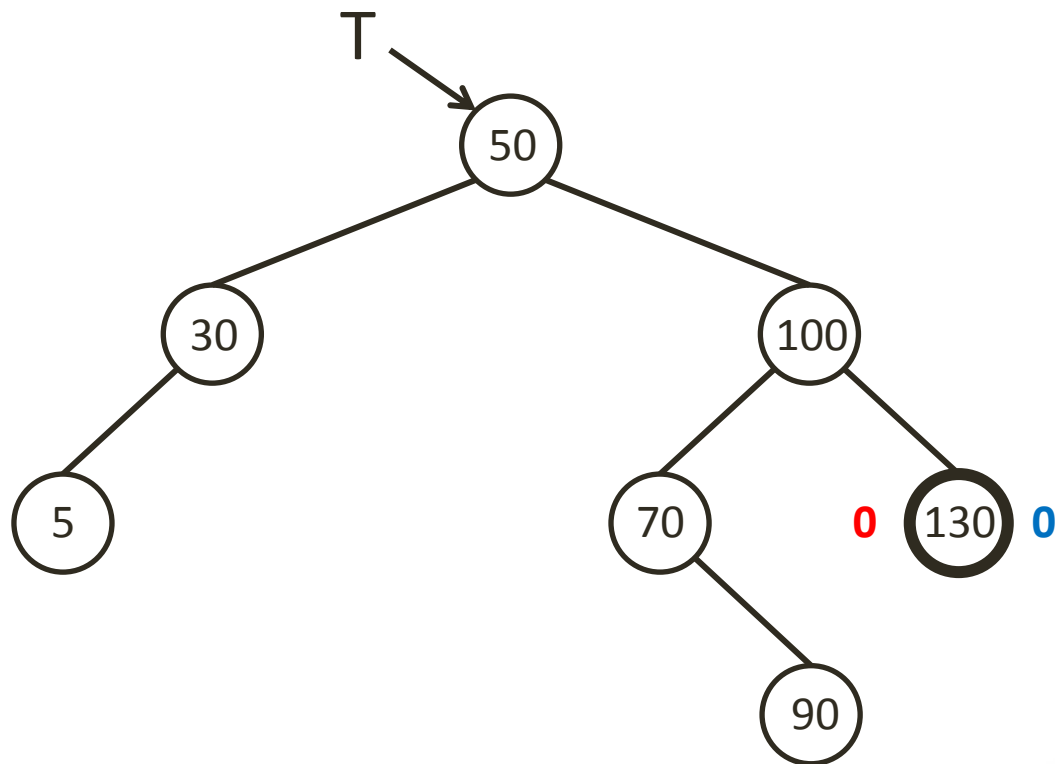
Árvores AVL

- Árvores AVL são árvores balanceadas onde, para todo nó P desta árvore, a diferença de altura entre as subárvores direita e esquerda de P é no máximo **1**.



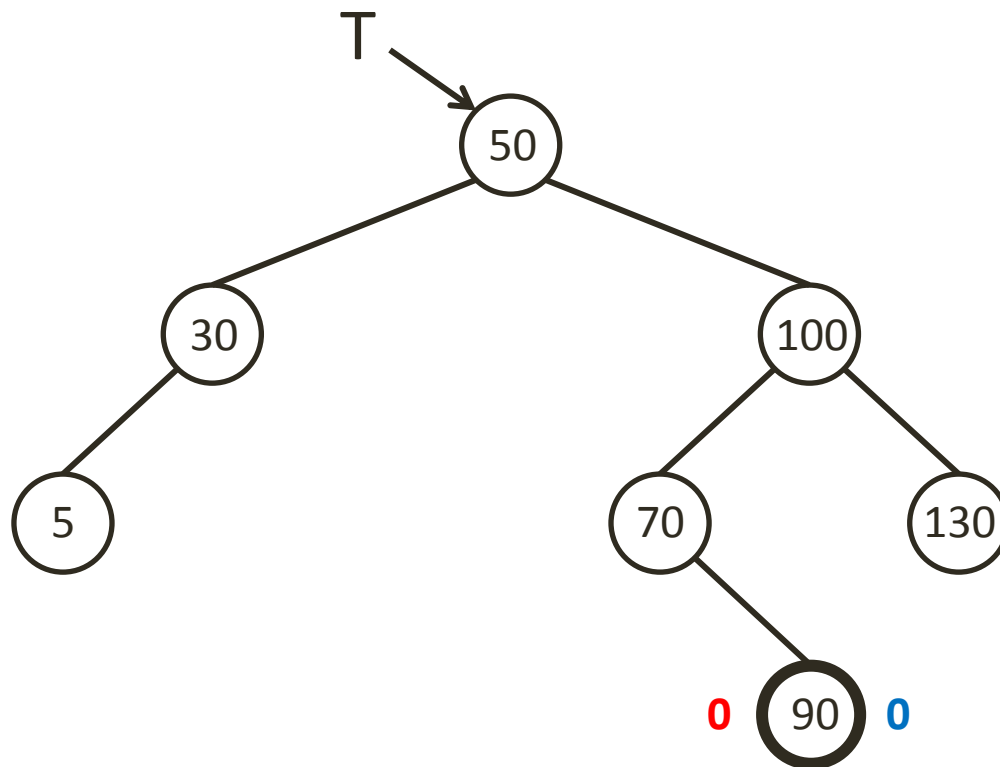
Árvores AVL

- Árvores AVL são árvores balanceadas onde, para todo nó P desta árvore, a diferença de altura entre as subárvores direita e esquerda de P é no máximo **1**.



Árvores AVL

- Árvores AVL são árvores balanceadas onde, para todo nó P desta árvore, a diferença de altura entre as subárvores direita e esquerda de P é no máximo **1**.



Árvores AVL

- Considere a função $h(T)$ que calcula a altura de uma árvore binária T da seguinte forma:

$$h(T) = \begin{cases} 0, & \text{se } T \text{ for uma árvore vazia,} \\ \text{altura}(T) + 1, & \text{caso contrário.} \end{cases}$$

- Uma árvore binária de busca T é uma árvore AVL se todo nó P satisfaz

$$| h(P^{\text{esq}}) - h(P^{\text{dir}}) | \leq 1$$

Árvores AVL

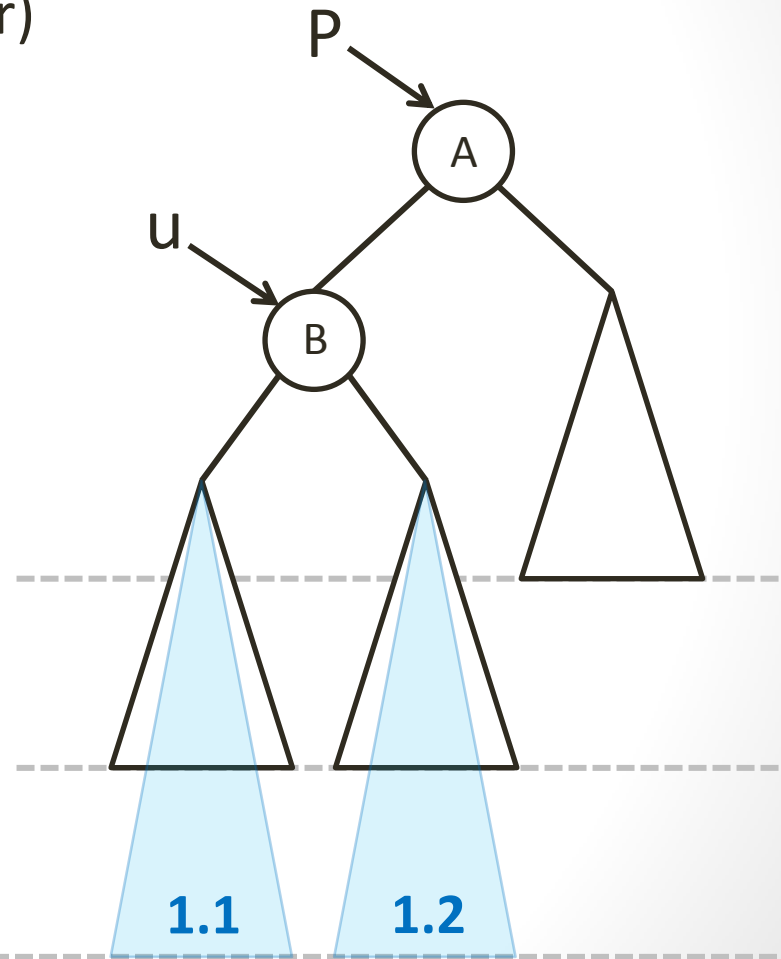
- Considere uma árvore AVL T . Considere uma operação de inserção ou remoção nesta árvore
- Suponha que T não é mais AVL **após** a operação realizada
- Considere que P é o nó desbalanceado mais próximo de onde ocorreu a operação
- Então, $|h(P^{.esq}) - h(P^{.dir})| = 2$

Árvores AVL

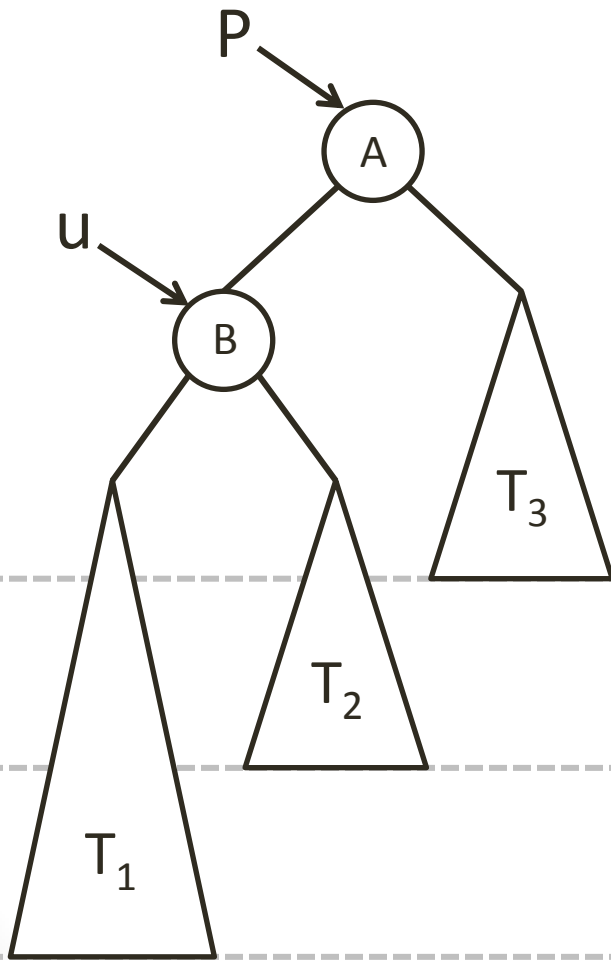
- Considere que P é o nó desbalanceado mais próximo de onde ocorreu a operação
 - Então, $|h(P^{.esq}) - h(P^{.dir})| = 2$
 - São dois casos que requerem balanceamento:
 - **CASO 1 :** $h(P^{.esq}) > h(P^{.dir})$
 - **CASO 2 :** $h(P^{.esq}) < h(P^{.dir})$
- (os dois casos são simétricos)

Árvores AVL

- **CASO 1 :** $h(P^{.esq}) > h(P^{.dir})$
- P tem um filho esquerdo u
- O nó u está balanceado
- $h(u^{.esq}) \neq h(u^{.dir})$
- Dois subcasos:
- **Caso 1.1 :** $h(u^{.esq}) > h(u^{.dir})$
- **Caso 1.2 :** $h(u^{.esq}) < h(u^{.dir})$



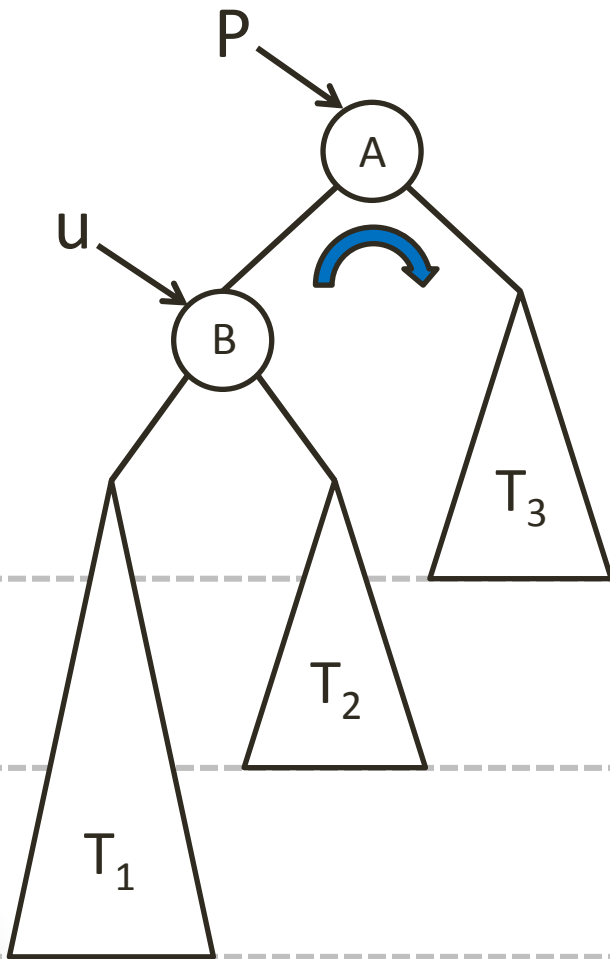
Árvores AVL



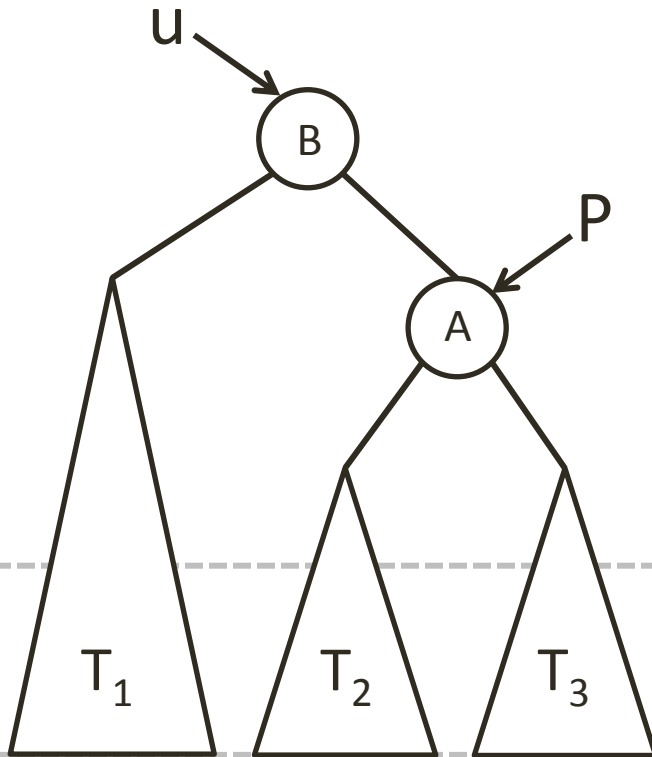
- **Caso 1.1 :**
 $h(u^{esq}) > h(u^{dir})$
- $h(T_1) - h(T_2) = 1$
- $h(T_2) = h(T_3)$

Árvores AVL

- Caso 1.1 :
rotação simples à direita

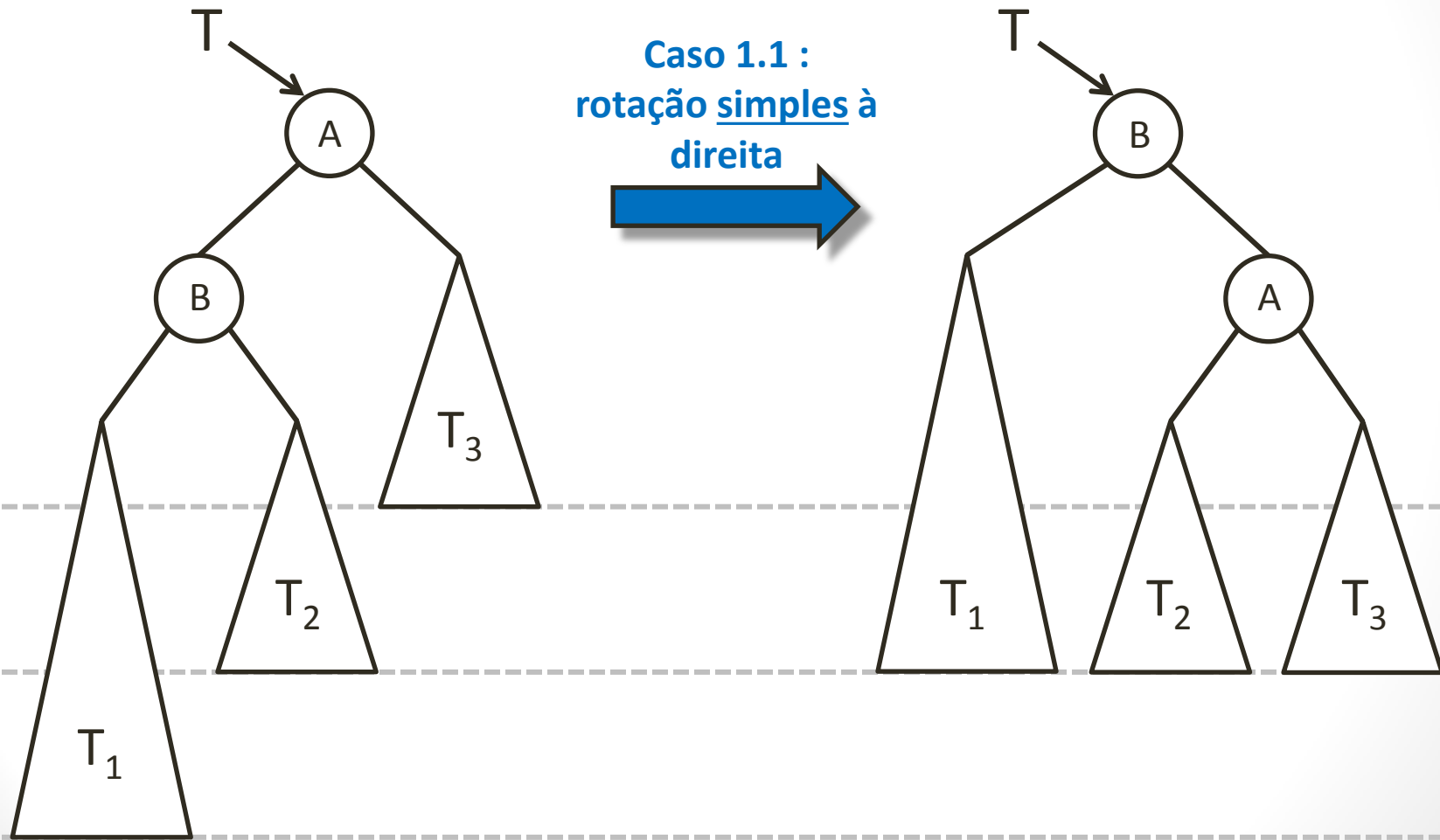


Árvores AVL



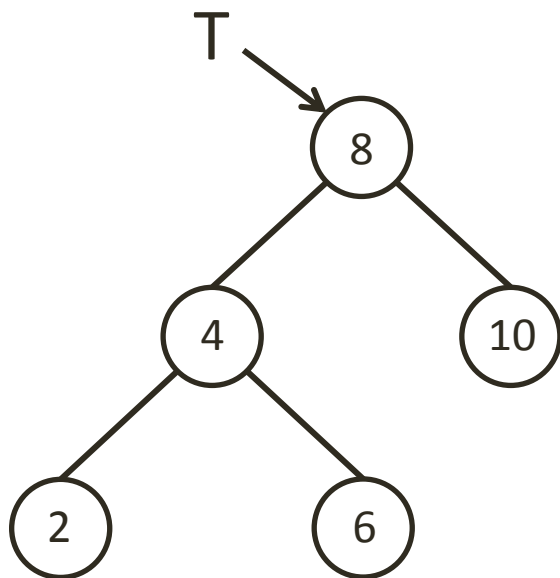
- **Caso 1.1 :**
rotação simples à direita
- P passa a ser filho direito de u
- A árvore T_2 passa a ser a subárvore esquerda de P

Árvores AVL

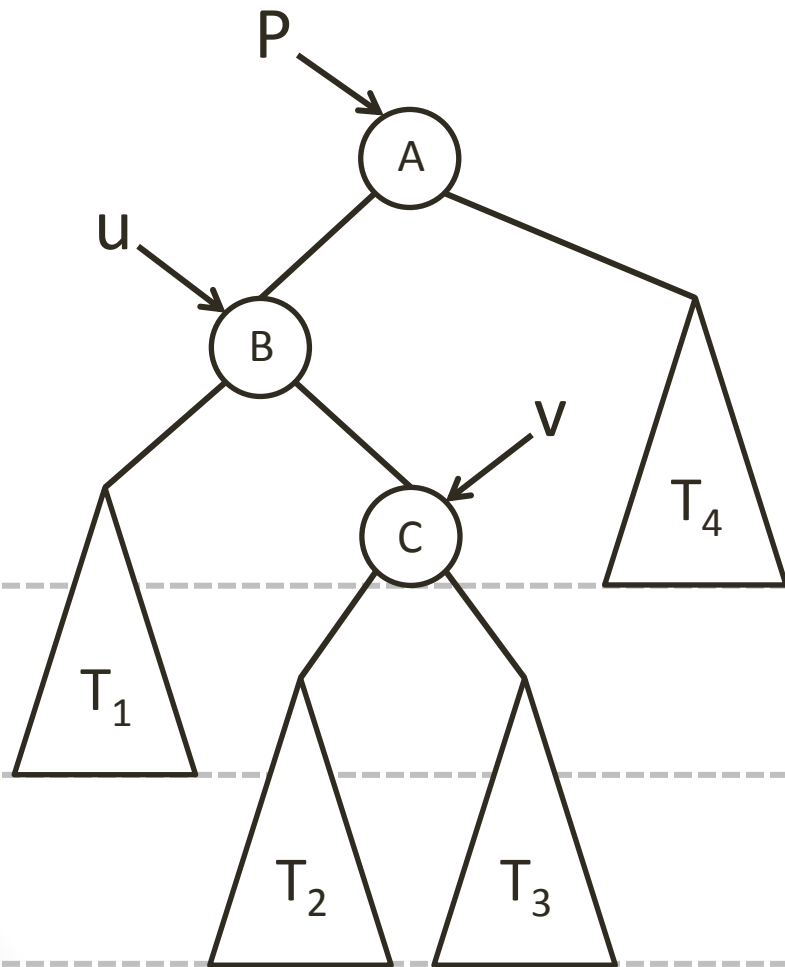


Árvores AVL

- **Caso 1.1 :**
- Exemplo : inserção da chave 1



Árvores AVL

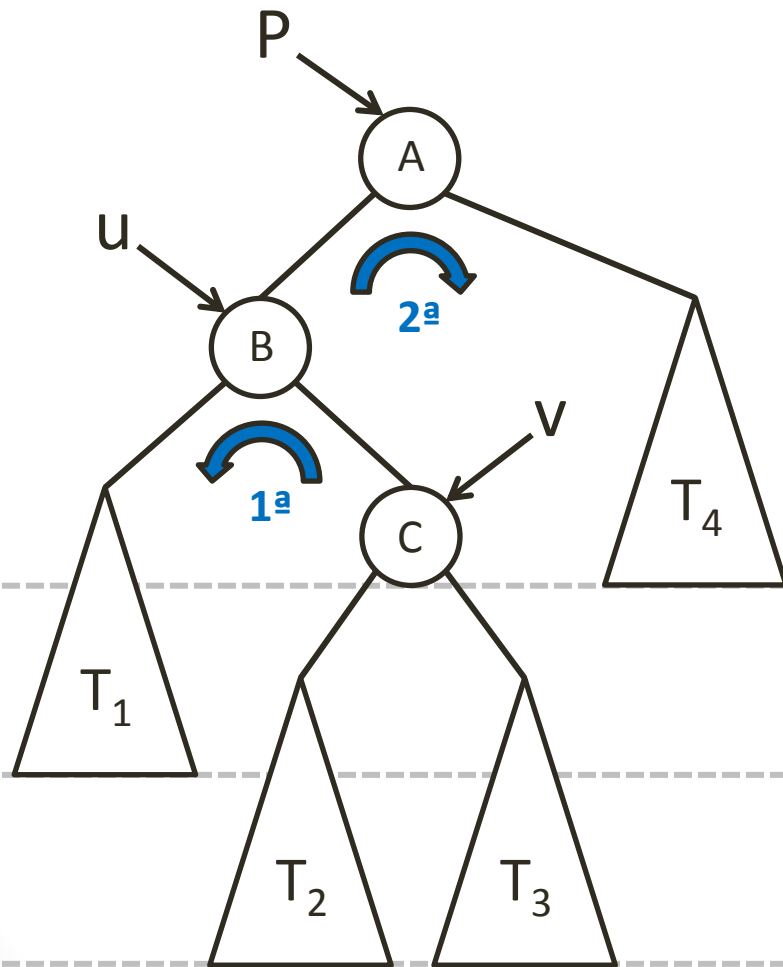


- **Caso 1.2 :**

$$h(u^{\wedge}.esq) < h(u^{\wedge}.dir)$$

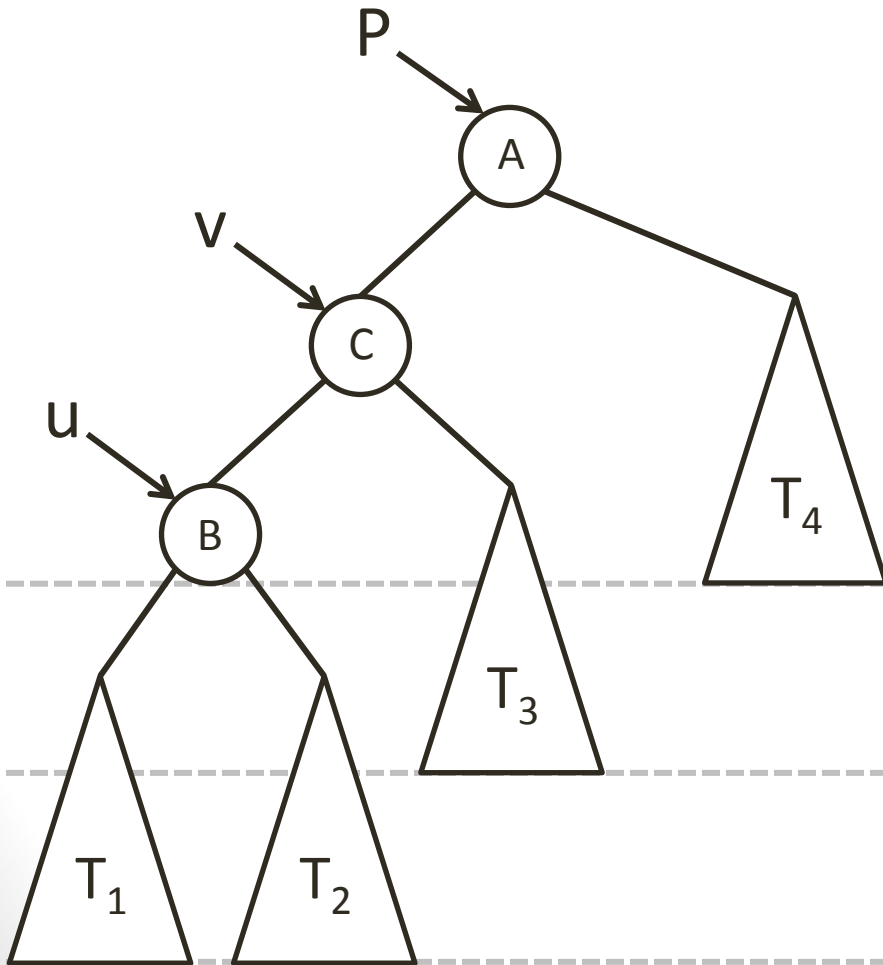
- u possui filho direito v
- $|h(T_2) - h(T_3)| \leq 1$
(v é balanceado)
- $\max\{h(T_2), h(T_3)\} = h(T_1) = h(T_4)$

Árvores AVL



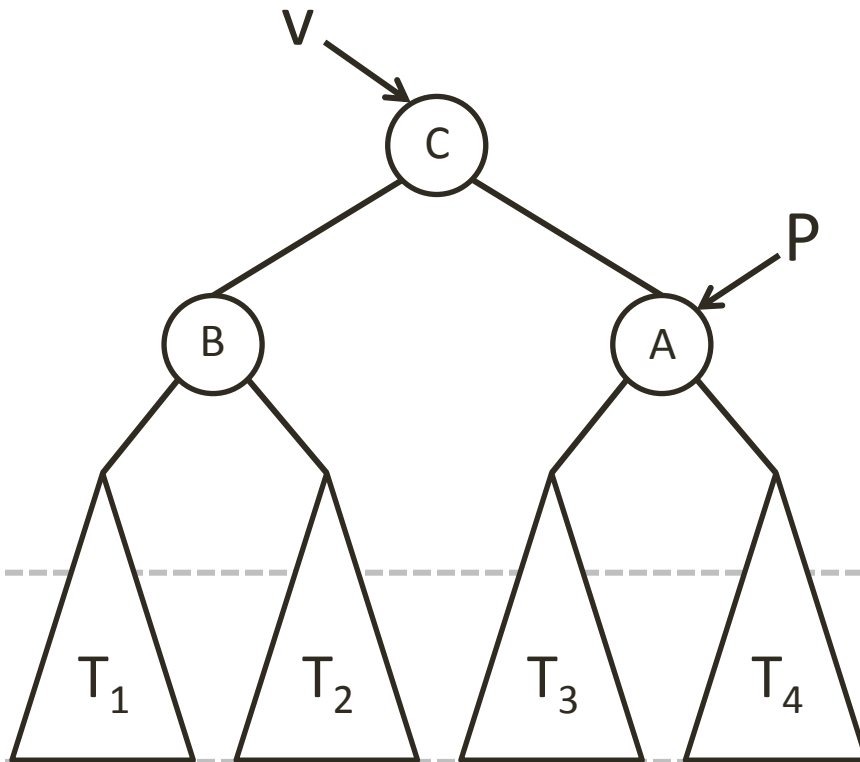
- Caso 1.2 :
rotação dupla à direita

Árvores AVL



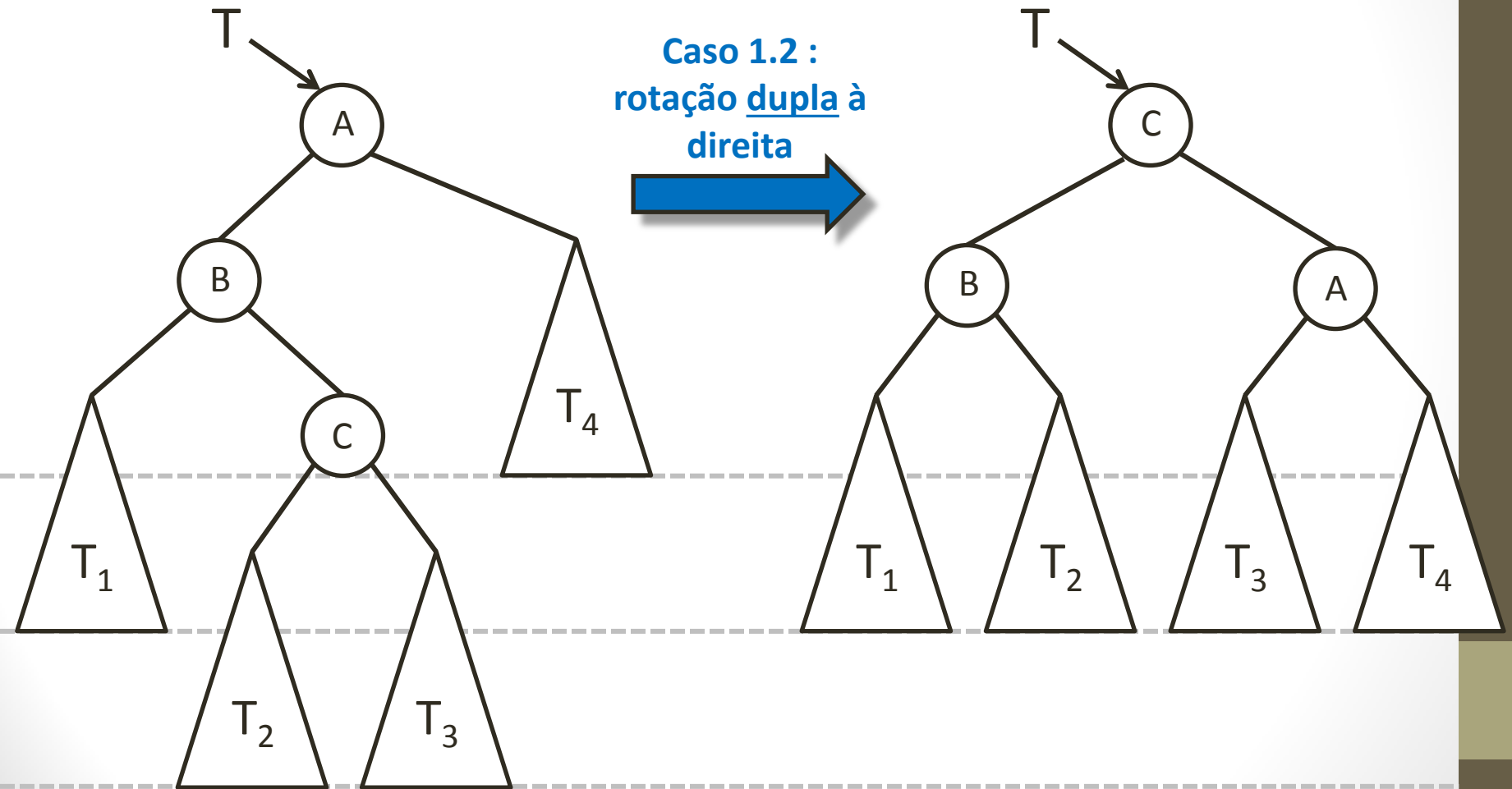
- **Caso 1.2 :**
rotação dupla à direita
- primeira rotação : u passa a ser filho esquerdo de v
- A árvore T₂ passa a ser a subárvore direita de u

Árvores AVL



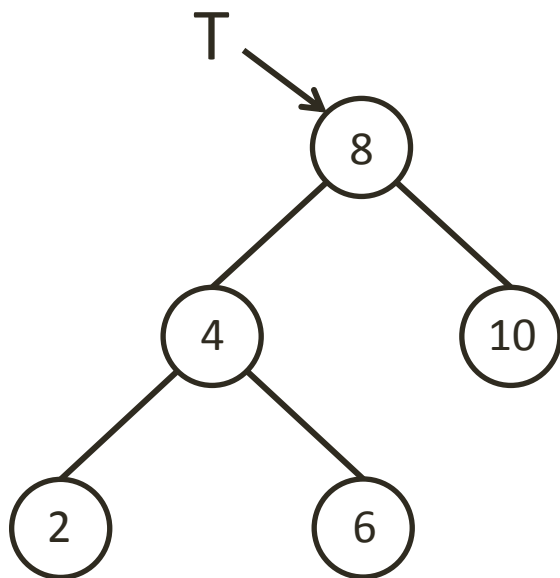
- **Caso 1.2 :**
rotação dupla à direita
- segunda rotação : P passa a ser filho direito de v
- A árvore T_3 passa a ser a subárvore esquerda de P

Árvores AVL



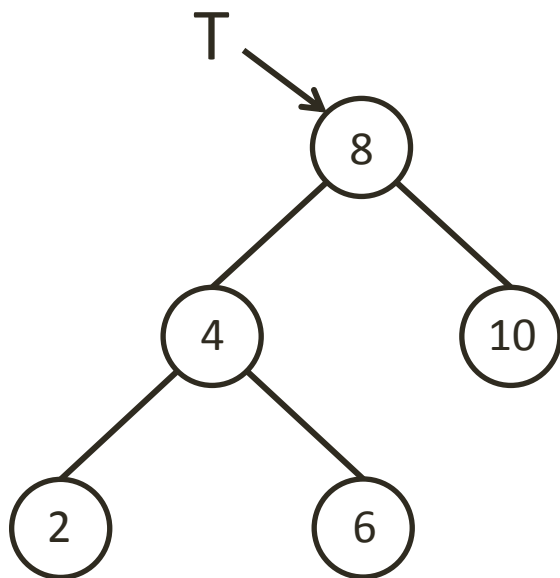
Árvores AVL

- **Caso 1.2 :**
- Exemplo : inserção da chave 5



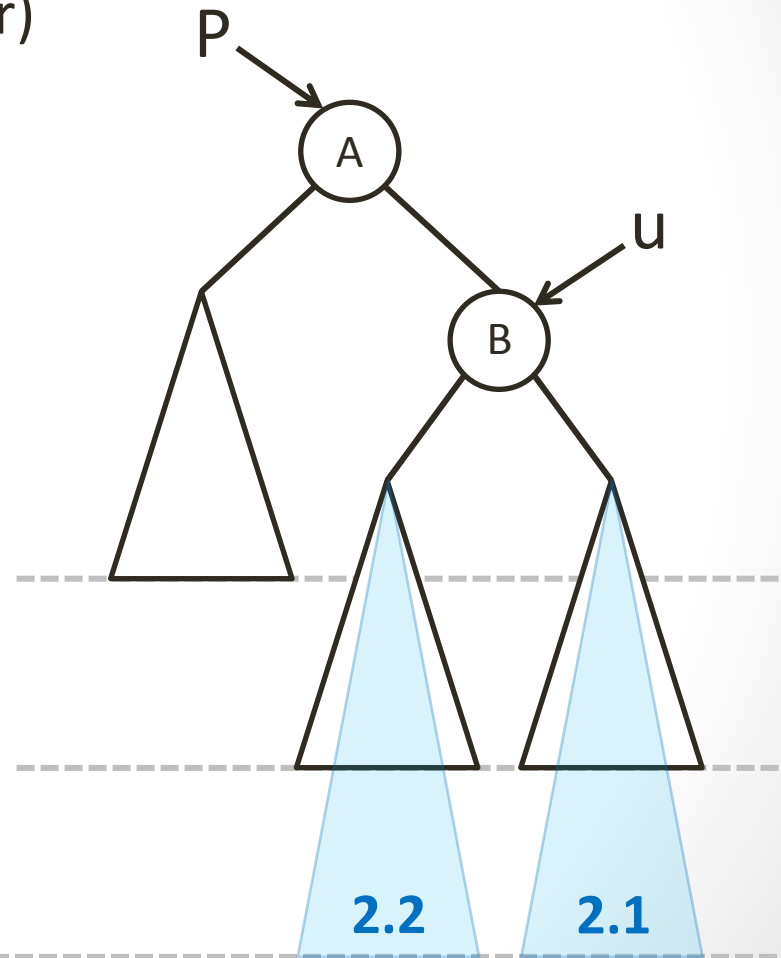
Árvores AVL

- **Caso 1.2 :**
- Exemplo : inserção da chave 7

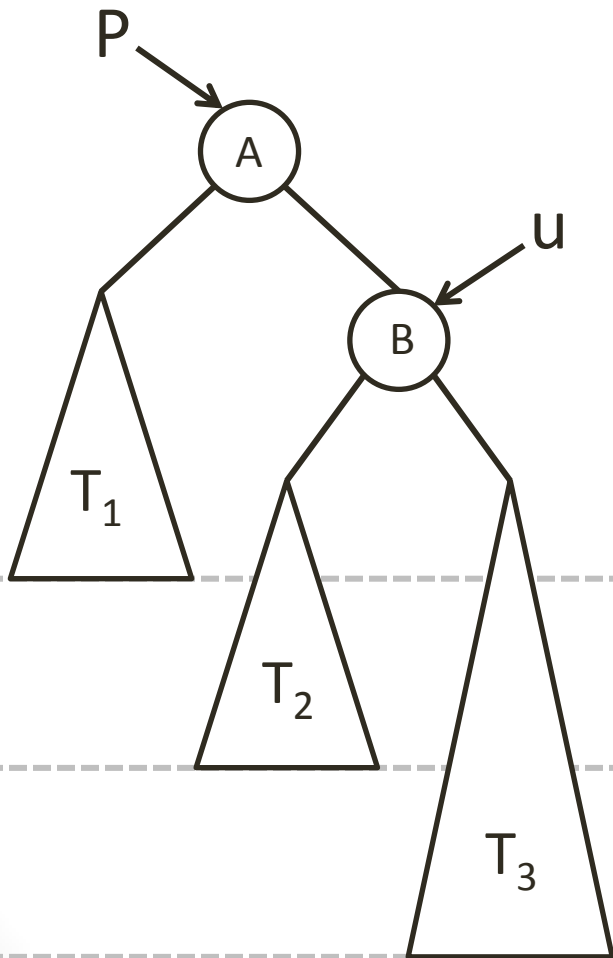


Árvores AVL

- **CASO 2 :** $h(P^{.esq}) < h(P^{.dir})$
- P tem um filho direito u
- O nó u está balanceado
- $h(u^{.esq}) \neq h(u^{.dir})$
- Dois subcasos:
- **Caso 2.1 :** $h(u^{.esq}) < h(u^{.dir})$
- **Caso 2.2 :** $h(u^{.esq}) > h(u^{.dir})$



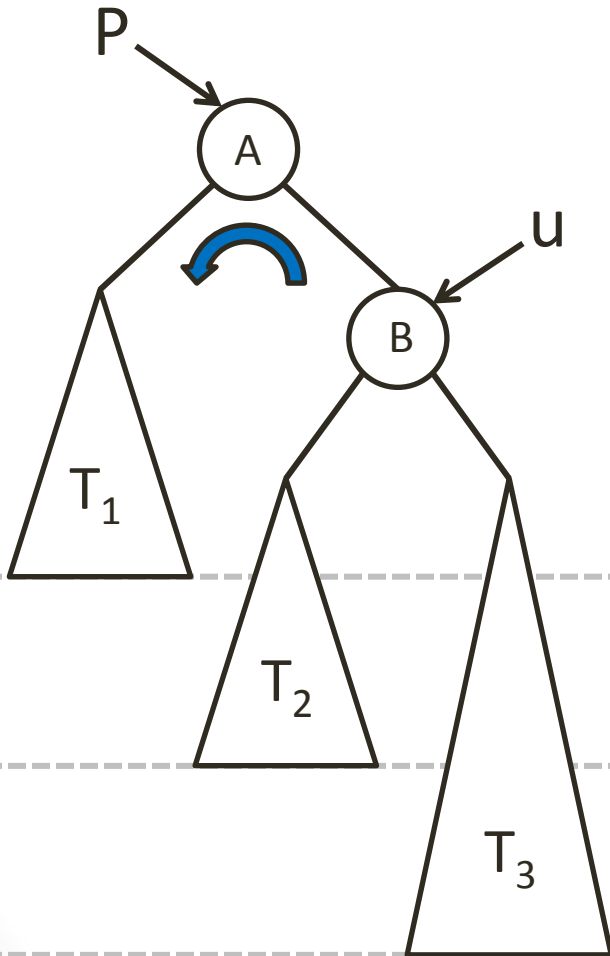
Árvores AVL



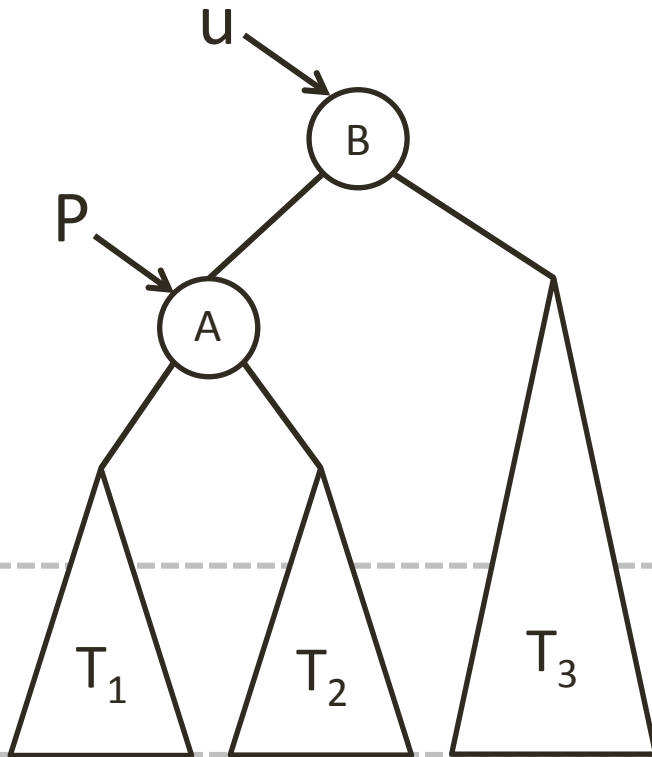
- **Caso 2.1 :**
 $h(u^{\wedge}.esq) < h(u^{\wedge}.dir)$
- $h(T_3) - h(T_2) = 1$
- $h(T_1) = h(T_2)$

Árvores AVL

- Caso 2.1 :
rotação simples à esquerda

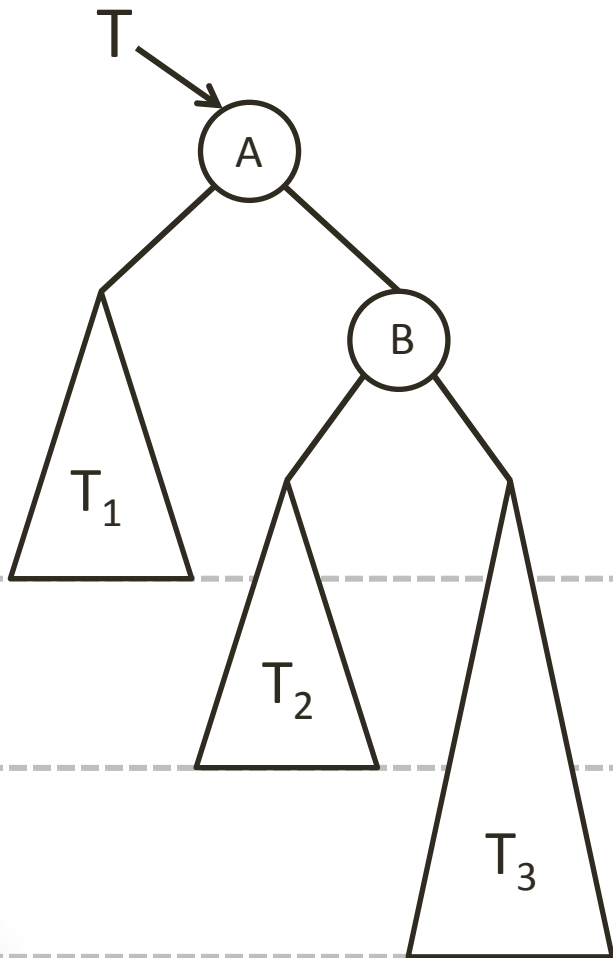


Árvores AVL

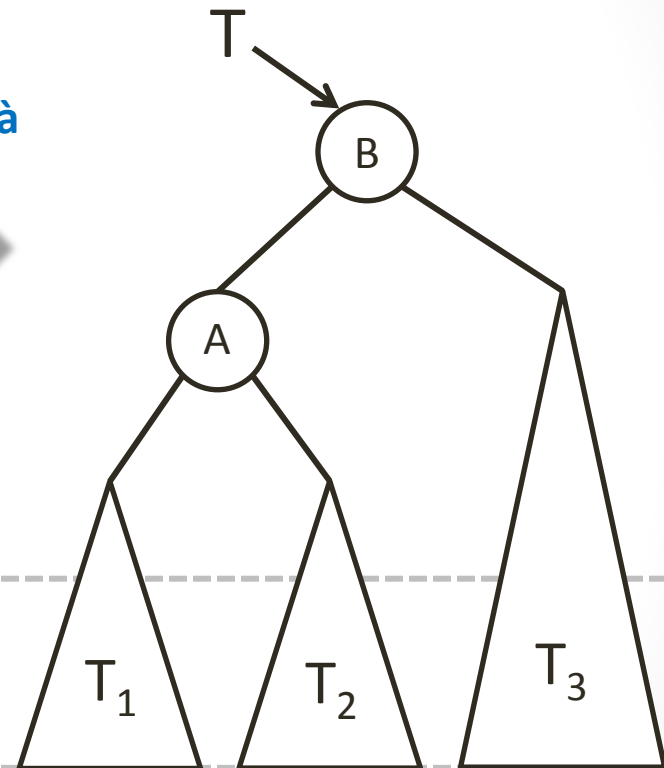


- **Caso 2.1 :**
rotação simples à esquerda
- P passa a ser filho esquerdo de u
- A árvore T₂ passa a ser a subárvore direita de P

Árvores AVL

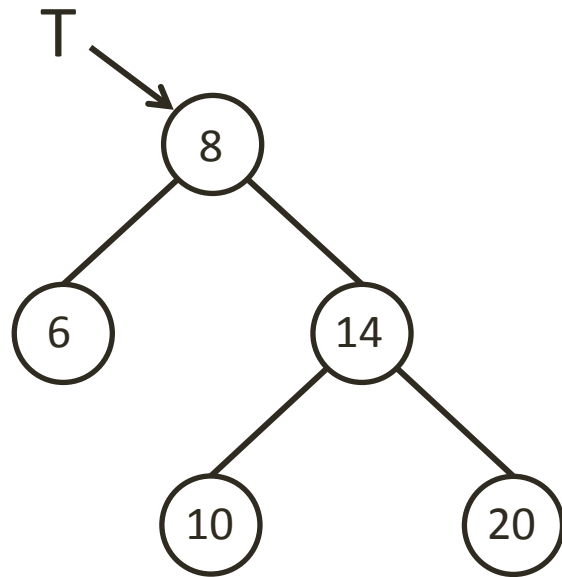


Caso 2.1 :
rotação simples à
esquerda

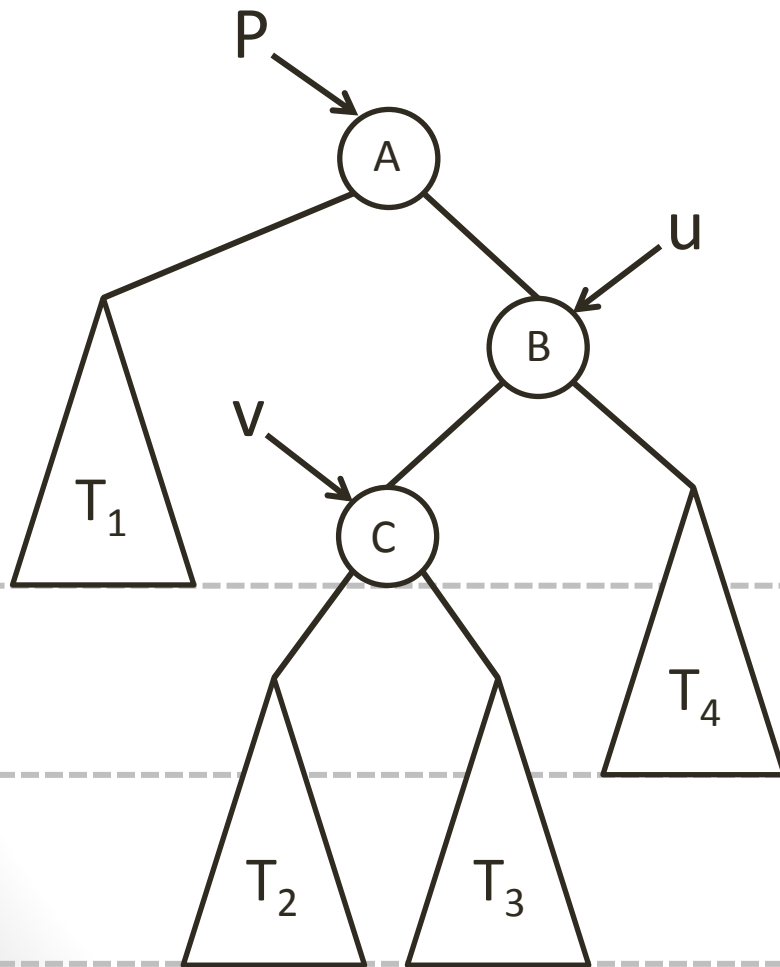


Árvores AVL

- **Caso 2.1 :**
- Exemplo : inserção da chave 30

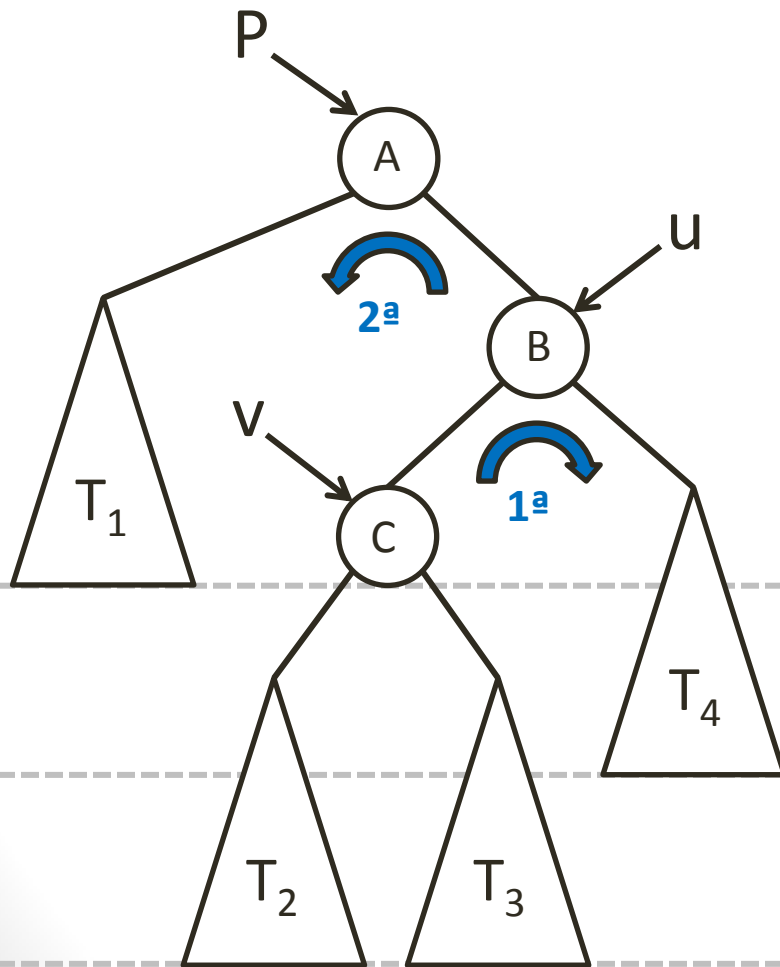


Árvores AVL



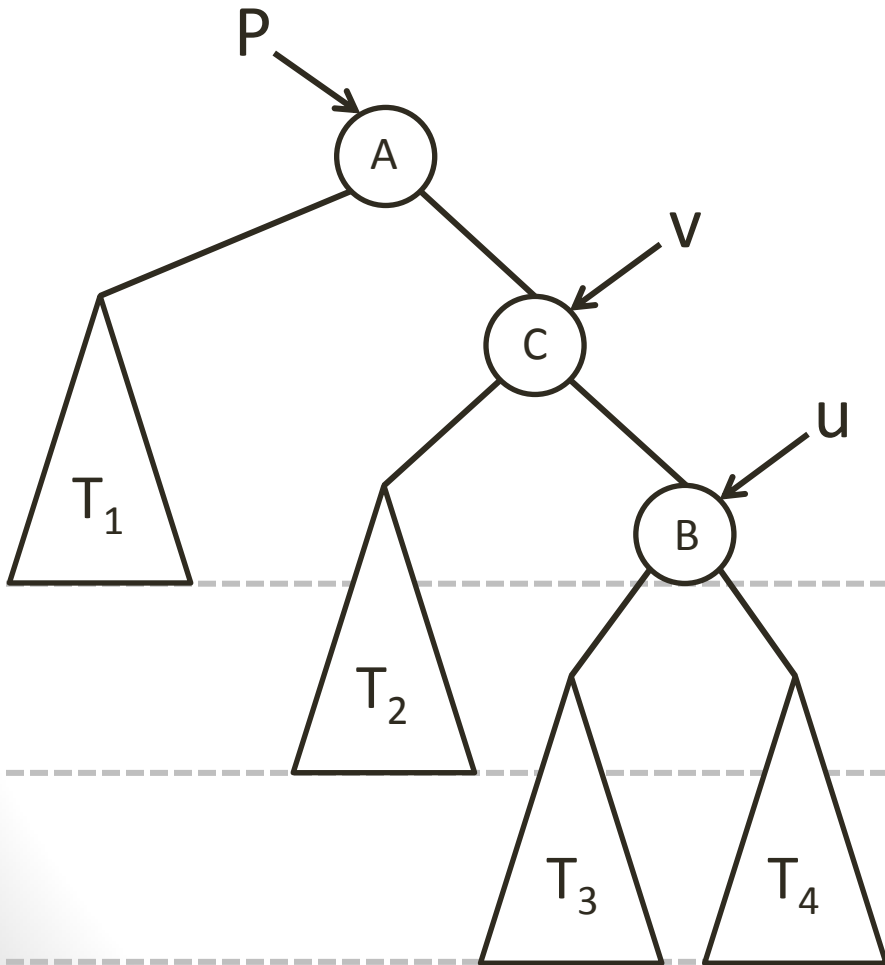
- **Caso 2.2 :**
 $h(u^{\wedge}.esq) > h(u^{\wedge}.dir)$
- u possui filho esquerdo v
- $| h(T_2) - h(T_3) | \leq 1$
(v é balanceado)
- $\max\{h(T_2), h(T_3)\} = h(T_1) = h(T_4)$

Árvores AVL



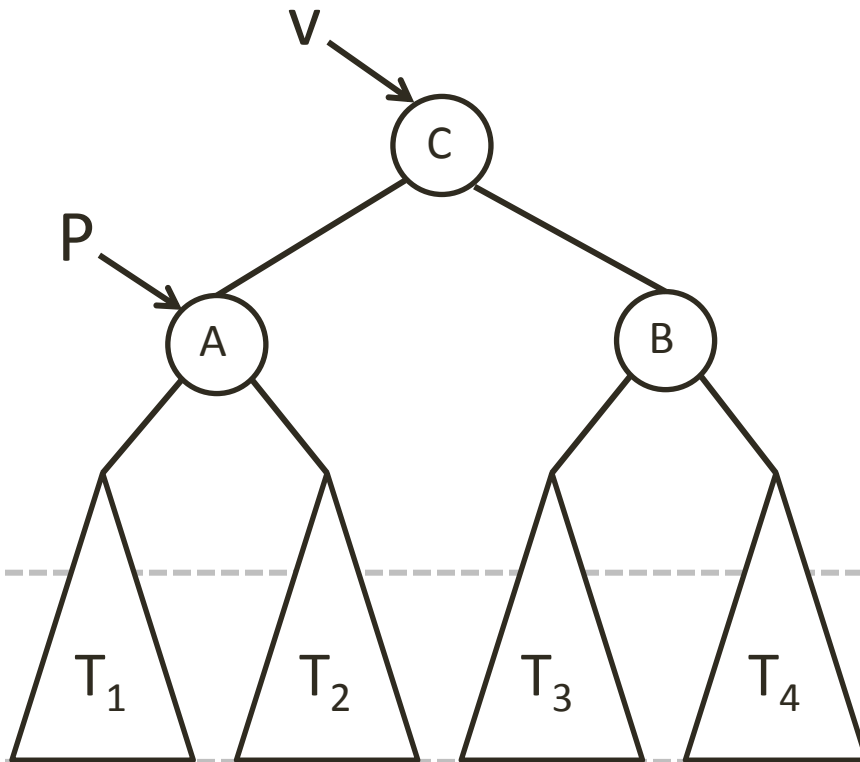
- Caso 2.2 :
rotação dupla à esquerda

Árvores AVL



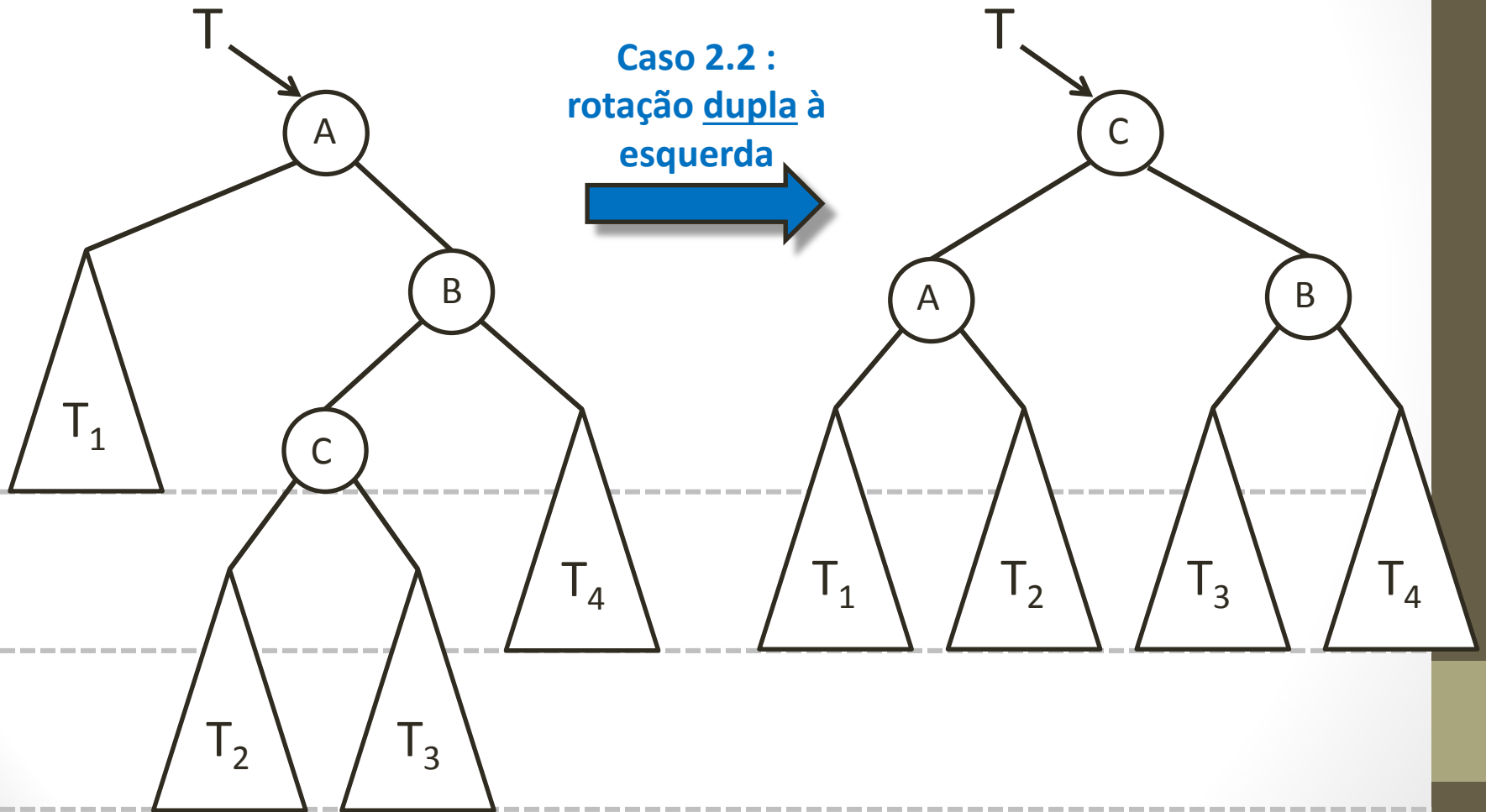
- **Caso 2.2 :**
rotação dupla à esquerda
- primeira rotação : u passa a ser filho direito de v
- A árvore T₃ passa a ser a subárvore esquerda de u

Árvores AVL



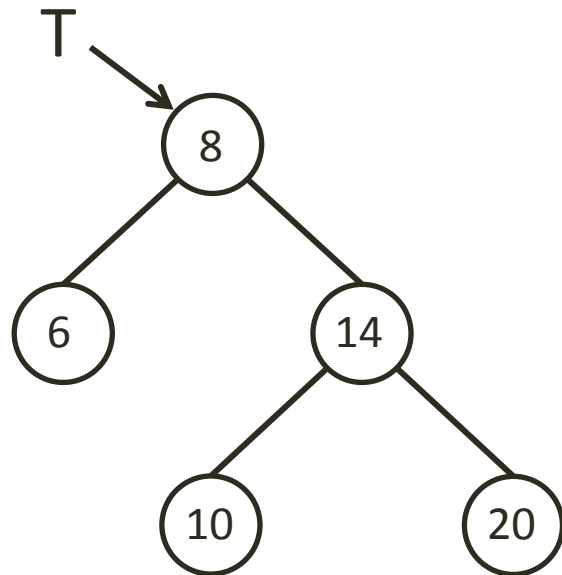
- **Caso 2.2 :**
rotação dupla à esquerda
- segunda rotação : P passa a ser filho esquerdo de v
- A árvore T_2 passa a ser a subárvore direita de P

Árvores AVL



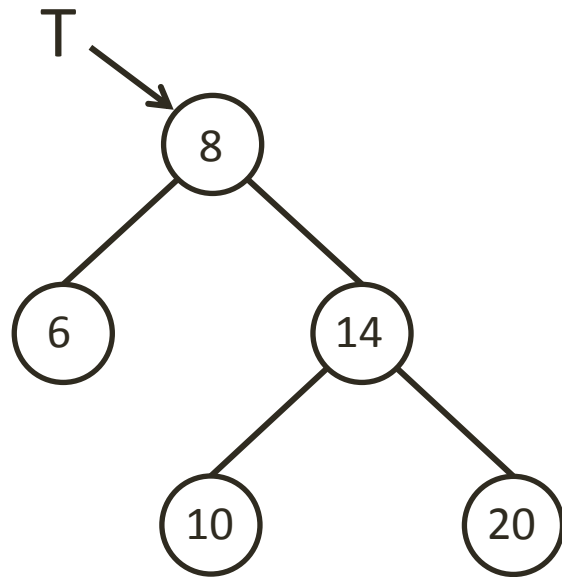
Árvores AVL

- **Caso 2.2 :**
- Exemplo : inserção da chave 12



Árvores AVL

- **Caso 2.2 :**
- Exemplo : inserção da chave 9



Árvores AVL

- Exercício: passo a passo, construa uma árvore AVL pela inserção da seguinte sequência de chaves:

4 5 7 2 1 3 6

Árvores AVL

- Algumas observações:
- Para fins de verificação de balanceamento de um nó P, pode-se acrescentar a ele um fator de balanceamento, que é a diferença

$$fb(P) = h(P^.esq) - h(P^.dir)$$

- Se $fb(P)$ for um valor entre -1 e 1, o nó P está balanceado
- Se $fb(P) = 2$, o nó P está desbalanceado (**CASO 1**)
- Se $fb(P) = -2$, o nó P está desbalanceado (**CASO 2**)

Árvores AVL

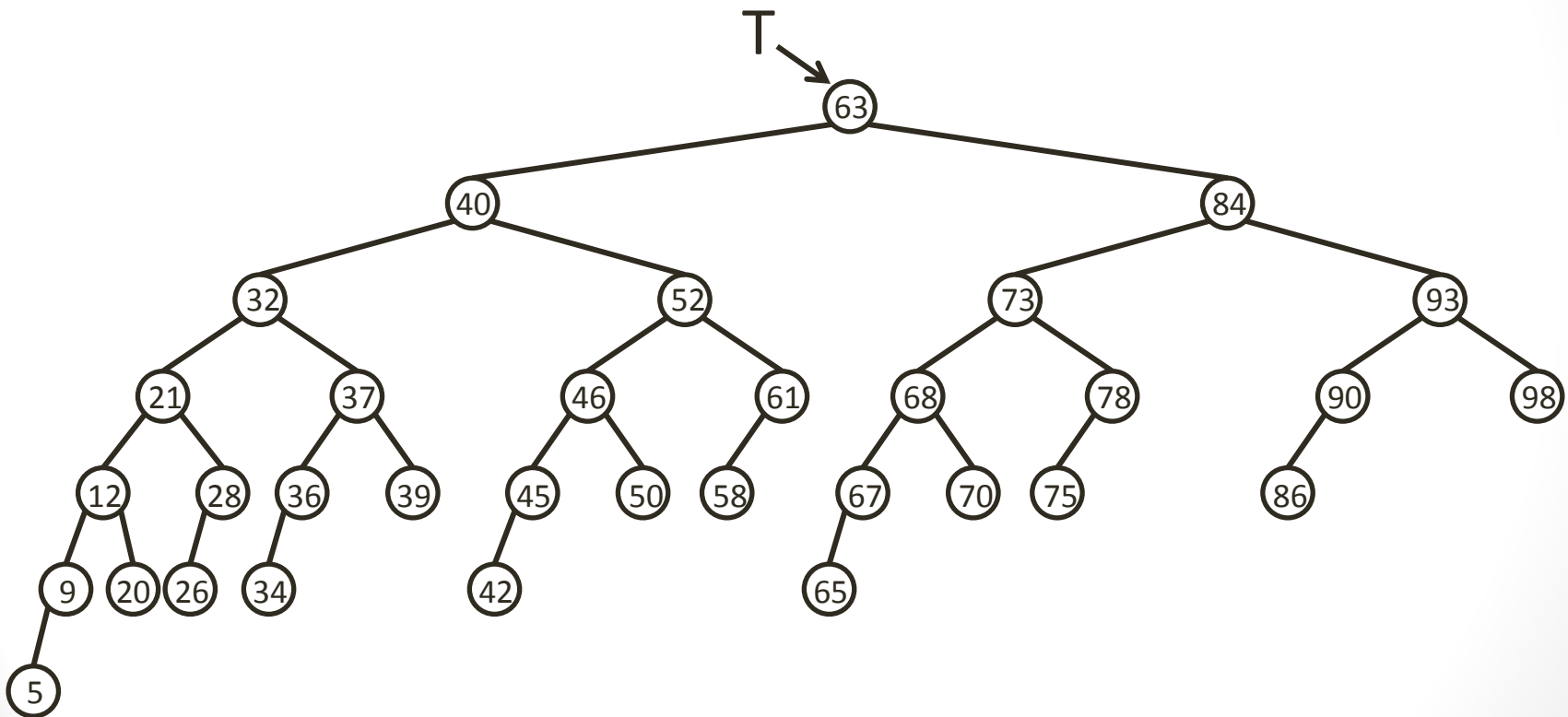
- Algumas observações:
- As operações de balanceamento, tanto as simples quanto as duplas, têm complexidade $O(1)$
- A inserção em uma árvore AVL tem complexidade $O(\log_2 n)$.
- Na inserção, se houver necessidade de balanceamento, uma única rotação (simples ou dupla) é suficiente para balancear a árvore.
- Neste caso, o ancestral mais próximo do nó inserido que estiver desbalanceado deve ser corrigido

Árvores AVL

- Algumas observações:
- As operações de balanceamento, tanto as simples quanto as duplas, têm complexidade $O(1)$
- A remoção em uma árvore AVL tem complexidade $O(\log_2 n)$.
- Na remoção, se houver necessidade de balanceamento, podem ser requeridas mais de uma rotação (simples ou dupla) para balancear a árvore.
- Percorre-se o caminho de volta até a raiz (a partir do pai do nó removido) corrigindo-se o balanceamento dos nós neste caminho quando necessário

Árvores AVL

- Árvore de Fibonacci:



Árvores AVL

- Algumas observações:
- As rotações apresentadas podem ser encontradas na literatura com outros nomes:
 - Left – Left (ou LL) : **Caso 1.1**
 - Left – Right (ou LR) : **Caso 1.2**
 - Right – Right (ou RR) : **Caso 2.1**
 - Right – Left (ou RL) : **Caso 2.2**

Exercícios

- Considere a representação de árvores binárias utilizando vetores (alocação estática)

tipo TpNóVetor = registro

início

info : tipo;

esq, dir : inteiro;

fim;

onde:

- info : campo de informação (pode ter um ou mais campos)
- esq : índice no vetor onde está o filho esquerdo deste nó
- dir : índice no vetor onde está o filho direito deste nó

Exercícios

- Considere a representação de árvores binárias utilizando vetores (alocação estática)

T : vetor[0 .. TamMax] de TpNóVetor;



- Implemente uma árvore binária de busca com a representação descrita, bem como as funções de gerenciamento desta estrutura : busca, inserção, remoção, mínimo e máximo.
- Implemente os passeios in-ordem, pré-ordem e pós-ordem

Exercícios

- Ainda falando sobre vetores:
- Pesquise sobre a estrutura de dados *heap* e entenda o seu funcionamento e sua aplicação
- (A estrutura de dados *heap* será utilizada na implementação do algoritmo de ordenação Heapsort)

Exercícios

- Faça uma função recursiva que receba uma árvore binária T e que retorne a altura desta árvore
- Faça uma função recursiva que receba uma árvore binária T e que retorne o número de nós desta árvore que sejam de grau 2
- Uma árvore binária é estritamente binária se todos os nós não-folha são de grau 2. Faça uma função recursiva que receba uma árvore binária T e que devolva VERDADEIRO se T for estritamente binária, e FALSO, caso contrário

Exercícios

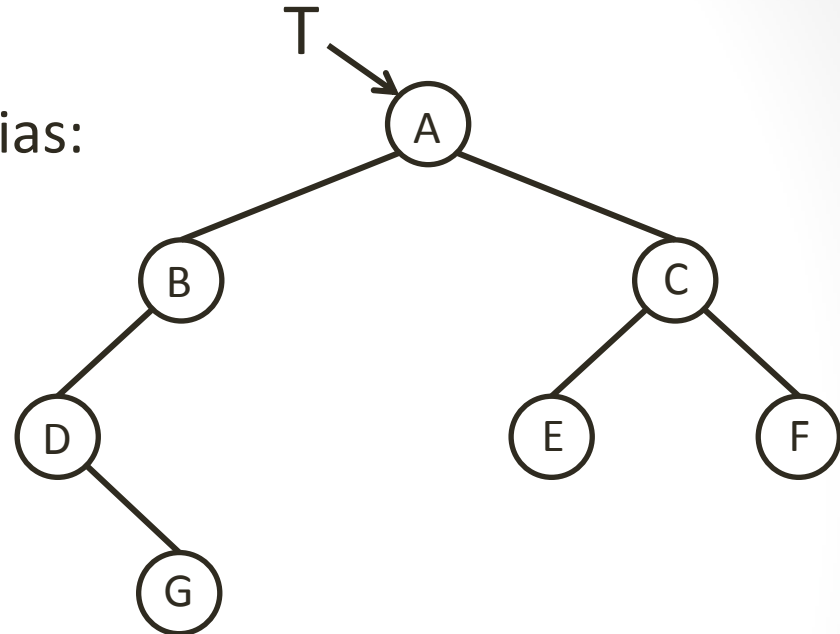
- Faça uma função recursiva que receba uma árvore binária T1 e que devolva uma árvore binária T2 que seja uma cópia espelhada de T1
- Faça uma função recursiva que receba duas árvores binárias T1 e T2 e que devolva VERDADEIRO se as duas árvores são idênticas, e FALSO, caso contrário
- Faça uma função recursiva que receba duas árvores binárias T1 e T2 e que devolva VERDADEIRO se T2 é uma subárvore de T1, e FALSO, caso contrário (dica: utilize o exercício anterior como parte da solução deste exercício)

Exercícios

- Faça uma função que receba uma árvore binária T qualquer e que devolva VERDADEIRO se T é uma árvore binária de busca, e FALSO, caso contrário
- Faça uma função que receba uma árvore binária T qualquer e que devolva VERDADEIRO se T é uma árvore AVL, e FALSO, caso contrário
- Descreva a execução dos passeios in-ordem, pré-ordem e pós-ordem de modo iterativo. Neste exercício, utilize uma pilha para gerenciar os passeios

Exercícios

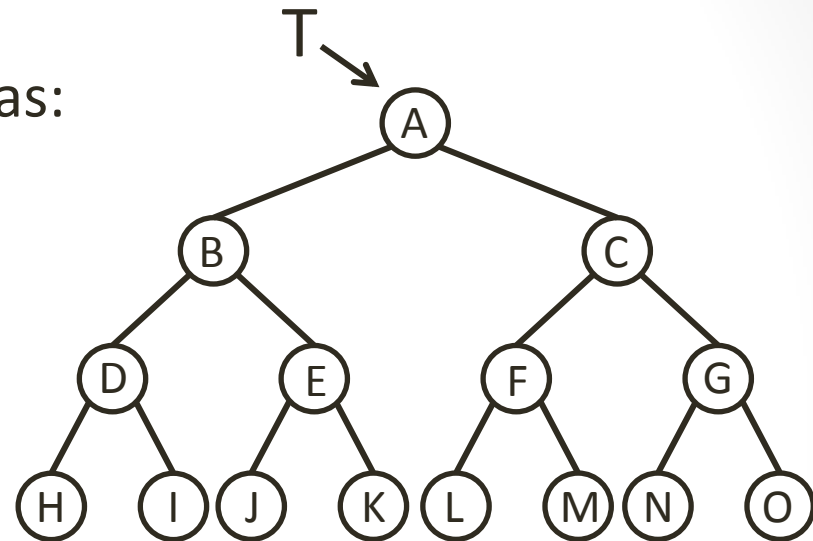
- Percursos em árvores binárias:



- Considerando esta árvore binária, escreva a ordem dos nós visitados nos seguintes percursos:
 - In-ordem :
 - Pré-ordem :
 - Pós-ordem :

Exercícios

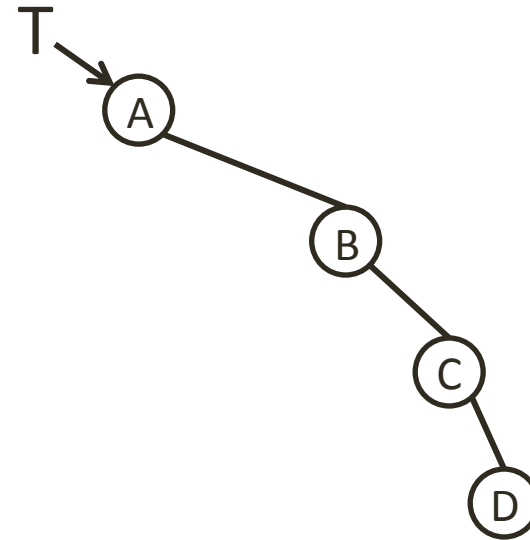
- Percursos em árvores binárias:



- Considerando esta árvore binária, escreva a ordem dos nós visitados nos seguintes percursos:
 - In-ordem :
 - Pré-ordem :
 - Pós-ordem :

Exercícios

- Percursos em árvores binárias:



- Considerando esta árvore binária, escreva a ordem dos nós visitados nos seguintes percursos:
 - In-ordem :
 - Pré-ordem :
 - Pós-ordem :

Exercícios

- Considere as seguintes sequências de elementos.
 - 15, 6, 2, 8, 20, 16, 7, 4, 10, 5
 - 35, 28, 5, 39, 52, 77, 78, 2, 42, 15
 - 16, 27, 10, 14, 11, 13, 29, 23, 25, 24
 - 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
-
- Construa, **passo-a-passo**, uma árvore binária de busca para cada uma das sequências
 - Os elementos devem ser inseridos na ordem da sequência

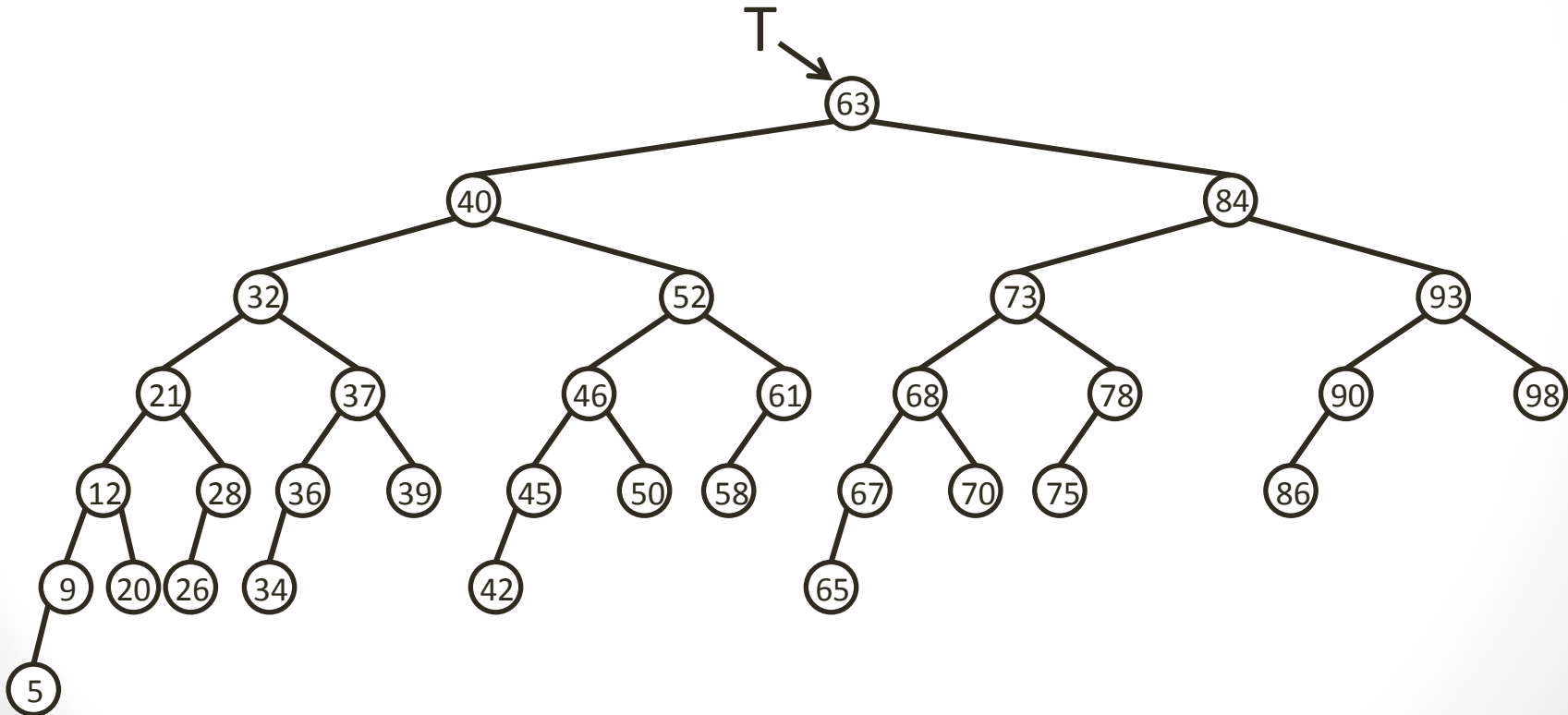
Exercícios

- Considere as seguintes sequências de elementos.
- 15, 6, 2, 8, 20, 16, 7, 4, 10, 5
- 35, 28, 5, 39, 52, 77, 78, 2, 42, 15
- 16, 27, 10, 14, 11, 13, 29, 23, 25, 24
- 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

- Construa, **passo-a-passo**, uma árvore AVL para cada uma das sequências
- Os elementos devem ser inseridos na ordem da sequência
- (Descreva, quando necessário, as rotações utilizadas. Rotações duplas devem ser descritas passo-a-passo)

Exercícios

- Considere a árvore AVL abaixo. Escreva a árvore AVL resultante da remoção do 98.



Bibliografia

- ZIVIANI, N.; *Projeto de Algoritmos com Implementações em Pascal e C*, 3ª Edição, Livraria Pioneira Editora, 1996.
- CORMEN, T. H. et al – *Algoritmos* , Trad. Segunda Edição Americana. Editora Campus, 2002.
- WIRTH, N. – *Algoritmos e Estruturas de Dados*. Editora Prentice-Hall, 1989.